第1章 Linux 操作系统概述

本章学习目标

知识目标
- 了解Linux操作系统起源、发展过程
- 掌握Linux操作系统的特点
- 掌握Linux内核源代码的组织结构
- 掌握Linux内核分析方法

能力目标
- 能够认识Linux操作系统版本的含义
- 能够分析与其他产品的区别

素质目标
- 增强思考观察能力
- 强化分析比较能力

第1章 Linux 操作系统概述

1. Linux的起源与发展
2. 自由软件GNU与GPL
3. Linux的特点
4. Linux系统及内核的组成
5. Linux版本的认识
6. Linux内核源代码的组织
7. Linux内核分析方法
8. 与其它几种操作系统的比较
9. Linux中的C语言及汇编语言

1、 Linux的起源与发展

- 1990年秋天，学生Linus在芬兰首都赫尔辛基大学学习操作系统课程。因为上机需要排队等待，Linus买了台PC机，开发了第一个程序。程序包括两个进程，向屏幕上写字母，然后用定时器来切换进程
- Linus需要终端仿真程序来存取Usenet新闻组的内容，于是他写了从调制解调器上收发信息的程序以及显示器、键盘和调制解调器的驱动程序
- 然后,写了磁盘驱动程序，文件系统
- 一旦有了进程切换、文件系统和设备驱动程序，就拥有了一个操作系统原型，或者至少是它的一个内核
- Linux就以这样极其古怪但也极其自然式问世

1、 Linux的起源与发展

Linux之父-Linus Tovalds

- 芬兰赫尔辛基大学:1990
- Linux是一匹自由而奔放的黑马
  - 诞生于学生之手
  - 成长于Internet
  - 壮大于自由而开放的文化

1、 Linux的起源与发展

Linux内核编年史
- 1990年：Linux 0.0.1
- 1991年：Linux 0.0.2，bash、gcc
- 1993年底：Linux 1.0版诞生
- 1994年：Linux 的第一个商业发行版 Slackware 问世
- 1996年：美国国家标准技术局的计算机系统实验室确认Linux 版本 1.2.13（由 Open Linux 公司打包），符合 POSIX 标准
- 1999年：Linux 2.2发布，64位平台，SMP，NTFS，SCSI，多媒体
- 2001年：Linux 2.4版内核发布
- 2003年：Linux 2.6版内核发布
- ……
- 2020年12月：Linux 5.10版内核发布

……
- 2022年10月： Linux6.0.10版内核发布

2、自由软件与GPL

- 自由软件：指允许自由地使用、复制、修改、分发的软件，软件的源代码必须是可得到的。
  - GNU/Linux、freeBSD
  - 电子邮件服务器 sendmail
  - 服务器 Apache
- 自由软件基金会FSF（Free Software Foundation）
- 自由软件之父： Richard Stallman
- Linux内核的开发工作都是基于GNU推出的自由软件完成的。GNU和Linux相辅相承。

2、自由软件与GPL

- POSIX
  - UNIX发展过程中，出现了许多变种（BSD，System V等）。
  - 1986年，IEEE的一个委员会指定了一套“基于Unix的可移植操作系统”的标准，命名为POSIX标准（Portable Operating System Interface Standard）
  - POSIX定义了一套标准的操作系统接口和工具，并由ISO接受为国际标准。几乎所有的正式操作系统都以支持POSIX标准为目标。易移植。

2、自由软件与GPL

◆GNU项目
- 开始于1984年， 由Richard Stallman（自由软件之父）创立，主要由自由软件基金（FSF）资助的一个项目。
- 目标：是开发一个完全自由的、UNIX类型的操作系统，称为GNU系统。与此相应的有一份通用公共许可证GPL。
- GNU软件的开发不是为了经济目的，而是不断开发并传播新的软件，每个人都能自由获得。

2、自由软件与GPL

◆通用公共许可证GPL(General public License)
- 保证自由软件对所有用户都是自由的(不一定免费)
- 所有的GNU软件和派生工作均遵循GNU通用公共许可证，即GPL。http://www.gnu.org/licenses/licenses.html

◆GPL类软件遵循一定的规则
- 传播者不能限制购买软件的用户自由权。如果用户买了一套GPL软件，就可免费复制、传播或出售。
- 传播者必须清楚告诉用户该软件属于GPL软件。
- 传播者必须免费提供软件的完整源代码，且任何用户能以源代码形式将软件复制或发布给别的用户。
- 如果用户的软件使用了受GPL保护的任何软件的一部分，则该软件也成为GPL软件，它必须随应用程序一起发布源代码。

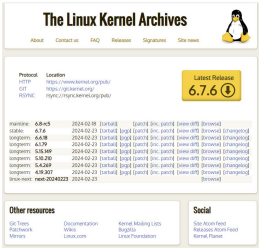
2、自由软件与GPL

- GNU/Linux
  - Linux:内核
  - GNU/Linux：操作系统名称。
- 使Linux内核变得强大的大量应用程序都是GNU软件，都是在GPL许可证下发布的。如：窗口管理工具、编译器、各种shell、浏览器等。

2、自由软件与GPL

http://www.kernel.org/

Linux内核维护网站



3、Linux系统的主要特点

- Linux是一套免费使用的，遵循 GPL、POSIX标准的、与UNIX兼容的操作系统。
- 能够在从微型计算机到大型计算机的任何环境中和任何平台上运行，良好的可移植性。
- 可运行于多种平台（x86、Alpha、MIPS、PowerPC、 SPARC等）之上，是目前支持硬件最多的操作系统
- 拥有现代操作系统具有的所有内容，例如:
  - 抢占式多任务处理，支持多用户
  - 内存保护
  - 支持SMP，支持TCP/IP
  - 支持绝大多数的32位和64位CPU 等
- 还有一些其他操作系统没有的特色，比如:
  - NFS，VFS，高效的EXT系列文件系统等

3、Linux系统的主要特点

◆广泛的支持
- 处理器芯片的广泛支持：Intel x86,SPARC, PowerPC,Alpha,IA64
- 硬件公司：IBM，HP，Sun，Intel，AMD，Sony
- 软件公司：Oracle、Informix、Ingres，SAP，Borland等
- 对即插即用设备的支持
- 广泛的文件系统的支持：VFAT,FAT,HFS,HPFS,NTFS,Ext2等
- 大型数据库公司的支持：Oracle、Informix、Ingres

3、Linux系统的主要特点

- 企业应用领域
  - 成熟的服务器级的操作系统、大型文件系统、多处理器、日志型文件系统、高可用集群、提供Web服务等
- 嵌入式领域
  - 商业化的嵌入式操作系统：VxWorks,pSOS, Windows CE等：高价格，源代码封闭
  - Linux在嵌入式系统蓬勃发展，广泛应用
    - 路由器，防火墙，机顶盒，电器设备

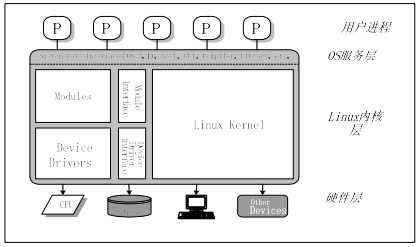
3、Linux系统的主要特点

- 设备独立性：把所有外部设备统一当作成文件来看待，只要安装它们的驱动程序，用户可以象使用文件一样使用设备
- 丰富的网络功能：具有完善的内置网络，大量有用的实用程序和通信、联网工具，可以实现远程管理功能
- 性能高和安全性强：Linux采取了许多安全技术措施，包括对读、写控制、带保护的子系统、审计跟踪、核心授权等，可以作为中小型ISP或Web服务器工作平台。
- 便于定制和再开发：在遵从GPL版权协议的条件下， 各部门、企业、单位或个人可根据自己的实际需要和使用环境对Linux系统进行裁剪、 扩充、 修改或者再开发。

3、Linux系统的主要特点

- 一个杰出的学习系统
  - 很多顶级的系统开发平台，友好的编程语言及编程工具都免费包含在操作系统里面
  - 病毒入侵，计算机的“后门”，软件提供商的“特别要求”，强迫性的软件升级，专有的文件格式，软件使用许可证和市场策略，产品注册登记，高得惊人的软件价格，等等 ...---Linux都没有

4. Linux 系统及内核的组成

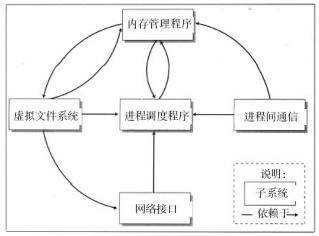


4. Linux 系统及内核的组成

◆Linux 系统由四部分组成

- **用户进程**：当一个用户程序在操作系统之上运行时，它成为操作系统中的一个进程。
- **系统调用接口(OS服务层)**：在应用程序中，可通过系统调用来调用操作系统内核中特定的过程，以实现特定的服务。
- **Linux内核**：内核是操作系统的灵魂，它负责 管理磁盘上的文件、内存，负责启动并运行程序，负责从网络上接收和发送数据包等等。
- **硬件:...**

4. Linux 系统及内核的组成



Linux 内核的组成

4. Linux 系统及内核的组成

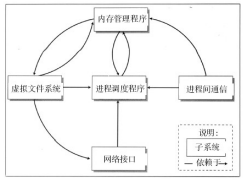
Linux 内核由4部分组成：

- **进程调度程序**：负责控制进程访问CPU。确保所有的进程都能公平访问CPU。
- **内存管理程序**：使多个进程可以安全地共享机器的主存系统，并支持虚拟内存。
- **虚拟文件系统**：提供一个所有设备的公共文件接口，抽象了不同硬件设备的细节
- **网络接口**：提供对多种网络协议标准和网络设备的访问。
- **进程间通信**：.....

4. Linux 系统及内核的组成

◆分析：

- 进程调度与内存管理之间的关系
- 进程间通信与内存管理的关系
- 虚拟文件系统与网络接口之间的关系
- 内存管理与虚拟文件系统之间的关系



Linux内核各个子系统之间的依赖关系

4. Linux 系统及内核的组成

Linux内核的特征如下：

(1)使用单一内核结构。但是,与传统的UNIX的单一内核结构不同的是，Linux可以动态装入和卸载内核中的部分代码。 Linux将这些代码段称为模块(module)，并给予强有力的支持，可根据需要自动装入和卸载。

也就是说，Linux内核运行在单独的内核地址空间。Linux汲取了微内核的精华：模块化设计、抢占式内核、支持内核线程以及动态装载内核模块的能力。Linux还避其微内核设计上性能损失的缺陷，让所有事情都运行在内核态，直接调用函数，无需消息传递。

4. Linux 系统及内核的组成

- **(2)进程调度方式简单而有效**。对于用户进程，Linux采用简单的动态优先级调度方式；对于内核中的例程(如设备驱动程序、中断服务程序等)采用软中断机制，保证了内核例程的高效运行。
- **(3)动态加载内核模块**。尽管Linux内核也是单内核，可是允许在需要的时候动态地卸除和加载部分内核代码。模块机制使内核保持独立而又易于扩充，如可以裁减得非常小适合于嵌入式系统。
- **(4)支持对称多处理机制**:尽管许多Unix的变体也支持SMP，但传统的Unix并不支持这种机制。
- **(5)Linux内核可以抢占**。与传统的Unix不同，Linux内核具有允许在内核运行的任务优先执行的能力。在其他各种Unix产品中，只有Solaris和IRIX支持抢占，但是大多数传统的Unix内核不支持抢占。

4. Linux 系统及内核的组成

(6)Linux对线程的支持：内核并不区分线程和其他的一般进程。对于内核来说，所有的进程都一样—只不过其中的一些共享资源而已。

Linux提供具有设备类的面向对象的设备模型、热插拔事件，以及用户空间的设备文件系统。

5、Linux版本认识

- **版本：分为内核版、发行版**
  - **内核版：分为稳定版、测试版**
  - **发行版：与基本组成与安装、配置、技术支持、第三方应用有关**
  - **发行版 = 内核 + tools**
- **Linux内核采用的是双树系统**
  - 一棵是**稳定树**，主要用于发行，新推出的稳定内核大部分都只是修正了一些Bug或是加入了一些新的设备驱动程序。
  - 另一棵是非稳定树或称为**开发树**，用于产品开发和改进，变化很快。

5、Linux版本认识

Linux内核版本号由3位数字组成r. x. y

- 使用三个或者四个用“.”分隔的数字来代表不同内核版本。
- 第一个数字是主版本号，第二个数字是从版本号，第三个数字是修订版本号，第四个可选的数字为稳定版本号（stable version）。
- 从版本号可以反映出该内核是一个稳定版本还是一个处于开发中的版本：该数字如果是偶数，那么此内核就是稳定版；如果是奇数，那么它就是开发版。
- 例如：版本号为2.6.30.1的内核---稳定版。内核的主版本号是2，从版本号是6，修订版本号是30，稳定版本号是1。
- 头两个数字在一起描述了“内核系列”—在这个例子中，就是2.6版内核系列。

5、Linux版本认识

◆ 例：

主版本号　　修订版本号

2 . 6 . 26 . 1

从版本号　　稳定版本号

◆显示内核信息

#uname –r 内核版本号
#uname –s 内核名称
#uname –v 内核版本发布时间
#uname –a 所有信息

5、Linux版本认识

- 最近几年，每一个2.6系列的修订版本（比如2.6.29）都会较其前身有深刻的变化，也都会经历数月才面世。它更有利于在引入新特性的同时，维持系统的稳定性。
- 为了解决版本发布周期变长的副作用，内核开发者们引入了**稳定版本号**。
- 稳定版本号（如2.6.32.8中的8）包含了关键性bug的修改，并且常会向前移植处于开发版内核（如2.6.33）的重要修改。依靠这种方式，以前版本保证了仍然能将重点放在稳定性上。

5、Linux版本认识

- **发行版本（Distribution）**

几种流行的Linux发行版本

5、Linux版本认识

5、Linux版本认识

发行版基本组成

- **内核**：系统核心、网络堆栈
- **引导管理器**：MBR、LILO
- **用户界面**：命令行、图形（CDE/KDE/GNOME）
- **X Window**：图形子系统，Xfree86
- **网络**：DNS/HTTP/SMTP/FTP/NNTP/PPP
- **文件及打印服务**：NFS、Samba
- **数据库**：mysql、PostgreSQL、DB2、Oracle 8、Sybase
- **应用程序**：Netscape/StarOffice/xIRC/LICQ/Gimp
- **包管理器**：安装/反安装/升级应用程序、DPM / RPM
- **开发工具与库**：gcc、gdb、glibc、libc

6、Linux内核源代码的组织

- **理解Linux源码的布局规则**
  - 指导Linux移植,添加驱动程序时需要修改哪些目录或者文件
- **如何得到源代码?**
- 在Linux系统根目录下，usr/src/linux目录下就是内核源代码。可以看到大量子目录。按树形结构组织。

6、 Linux内核源代码的组织

- **arch**： 包含了所有体系结构相关的核心代码。它还包含每种支持的体系结构的子目录，如i386。每个体系结构子目录下包含几个主要的子目录：
  - kernel 与体系结构相关的核心代码
  - mm 与体系结构相关的内存管理代码
  - lib 与体系结构相关的库代码
- **include**：包括了用来编译内核的大多数头文件。对于每种支持的体系结构分别有一个子目录。
- **init**：包含内核的初始化代码，内核开始工作的起点。
- **mm**：包含独立于CPU体系结构的所有的内存管理代码。

6、Linux内核源代码的组织

- **modules**： 仅仅包含已建好的模块。
- **fs**： 所有的文件系统代码。它被划分成对应不同文件系统的子目 录，如vfat和ext2。
- **kernel**：存放进程管理和调度等的主要核心代码，用于实现系统的核心模块。如：进程管理、进程调度、中断处理、时钟管理、同步机制等。如fork.c,sched.c,time.c
- **net**： 核心的网络部分代码。如各种网络协议，以及网卡驱动程序
- **lib**： 此目录包含了核心的库代码。与处理器结构相关库代码被放在arch/lib/目录下。
- **scripts**： 此目录包含用于编译内核时用的脚本文件（如awk和tk脚本）
- **drivers** :系统中所有的设备驱动都位于该目录中。它又进一步划分成几类设备驱动，如block。
- **ipc**: 包含了内核的进程间通讯代码。

6、Linux内核源代码的组织

- 每个目录下都有一个.depend文件和makefile文件，是编译时使用的辅助文件。
  - 仔细阅读这两个文件，对搞清楚各个文件之间的联系和依托关系很有帮助。
- Linux内核源代码
  - 放置在/usr/src/linux目录下
  - 对内核的修改是用补丁文件的方式发布的，patch实用程序用来对内核源文件进行一系列修改。
    - 例：现有内核 2.2.xx，想升级到2.4.18。则需要先获得2.4.18的补丁文件，应用patch来修改2.2.xx文件。
    - # cd /usr/src/linux
    - # patch –pl < patch-2.4.18

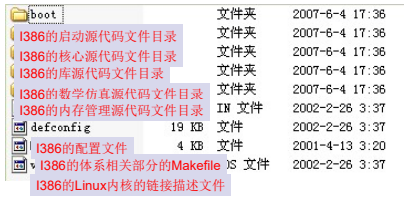
6、Linux内核源代码的组织

Linux内核对SMP的支持

- 每个处理器都在运行着进程，当运行在某个处理器上的进程耗尽其时间片或该进程处于等待状态时，该处理器将单独运行调度程序来选择新的进程。
- 每个处理器有个自己的空闲进程，都有自己的当前进程。
  - Task_struct结构中包含：
    - Processor字段，当前运行该进程的处理器编号
    - Last_ processor字段，上次运行该进程的处理器编号
    - Processor_musk字段，该字段第N位为1，表示该进程可以运行在第几个处理器上。利用该字段可以将某个进程限制在单个处理器上运行。

6、Linux内核源代码的组织

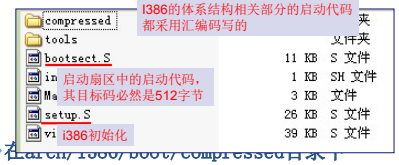
- 在arch/i386下存在如下目录



考虑arch为i386

6、Linux内核源代码的组织

- 在arch/i386/boot目录下



- 在arch/i386/boot/compressed目录下

i386的启动代码文件

6、Linux内核源代码的组织

- 在arch/i386/kernel目录下的.S文件

i386的启动代码文件

6、Linux内核源代码的组织

Linux内核源代码的几大组成部分

1、 系统启动与初始化
2、 内存管理
3、 内核
4、 进程间通信
5、 中断处理
6、 设备驱动程序
7、 文件系统
8、 网络
9、 模块

7、Linux内核分析方法

- **如何分析内核源代码**
  - 由于内核代码冗长、内核体系结构的庞杂，所以分析内核也是一个很艰难、很需要毅力的事
    - 弄懂源码的文件组织形式
    - 以程序流程为线索，一线串珠
    - 以数据结构为基点，触类旁通
    - 以功能为中心，各个击破
- **分析思路**
  - 概念和数据结构
  - 对外接口
  - 模块内部处理过程：算法
- **本课程以什么样的版本进行研究?**
  - Linux2.4.18+Linux 2.6的新增功能

7、Linux内核分析方法

后缀名	文件类型
.c	C语言源代码文件
.c.或cc.或cxx	C++源代码文件
.m	Objective_C源代码文件
.s	汇编语言源代码文件
.h	程序所包含的头文件
.i	已预处理过的C源代码文件
.ii	已预处理过的C++源代码文件
.S	已预编译的汇编语言源代码文件
.o	编译后的目标文件
.so	目标文件构成的动态库文件
.a	目标文件构成的静态库文件

Linux程序文件的后缀名

7、Linux内核分析方法

分析和实验验证环境-VMvare

- 模拟x86硬件平台
- 可以在Windows、Linux等平台上运行

- VMWare模拟出来的硬件包括：
  - 主板、内存、硬盘(IDE和SCSI)、DVD/CD-ROM、软驱、网卡、声卡、串口、并口和USB口。

- 可以直接从ISO文件上安装
- 在一个VMware上安装的的操作系统形成的虚拟机，可以拷贝到其他装有VMware的机器上

7、Linux内核分析方法

内核分析工具

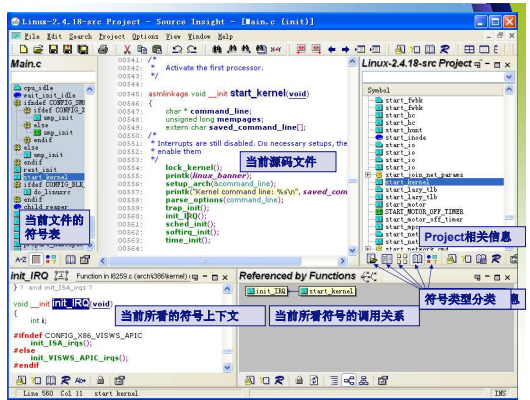
- **Source Insight**：Windows平台上的
- **LXR（Linux Cross Reference)**：Linux代码交叉检索工具，Linux平台上的，通过对符号建立索引，用户界面是浏览器http://lxr.Linux.org
- **Kscope**：http://kscope.sourceforge.net
- **QEMU**:模拟仿真工具，对内核代码进行调试、跟踪等

7、Linux内核分析方法

SourceInsight

- 源代码阅读和编辑工具
- 拥有内置的对C/C++，C#和Java等程序的分析。
- 能分析源代码并动态维护符号数据库，能自动显示有用的上下文信息。

- 到http://www.sourceInsight.com/上可以下载到它的试用版，试用期估计在30天左右



8、Linux与其他系统的区别

- **Linux与MS-DOS**
  - 任务调度：MS-DOS是单任务的操作系统，而Linux是多任务的操作系统，用户可以同时运行多个应用程序。
- **Linux与Windows**
  - Linux是从一个比较成熟的操作系统UNIX发展而来的，是一种开放、免费的操作系统，Windows是封闭的系统，有偿使用；
  - Linux软件丰富，功能完备的GUI
  - 稳定性好，硬件适应性强
  - 网络功能丰富

8、Linux与其他系统的区别

- **Linux与商用UNIX**
  - Linux和商用UNIX都基本支持同样的软件、程序设计环境和网络特性，可以说Linux是UNIX的PC版本，Linux在PC机上提供了相当于UNIX工作站的性能。
  - Linux是免费软件，用户可以从网上下载，而商用的UNIX除了软件本身的价格外，用户还需支付文档、售后服务费用
  - Linux拥有GNU软件支持，Linux能够运行GNU计划的大量免费软件，这些软件包括应用程序开发、文字处理、游戏等方面的内容
  - Linux的开发是开放的，任何志愿者都可对开发过程做贡献；而商用UNIX则是由专门的软件公司进行开发的.

8、Linux与其他系统的区别

- **Linux与自由的UNIX**
  - 组织方式：FreeBSD由它的核心团队领导，他们负责原始程序的开发与维护。Linux没有核心团队，来自世界各地的爱好者都可以发布自己的补丁 。
  - 发展方向：由于人力有限，FreeBSD的核心团队将主要精力投入在UNIX自身的风格和特点上，不像Linux那样对相关硬件支持多，因而在易用上稍微逊色一些。
  - 系统核心功能：二者区别不大，但由于Linux庞大的开发队伍，在Linux系统上可找到任何需要的功能，许多商用软件才具有的功能在Linux上很快就会出现。
  - 性能：据专家分析，FreeBSD在网络性能、软件移植性和系统规范化上略胜一畴，而在硬件支持、磁盘IO操作等方面Linux略强一些 。

9.Linux中的C语言与汇编语言

- **Linux源代码中的C语言代码**
  - Linux主体是用GNU的C语言编写
  - 从C++中吸收了“inline”和“const”
  - 支持“属性描述符”（attribute）
  - 增加了新的基本数据类型“long long int”用于支持64位CPU
- **Linux源代码中的汇编语言代码：两种**
  - 完全的汇编代码，以.s作为文件名后缀(可以单独用gcc汇编成目标代码)
  - 嵌入在c程序中的汇编代码

9.Linux中的C语言与汇编语言

- 在Linux内核中使用的C语言与通常的C语言有所不同，gcc编译器是一个c/c++编译器，所以在Linux设计中引入了很多C++技术。如：

```
struct Student
{……
}
```

引用时：

普通C中：

```
Student S;
```

gcc中的C：Struct Student S;

9.Linux中的C语言与汇编语言

- 与一般的386汇编语言采用intel定义不同，它采用的是AT&T定义的格式。主要差别如下：
  - Intel中多使用大写字母，而这里大多使用小写字母
  - 寄存器名前面要加“%”作为前缀
  - 指令的源操作数与目标操作数的顺序与intel的正好相反。AT&T格式中，源在前，目标在后
  - 访问内存的指令的操作数大小（即宽度）由操作码名称的最后一个字母决定，用作操作码后缀的字母有b（8位），w（16位），l（32位），e.g movb
  - 直接操作数要加“$”作为前缀，intel中不用

9.Linux中的C语言与汇编语言

区别项	AT&T格式	Intel格式
指令大小写	多用小写.如: movw %eax,%ebx	多用大写，如PUSH AX
寄存器名	寄存器名前加%号.如: %eax ,%edi	寄存器名前不加任何符号，EAX， EDI
源/目的操作数顺序	操作指令 源数 目的数 mov %eax,%ebx;eax→ebx	操作指令 目的数 源数 MOV EAX,EBX;EBX→EAX
操作数长度数位表示	操作码最后一字母表示长度位数，其中b(16位)，w(32位)，l(64位) movb FOO,%al	操作数前加BYTE PTR，WORD PTR，DWORD PTR限定位数 MOV AL,BYTE PTR FOO
立即数表示	立即数前加$号，如: push $2	不加任何符号，如PUSH 2
远程转移/调用指令	指令前加字符l,如lcall,ljmp	指令后加FAR 如CALL FAR，JMP FAR

9.Linux中的C语言与汇编语言

- **是一种嵌入在C程序中的汇编**
- **AT&T行内汇编**：在C代码中嵌入汇编代码。在GCC中提供asm或者_asm_两个关键字，这样使得C代码中嵌入汇编代码非常的简单、方便。
- asm("statements");
  - 例如：asm("pushb %al");asm("popb %al");
- 多行汇编可以多个单行汇编，也可以用一个asm修饰符括起，中间插入格式分隔符号，\n表示回车，\t表示跳格，例如：

```
asm("pushl %eax\n\t "
    "movl $0,%eax\n\t"
    "popl %eax");
```

  - 实际上GCC在处理汇编时，把asm括号内的内容按原样拷贝到汇编文件中，所以格式控制字符是必要的。

9.Linux中的C语言与汇编语言

- **扩展行内汇编**：为解决代码中因使用寄存器可能出现的问题，引入了扩展行内汇编。它的基本格式是：

```
asm("statements"
:output_regs   //输出寄存器说明
:input_regs    //输入寄存器说明;
:clobbered_regs  //被改变的寄存器。
```

  - 输出和输入寄存器统一按顺序编号，起始是%0
  - 操作数如何与C代码中的变量结合是个问题。程序员提供具体的指令,而对寄存器的使用只需给出约束条件,具体如何将寄存器与变量结合起来则完全由GCC等负责.

关于嵌入式汇编，请参考http://gcc.gnu.org/onlinedocs/上的文档：
1）在GCC的文档中有部分

AT&T行内汇编

9.Linux中的C语言与汇编语言

- int a=10, b;

```
asm (
"movl %1, %%eax;
 movl %%eax, %0;"
 :"=r"(b) /* output */
 :"r"(a) /* input */
 :"%eax" /* clobbered register */
 );
```

9.Linux中的C语言与汇编语言

- 上面代码所做的就是用汇编代码把a的值赋给b:
  1）“b”是输出操作数，用%0来访问,”a”是输入操作数，用%1来访问。
  2）“r” 是一个constraint, constraint ” r”让GCC自己选择一个寄存器去存储变量a。
- 输出部分的constraint前必须要有个 ” =”，用来说明是一个这是一个输出操作数，并且只写。
  3) 有的寄存器名字前面写了两个%，这是用来帮助GCC区分操作数和寄存器。操作数只需要一个%前缀。
  4) 在第三个冒号后面的clobbered register部分，%eax 说明在内联汇编代码中将要改变eax中的内容，GCC不要用他存储其他值。
  当这段代码执行结束后，” b”的值将会被改掉，因为它被指定作为输出操作数。换句话说，在“asm”内部对b的改动将影响到asm外面.

本章作业

- 请通过进一步查阅资料了解Linux的发展过程。
- 解释下列名词：自由软件，GPL，POSIX，GNU
- Linux内核版本中包含哪些信息？
- Linux内核的基本组成是什么？各模块之间有什么关系？
- Linux内核目录结构如何？各目录主要用途有哪些？
- 什么是Linux的发行版本？写出你知道的发行版本
- Linux系统有哪些特点？
- 解释Linux 2.4.18.10内核版本中各数字的含义。

第2章 Linux进程管理

2.0 进程与线程知识回顾
2.1 Linux进程描述
2.2 Linux进程状态与转换
2.3 Linux进程标识与组织
2.4 Linux进程的系统堆栈
2.5 Linux进程的链表结构
2.6 Linux进程的家族关系
2.7 Linux进程控制
2.8 Linux进程与线程实现

本章教学目标

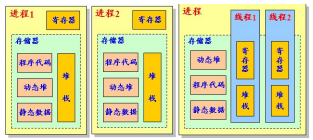
- 理解和掌握Linux的进程/线程描述机制
  - Linux进程描述符
  - Linux进程的系统堆栈结构
- 理解和掌握Linux进程的内核管理机制
  - 内核链表结构
  - Linux的进程家族结构
  - Linux进程系统启动过程
- 理解和掌握Linux进程创建与实现机制
  - fork()、vfork()、clone()的使用场景及实现机制的差别
  - do_fork()的工作机理
- 理解和掌握Linux线程创建与实现机制
  - Linux线程的实现机制
  - Linux用户线程/内核线程的创建方法

2.0 进程与线程知识回顾

- Linux是多任务、多用户操作系统，一个任务(Task)就是一个进程(Process)。
- 每一个进程都具有一定的功能和权限，它们都运行在各自独立的虚拟地址空间。
- **在Linux中，进程是系统资源分配的基本单位，也是CPU的基本调度单位。**
- 进程在OS内用PCB表示（Process Control Block），PCB是进程的属性之一。

2.0 进程与线程知识回顾

- 进程（process）
  - **程序的一次执行及其所包含资源的总称**
    - 程序：可执行程序代码
    - 资源：打开文件、挂起信号、地址空间、数据段等
- 线程（thread）
  - **进程中活动的对象**
    - 有独立的程序计数器、进程栈及一组进程寄存器



2.0 进程与线程知识回顾

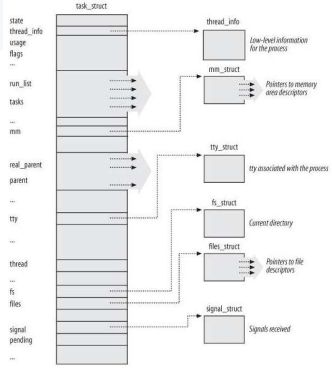
- 从形态角度
  - 一个进程可包含一个或多个线程
- 从调度角度
  - 进程是资源分配的基本单位
  - 线程是处理器调度的独立单位
- 从虚拟化角度
  - 进程提供两种虚拟机制
    - 虚拟处理器：进程独享处理器的假象
    - 虚拟内存：进程拥有系统内所有内存资源的假象
  - 线程之间可共享虚拟内存，但各自拥有独立虚拟处理器

2.0 进程与线程知识回顾

- 正文段
  - 存放进程运行的代码，描述进程需完成的功能
- 进程数据段
  - 存放正文段在执行期间所需的数据和工作区
    - 包括全局变量、动态分配的空间（调用malloc函数）
  - 用户栈也在该数据段开辟，存放函数调用时的栈帧、局部变量等
- 系统堆栈
  - 每个进程捆绑一个，进程在内核态下工作时使用
  - 保存中断现场、执行函数调用时的参数和返回地址等
  - 其中最重要的数据结构是进程控制块

2.1 Linux进程描述

- 内核必须对每个进程进行清晰的描述。Linux 内核利用数据结构 task_struct 来描述，它代表一个进程的PCB。
- task_struct定义于include/linux/sched.h 中，内核以此感知进程的存在。
- 当建立新进程的时候，Linux 为新的进程分配一个 task_struct 结构。



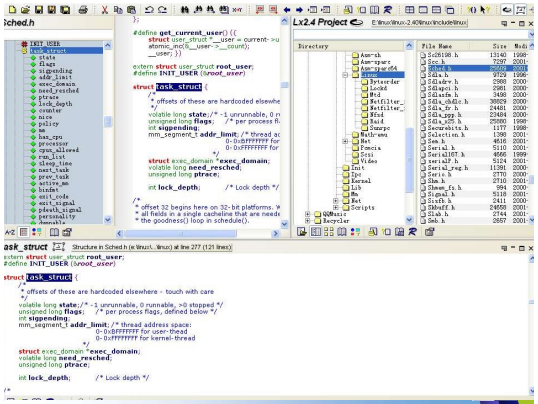
2.1 Linux进程描述符

- 进程描述符提供了内核所需了解的进程信息
- 进程描述符
  - 数据结构：task_struct；定义位置：include/linux/sched.h
  - include/linux/sched.h 中的struct task_struct
- 进程描述符向量结构
  - 数据结构：task[NR_TASKS]
  - 定义位置： include/linux/sched.h
  - 定义格式
    - struct task_struct *task[NR_TASKS] = {&init_task}
    - #define NR_TASKS 512
  - 说明
    - 全局变量NR_TASKS记录系统可容纳进程数，默认大小是512

2.1 Linux进程描述符

- 进程状态信息(state, flags, ptrace)
- 调度信息(static_prio, normal_proi, run_list, array, policy)
- 内存管理(mm, active_mm)
- 进程状态位信息(binfmt, exit_state, exit_code, exit_signal)
- 身份信息(pid, tgid, uid, suid, fsuid, gid, egid, sgid, fsgid)
- 家族信息(real_parent, parent, children, sibling)
- 进程耗间信息(realtime, utime, stime, starttime)
- 时钟信息(it_prof_expires, it_virt_expires, it_sched_expires)
- 文件系统信息(link_count, fs, files)
- IPC信息(sysvsem, signal, sighand, blocked, sigmask, pending)

Linux进程描述符的信息组成



2.1 Linux进程描述符

再看Linux 2.4内核：

```
struct task_struct {
…
    struct list_head run_list;
    struct task_struct *next_task, *prev_task;
    struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr,
    *p_osptr;
…
    struct task_struct *pidhash_next;
   struct task_struct **pidhash_pprev;
… }
```

1) p_opptr指向进程的原始祖先，通常和p_pptr类似
2) p_pptr指向进程的当前祖先
3) p_cptr指向进程的最年青(最近)子孙
4) p_ysptr指向进程的下一个最年青(下一个最近)兄弟
5) p_osptr指向进程的下一个最古老(下一个最远)兄弟

2.2 Linux进程的状态及转换

- task_struct中的state 表示进程当前的状态：
  volatile long state
- 功能：表征进程的可运行性
- 状态定义[include/linux/sched.h]

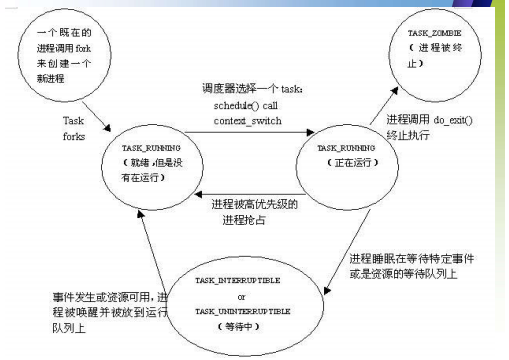
```
#define TASK_RUNNING        0
#define TASK_INTERRUPTIBLE  1
#define TASK_UNINTERRUPTIBLE  2
#define TASK_STOPPED        4
#define TASK_TRACED         8
#define TASK_ZOMBIE         16
#define TASK_DEAD           32
```

- Linux 2.4与Linux 2.6的状态定义变化
  - 新增两种状态：TASK_TRACED、TASK_DEAD
  - 宏定义数值有较大差别

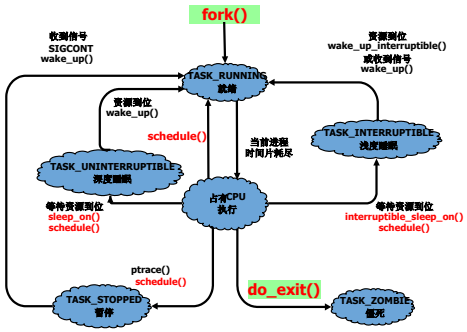
2.2 Linux进程的状态及转换

- 运行态/就绪态
  - TASK_RUNNING：正在运行或已处于就绪只等待CPU调度。由current 指针指向当前运行的进程。
  - TASK_TRACED：供调试使用
- 被挂起状态
  - TASK_INTERRUPTIBLE：可被信号或中断唤醒进入就绪队列
  - TASK_UNINTERRUPTIBLE：等待资源不可被其他进程中断
  - TASK_STOPPED：被调试暂停，或收到SIGSTOP等信号
- 不可运行态
  - TASK_ZOMBIE：正在终止（已结束，已释放内存、文件等资源，但其父进程未接到通知，描述符未释放），等待父进程通过wait4()或waitpid()回收
  - TASK_DEAD：已退出且不需父进程回收的进程的状态

2.2 Linux进程的状态及转换



2.2 Linux进程的状态及转换



2.2 Linux进程的状态及转换

✓ sleep_on():

TASK_RUNNING → TASK_UNINTERRUPTIBLE

- 运行进程申请资源无效时，通过sleep_on()将进程从TASK_RUNNING切换到ASK_UNINTERRUPTIBLE状态，加入等待队列中。
- 一般引起状态变成TASK_UNINTERRUPTIBLE的资源申请，都是对一些硬件资源的申请。如果得不到这些资源，进程将不能执行下去，不能由signal信号或时钟中断唤醒回到TASK_RUNNING状态。

2.2 Linux进程的状态及转换

✓ sleep_on_timeout():

TASK_RUNNING → TASK_UNINTERRUPTIBLE

✓ interruptible_sleep_on_timeout（）：

TASK_RUNNING → TASK_INTERRUPTIBLE

- 虽然在申请资源或运行中出现了某种错误，但是系统仍然给进程一次重新运行的机会。调用该函数将进程从TASK_RUNNING切换到TASK_INTERRUTIBLE状态，并等待规定的时间片长度,再重新试一次。

2.2 Linux进程的状态及转换

✓ wake_up()

TASK_UNINTERRUPTIBLE→TASK_RUNNING

TASK_INTERRUPTIBLE → TASK_RUNNING

- 处于TASK_INTERRUPTIBLE状态的进程可在资源有效时被wake_up()、wake_up_interruptible()或wake_up_process()唤醒
- 处于TASK_UNINTERRUPTIBLE状态的进程不能由signal信号或时钟中断唤醒，只能由wake_up()或wake_up_process()唤醒。

2.3 Linux进程的标识与组织

进程标识

- 成员名：pid
- 功能
  - 内核通过pid标识每个进程
  - pid与进程描述符之间一一对应关系
- 数据类型说明
  - pid_t 实际上是一个int类型
  - 取值范围：0 ~ 32767
  - 最大值修改：/proc/sys/kernel/pid_max
  - 生成新pid：get_pid()函数
    - +1；循环
  - 获取进程pid
    - ps命令；访问/proc/pid
    - getpid()←sys_getpid()

2.3 Linux进程的标识与组织

通过ps命令获取进程信息

- 功能
  - 查看系统中正在运行的进程
- 语法
  - ps [-ef][-n name][-t ttys][-p pids][-u users][-groups]

用户ID　进程号　父进程号　进程占用CPU的百分比　启动进程的终端号　进程开始的时间和日期

```
[root@localhost root]# ps -ef | more
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  2 20:51 ?        00:00:04 init
root         2     1  0 20:51 ?        00:00:00 [keventd]
root         3     1  0 20:51 ?        00:00:00 [kapmd]
root         4     1  0 20:51 ?        00:00:00 [ksoftirqd/0]
root         7     1  0 20:51 ?        00:00:00 [bdflush]
root         5     1  0 20:51 ?        00:00:00 [kswapd]
root         6     1  0 20:51 ?        00:00:00 [kscand]
```

进程已占用的时间　运行的命令

2.3 Linux进程的标识与组织

进程组标识

- 成员名：pid_t tgid
- 功能
  - 标识进程是否属于同组，组ID是第一个组内线程（父进程）的ID
  - 线程组中的所有线程共享相同的PID
- 组ID赋值相关操作

```
p->tgid = p->pid;
if (clone_flags & CLONE_THREAD)
    p->tgid = current->tgid;
```

- 单线程进程：tgid和pid相等
- 多线程进程：组内所有线程tgid都相等，且等于父进程pid

2.3 Linux进程的标识与组织

- 功能：控制用户对系统资源的访问权限

用户相关的进程标识信息

  - **用户标识uid及组标识gid**
    - 通常是进程创建者的uid和gid
  - **有效用户标识euid及有效组标识egid**
    - 有时系统会赋予一般用户暂时拥有root的uid和gid(作为用户进程的euid和egid)，以便于进行运作。
  - **备份用户标识suid及备份组标识sgid**
    - 在使用系统调用改变uid和gid时，利用其保留真正uid和gid
  - **文件系统标识fsuid及文件系统组标识fsgid**
    - 检查对文件系统访问权限时使用，通常与euid及egid相等
    - 在NSF文件系统中，NSF服务器需要作为一个特殊的进程访问文件，此时只修改客户进程的fsuid和fsgid，而不改变euid及egid，可避免受恶意攻击

2.3 Linux进程的标识与组织

- 获取用户相关的进程标识信息的方法：getXXX()

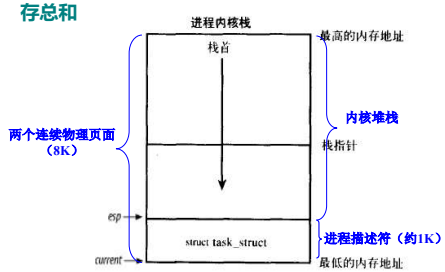
```
#include <sys/types.h>
int main(int argc,char **argv) {
  pid_t my_pid,parent_pid;
  uid_t my_uid,my_euid;
  gid_t my_gid,my_egid;
  my_pid=getpid( );
  parent_pid=getppid( );
  my_uid=getuid( );
  my_euid=geteuid( );
  my_gid=getgid( );
  my_egid=getegid( );

  printf("Process ID:%ld\n",my_pid);
  printf("Parent ID:%ld\n",parent_pid);
  printf("User ID:%ld\n",my_uid);
  printf("Effective User ID:%ld\n",my_euid);
  printf("Group ID:%ld\n",my_gid);
  printf("Effective Group ID:%ld\n",my_egid):
}
```

2.4 Linux进程的系统堆栈

- 每个进程都要单独分配一个系统堆栈。Linux的进程描述符与所属进程的内核堆栈放在一起。
- Linux为每个进程分配一个8KB大小的内存区域，用于存放该进程两个不同的数据结构：
  - 进程描述符task_struct：位于8K低端
  - 进程的内核堆栈：进程处于内核态时使用，位于8K高端
- 系统堆栈的结构特点
  - 8192（2^{13}）字节，两个页框
  - 占用连续两个页框，且第一个页框起始地址为2^{13}的倍数

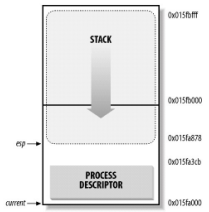
2.4 Linux进程的系统堆栈

- Linux系统进程个数限制
  - 所有进程的PCB及系统堆栈占用空间≤1/2的物理内存总和



2.4 Linux进程的系统堆栈

问题：为什么这样存放？



这对当前进程的定位有好处：因为内核很容易从esp寄存器的值获得指向当前运行进程的PCB指针,由current宏完成。这样就既节省内存，管理上也比较方便。

2.4 Linux进程的系统堆栈

- C语言用如下的一个union结构来表示内核堆栈。

```
union task_union {
    struct task_struct task;
    unsigned long stack[INIT_TASK_SIZE/sizeof(long)];
};
```

2.4 Linux进程的系统堆栈

- 通常OS应有一个指向当前运行进程的PCB指针，以便系统可以随时访问它。
- Linux却不是这样
  - 它利用其起始地址一定在堆栈所在的两页空间的低地址页边界的特点，根据堆栈指针推算进程控制块的首地址。
- 从进程描述符和内核态堆栈之间的配对，内核可以很容易地从esp寄存器的值获得当前在CPU上运行的进程的控制块指针。

2.4 Linux进程的系统堆栈

- 这个内存区是8KB=2^{13}字节大小，内核必须让esp有13位的有效位，以获得进程控制块的基地址。这个工作由current宏来完成，该算法的伪代码如下。
- 通过将该值低13位清零，来得到8KB空间的起始地址，即PCB的地址。
- current宏可以看成当前进程的进程控制指针，在内核中直接使用，这样可以节省一个寄存器。
- 举例：current→pid返回正在执行的进程的PID

current宏：
current=(struct task_struct(*)STACK_POINTE&0xFFFE000

2.4 Linux进程的系统堆栈

- 结构定义(/include/linux/sched.h)

```
union task_union {
    struct task_struct task;
    unsigned long stack[INIT_TASK_SIZE/sizeof(long)];
};
```

#define INIT_TASK_SIZE 2048*sizeof(long)

2048

- 进程描述符管理
  - 分配：alloc_task_struct()
  - 回收：free_task_struct()
  - 访问：get_task_struct()

2.4 Linux进程的系统堆栈

◆由current来获取当前进程的进程描述符

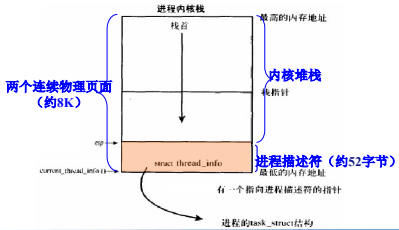
```
static inline struct task_struct * get_current(void)
{
    struct task_struct *current;
    __asm__("andl %%esp,%0; ":"=r" (current) : "0" (~8191UL));
    return current;
}
#define current get_current()
```

8191=8192-1=0x2000-1=0x1fff
取反：0xffffe000（最后13位为0）

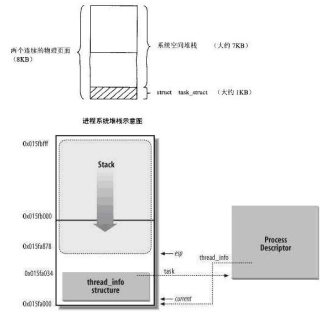
- 说明
  - esp寄存器是CPU栈指针，%esp指向内核堆栈，存放栈顶单元地址
  - ~8191UL表示最低13位为0，其余位全为1
  - 屏蔽掉%esp的最低13位后，得到“两个连续的物理页面”的开头（基地址），从而得到指向task_struct的指针

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
  - 进程描述符由slab分配器动态生成
  - 栈底用新结构struct thread_info，指向进程描述符



2.4 Linux进程的系统堆栈



Linux 2.6进程系统堆栈结构

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- Linux 2.6版本的内核栈底放的是thread_info结构，通过thread_info里的task指针可以找到进程的task_struct 。定义内核堆栈时使用的联合体：

```
union thread_union {
  struct thread_info thread_info;
  unsigned long stack[THREAD_SIZE/sizeof(long)];};
```

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- thread_info结构定义[include/asm-x86/thread_info_32.h]
  - thread_info（52个字节）

```
struct thread_info {
  struct task_struct *task; /* main task structure */
  struct exec_domain *exec_domain; /* execution domain */
  unsigned long flags; /* low level flags */
  unsigned long status; /* thread-synchronous flags */
  ... ..
}
```

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- thread_uion定义[/include/linux/sched.h]

```
union thread_union {
  struct thread_info thread_info;
  unsigned long stack[THREAD_SIZE/sizeof(long)];
};
```

- THREAD_SIZE定义[include/asm-x86/thread_info_32.h]

```
#ifdef CONFIG_4KSTACKS
#define THREAD_SIZE          (4096)
#else
#define THREAD_SIZE    (8192)
#endif
```

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- 根据内核的配置，THREAD_SIZE既可以是4K字节(1个页面)也可以是8K字节(2个页面)。thread_info是52个字节长。
- thread_info在这个内存区的开始处，内核堆栈从末端向下增长。
- Linux2.6中，进程描述符不在堆栈中，而分别通过task与thread_info指针使thread_info与进程描述符相联。

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构
- 由current来获取当前进程的进程描述符
  - 根据THREAD_SIZE大小，分别屏蔽掉内核栈的12-bit LSB(4K)或13-bit LSB(8K)，从而获得内核栈的起始位置

```
#define current get_current()
static inline struct task_struct * get_current(void){
  return current_thread_info()->task;
}
static inline struct thread_info *current_thread_info(void){
  struct thread_info *ti;
  __asm__("andl %%esp,%0; ":"=r" (ti) : "" (~(THREAD_SIZE - 1)));
  return ti;
}
```

2.4 Linux进程的系统堆栈

- Linux 2.6进程系统堆栈结构小结
- 与内核堆栈存放在一起的不再是进程控制块，而是thread_info结构。
- 但是由于thread_info结构中有一个指向进程控制块的task指针，所以系统仍然使用current宏查找进程控制块。
- 原因分析
  - PCB中最频繁被引用的是thread_info。
  - 随着版本的变化，进程的PCB中的内容越来越多，所需空间越来越大，使得留给内核堆栈的空间过小，所以将部分进程控制块移出这个空间，只在核心栈保留了访问频繁的thread_info.

2.5 Linux进程的链表结构

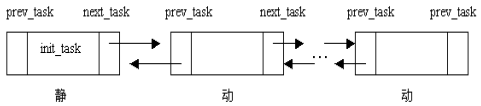
- 进程链表
- 运行队列链表
- 等待队列链表
- 哈希链表
- 指向当前进程的current指针

2.5 Linux进程的链表结构

◆为了对给定类型的进程，如所有在可运行状态下的进程进行有效搜索，内核维护了几个进程链表:
- 所有进程链表
- 运行队列
- 等待队列
- pidhash表

2.5 Linux进程的链表结构

- 所有进程链表：所有的进程用task_struct中的*prev_task和*next_task组成一个双向循环链表。链表的头和尾都是init_task。
- init_task的进程描述符是预先由编译器分配的，在运行的过程中保持不变。



在进程描述符中： struct task_struct *next_task, *prev_task;

2.5 Linux进程的链表结构

- 当CPU空闲时，内核调度程序寻找一个新的进程在CPU上运行时，必须只考虑可运行进程，因为扫描整个进程链表效率很低。
- 由此，引入了TASK_RUNNING状态的进程双向循环链表，也叫运行队列（running queue）。
- 进程描述符使用list_head用来实现运行队列

```
struct list_head run_list;
```

- list_head是Linux内核当中定义的一个数据结构用来实现双向链表 (include\list.h)

```
struct list_head {
    struct list_head *next, *prev;    };
```

2.5 Linux进程的链表结构

- 对可运行队列的一些操作函数

add_to_runqueue
del_from_runqueue — 增加/删除一个可运行进程

int nr_running; — 可运行队列的长度，可运行进程的个数

wake_up_process — 唤醒一个进程，将进程的状态设置为TASK_RUNNING，使一个进程可运行

...

2.5 Linux进程的链表结构

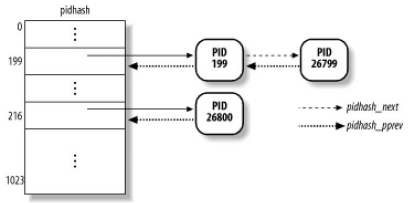
- 为了快速定位进程，系统以PID为关键字建立了一个哈希表（pidhash表）。若知道进程PID，可以通过该哈希表很快找到该进程。
- 例如：POSIX规范中规定的系统调用kill()是基于进程号向其它进程发送信号。因此，需要一种机制来高效地完成从进程号到进程描述符地址的转换。

```
#define PIDHASH_SZ (4096 >> 2)
struct task_struct *pidhash[PIDHASH_SZ];
```

- #define pid_hashfn(x) (((((x) >> 8) ^ (x)) & (PIDHASH_SZ - 1))

2.5 Linux进程的链表结构

- hash_pid(), unhashpid()在pidhash表中分别插入和删除一个进程。



2.5 Linux进程的链表结构

- 当要把除了TASK_RUNNING状态之外的进程组织在一起时，Linux使用了等待队列
  - TASK_STOPPED和TASK_ZOMBIE状态的进程不在专门的链表中，它们的父进程可以通过进程的PID或进程间亲属关系检索到子进程。
  - TASK_INTERRUPTIBLE和TASK_UNINTERRUPTIBLE状态的进程可以再分成很多类，每一类对应一个特定的事件。在这种情况下，进程状态提供的信息满足不了快速检索，因此，内核引进了另外的进程链表，叫做等待队列。

2.5 Linux进程的链表结构

- 等待队列使得进程可以在事件上进行的条件等待，并且当等待的条件为真时，由内核唤醒它们。尤其是对中断处理、进程同步和定时用处很大。
- 等待队列是以双循环链表为基础数据结构

```
struct __wait_queue {
    unsigned int flags;
    struct task_struct * task;
    struct list_head task_list;
};
typedef struct __wait_queue wait_queue_t;
```

- 其元素包括指向进程描述符的指针。每个等待队列都有一个等待队列头(wait_queue_head),等待队列头是一个类型为wait_queue_head_t的数据结构。

2.5 Linux进程的链表结构

- 要唤醒等待队列中所有睡眠的进程有时并不方便。
- 例如:如果两个或多个进程正在等待互斥访问某一要释放的资源，仅唤醒等待队列中一个进程才有意义。
  - 这个进程占有资源，而其他进程继续睡眠.
  - 这就避免了所谓的“雷鸣般群兽”问题，即唤醒多个进程只为了竞争一个资源，而这个资源只能有一个进程访问，结果是 其他进程必须再次回去睡眠。

2.5 Linux进程的链表结构

- 有两种处于阻塞状态的进程
  - 互斥进程(等待队列元素的flag字段为1)：由内核有选择地唤醒。
  - 非互斥进程(flag值为0)：总是由内核在事件发生时唤醒。
- 在等待队列上内核实现了一些操作函数
  - add_wait_queue()
  - remove_wait_queue()

2.5 Linux进程的链表结构

- 等待一个特定事件的进程能调用下面几个函数中的任一个
  - sleep_on()
  - sleep_on_timeout()
  - interruptible_sleep_on()
  - interruptible_sleep_on_timeout()
- 进程等待由需要等待的进程自己进行（调用）

2.5 Linux进程的链表结构

```
void sleep_on(wait_queue_head_t *q)
{
    unsigned long flags;
    wait_queue_t wait;
    wait.flags = 0;
    wait.task = current;
    current->state = TASK_UNINTERRUPTIBLE;
    add_wait_queue(q, &wait);
    schedule();
    remove_wait_queue(q, &wait);
}
```

2.5 Linux进程的链表结构

- 进程的唤醒：利用wake_up()或者wake_up_interruptible()等一系列的宏，都让插入等待队列中的进程进入TASK_RUNNING状态

```
#define wake_up(x)                  __wake_up((x),TASK_UNI
#define wake_up_nr(x, nr)           __wake_up((x),TASK_UNI
#define wake_up_all(x)              __wake_up((x),TASK
#define wake_up_sync(x)             __wake_up_sync((x
#define wake_up_sync_nr(x, nr)      __wake_up_syn
#define wake_up_interruptible(x)    __wake_up((x),TASK
#define wake_up_interruptible_nr(x, nr)    __wake_up
#define wake_up_interruptible_all(x)    __wake_up((x),
#define wake_up_interruptible_sync(x)   __wake_up_syn
#define wake_up_interruptible_sync_nr(x)   __wake_up_
```

2.6 Linux进程的家族关系

- 所有进程都是init进程(pid=1)的后代
- 每个进程必须有一个父进程
- 每个进程可以拥有零个或者多个子进程
- 具有相同父进程的进程称为兄弟

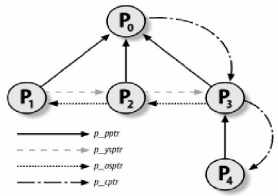
2.6 Linux进程的家族关系

- 进程之间具有父子关系，进程的父子关系是树型组织。
- 进程描述符中有几个域用来表示这样的关系

```
/*
 * pointers to (original) parent process, youngest child, younger sibling,
 * older sibling, respectively.  (p->father can be replaced with
 * p->p_pptr->pid)
 */
struct task_struct *p_opptr, *p_pptr, *p_cptr, *p_ysptr, *p_osptr;
```

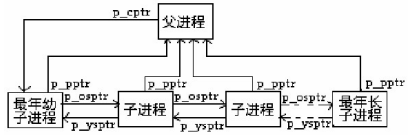
2.6 Linux进程的家族关系

- p_opptr : original parent（1号进程或者创建它的父进程）
- p_pptr: 父进程，有时候是调试时的调试监管进程
- p_cptr: child (指向自己最年轻的子进程)
- p_ysptr：指向比自己年轻的兄弟进程
- p_osptr：指向比自己老的兄弟进程



2.6 Linux进程的家族关系

```
/* 父进程(original parent) */
struct task_struct *p_opptr;
/* 父进程(parent) */
struct task_struct *p_pptr;
/* 子进程(youngest child) */
struct task_struct *p_cptr;
/* 新兄弟进程(younger sibling) */
struct task_struct *p_ysptr;
/* 老兄弟进程(older sibling) */
struct task_struct *p_osptr;
```



2.6 Linux进程的家族关系

- 父进程也有生父和养父之分。在task_struct结构中有个指针p_opptr指向其“original parent”也就是生父。另外还有个指针p_pptr则指向养父。
- 一个进程在创建之初其生父和养父是一致的，两个指针指向同一个父进程，但是在运行中p_pptr可以暂时地改变。
- 这种改变发生在一个进程通过系统调用ptrace()来跟踪另一个进程的时候，这时候被跟踪的进程的p_pptr的指针被设置成指向正在跟踪它的进程，那个进程就暂时成了被跟踪进程的养父。而被跟踪进程的p_opptr指向这个线程。
- 否则，就只好托付给系统中的init进程，所以这个init进程就好像是孤儿院。

2.6 Linux进程的家族关系

- Linux中的0号进程，是所有进程的祖先，在OS启动时创建。由它执行cpu_idle()函数，当没有其他进程处于TASK_RUNNING的时候，调度程序会选择0号进程运行
- 0号进程创建1号进程，1号进程通常称为init进程。它首先创建一些后台进程来维护系统，然后进行系统配置，执行shell编写的初始化程序。然后转入用户态运行
- 思考题：查资料，了解Linux2.6中的父子进程中间的养父关系。

2.6 Linux进程的家族关系

- 获取父进程
  - struct task_struct *my_parent = current->parent
- 获取所有子进程（实质是遍历list_head结构）

```
struct task_struct *task;
struct list_head *list;

list_for_each(list, &current->children) {
  task = list_entry(list, struct task_struct, sibling);
  /* task现在指向当前的某个子进程 */
}
```

2.7 Linux中的进程控制

- Linux进程的创建
- Linux进程的执行
- Linux进程的等待
- Linux进程的终止

2.7 Linux中的进程控制

- Linux的0号进程
  - 称为0号进程或idle进程或swapper进程
  - 维护Linux内核代码段、数据段及堆栈
  - 是唯一一个通过静态分配创建的进程[arch/x86/kernel/init_task.c]
    - INIT_MM, INIT_FS, INIT_FILES, INIT_SIGNALS, INIT_SIGHAND等宏初始化进程描述符的各个对应域
    - INIT_TASK完成init_task变量中进程描述符初始化
    - INIT_THREAD_INFO完成对init_thread_info中thread_info及内核堆栈的初始化
    - 主内核页全局目录存放在swapper_pg_dir变量中
  - start_kernel函数(/init/main.c)初始化内核所需所有数据结构、激活中断、创建1号内核线程

2.7 Linux中的进程控制

- Linux的1号内核进程
  - 又称为init进程（PID=1）
  - 由0号进程在start_kernel()函数中调用rest_init创建

```
static void rest_init(void)
{
    kernel_thread(init, NULL, CLONE_FS | CLONE_FILES | CLONE_SIGNAL);
    unlock_kernel();
    current->need_resched = 1;
    cpu_idle();
}
```

  - 调度程序选择到init进程时，init进程开始执行init()函数

2.7 Linux中的进程控制

- init()函数说明
  - 实现从内核态到用户态的切换
  - 为常规内核任务初始化一些必要的内核线程
    - kflushd：刷新‘脏’缓冲区中内容到磁盘，归还内存
    - kswapd：执行内存回收功能
  - 调用系统调用execve()装入可执行程序init
    - init内核线程变成一个普通进程(拥有自己的进程内核数据结构)
    - 但init进程从不终止，因为它创建和监控操作系统外层的所有进程的活动

2.7 Linux中的进程控制

- 传统的UNIX操作系统采用统一的方式来创建进程
  - 子进程复制父进程所拥有的资源
- 缺点：
  - 创建过程慢、效率低
  - 事实上，子进程复制的很多资源是不会使用到的
- 引入三种不同的机制来解决这个问题
  - 写时复制技术
  - sys_fork()/sys_clone()/sys_vfork()
  - 轻量级进程允许父子进程共享许多数据结构:阻塞父进程等

2.7 Linux中的进程控制

- 写时复制技术允许父子进程能读相同的物理页。
- 如果两者有一个进程试图写一个物理页，内核就把这个页的内容拷贝到一个新的物理页，并把这个新的物理页分配给正在写的进程
- fork()与以往不同，实际开销只是复制父进程的页表以及给子进程创建唯一的PCB。
- 父进程的资源被设置为只读，当父进程或子进程试图修改某些内容时，内核才在修改前将该部分进行拷贝——写时复制

2.7 Linux中的进程控制

- 先通过复制页表项共享这个页面，将父、子进程可写虚拟内存页的页表项均标志为只读
- 当父或子进程向该内存页写入数据时，就会引起一次页面异常(缺页中断)，页面异常处理程序将重新分配一个物理页面，并完成真正的内容复制
- fork()调用：可理解为逻辑拷贝整个进程的地址空间，仅当试图修改页面才真正的拷贝

2.7 Linux中的进程控制

- Linux提供了三个与进程创建相关的系统函数 sys_fork()/sys_clone()/sys_vfork()
  - 其中，sys_clone()创建轻量级进程，必须指定要共享的资源
- exec()系统调用:执行一个新程序
- exit()系统调用:终止进程（进程也可以因收到信号而终止）

2.7 Linux中的进程控制

- 引入三种机制优化进程创建效率
  - 写时复制技术（Copy-On-Writing）
  - 轻量级进程
    - 允许父子进程共享页表、打开文件列表、信号处理等数据结构
    - 但每个进程应该有自己的程序计数器、寄存器集合、核心栈和用户栈
  - vfork()
    - 新进程可共享父进程的内存地址空间

2.7 Linux中的进程控制

- 在终端输入命令，由shell进程创建一个新进程
- 进程创建函数
  - pid_t fork(void);
  - pid_t vfork(void);
  - int clone(int (*fn)(void * arg), void *stack, int flags, void * arg);
    - 创建轻量级线程
- 三函数都调用同一内核函数do_fork( ) [/kernel/fork.c]

2.7 Linux中的进程控制

- 函数调用形式
  - do_fork(unsigned long clone_flag, unsigned long stack_start, struct pt_regs *regs, unsigned long stack_size, int _user *parent_tidptr, int _user *child_tidptr);
- 参数说明
  - clone_flag：子进程创建相关标志
  - stack_start：将用户态堆栈指针赋给子进程的esp
  - regs：指向通用寄存器值的指针
  - stack_size：未使用（总设为0）
  - parent_tidptr：父进程的用户态变量地址，若需父进程与新轻量级进程有相同PID，则需设置CLONE_PARENT_SETTID
  - child_tidptr：新轻量级进程的用户态变量地址，若需让新进程具有同类进程的PID，需设置CLONE_CHILD_SETTID

2.7 Linux中的进程控制

参数标志	含义
CLONE_FILES	父子进程共享打开的文件
CLONE_FS	父子进程共享文件系统信息
CLONE_IDLETASK	将PID设置为0（只供idle进程使用）
CLONE_NEWNS	为子进程创建新的命名空间
CLONE_PARENT	指定子进程与父进程拥有同一个父进程
CLONE_PTRACE	继续调试子进程
CLONE_SETTID	将TID回写至用户空间
CLONE_SETTLS	为子进程创建新的TLS
CLONE_SIGHAND	父子进程共享信号处理函数
CLONE_SYSVSEM	父子进程共享System V SEM_UNDO语义
CLONE_THREAD	父子进程放入相同的线程组
CLONE_VFORK	调用vfork()，所以父进程准备睡眠等待子进程将其唤醒
CLONE_UNTRACED	防止跟踪进程在子进程上强制执行CLONE_PTRACE
CLONE_STOP	以TASK_STOPPED状态开始进程
CLONE_SETTLS	为子进程创建新的TLS(thread-local storage)
CLONE_CHILD_CLEARTID	清除子进程的TID
CLONE_CHILD_SETTID	设置子进程的TID
CLONE_PARENT_SETTID	设置父进程的TID
CLONE_VM	父子进程共享地址空间

子进程创建CLONE参数标志说明

2.7 Linux中的进程控制

```
asmlinkage int sys_fork(struct pt_regs regs)
{
    return do_fork(SIGCHLD, regs.esp, &regs, 0);
}
```

- 说明
  - 子进程完全复制父进程的资源
  - 子进程的执行独立于父进程
  - 进程间数据共享需通过专门的通信机制来实现
- 返回值
  - 父进程执行fork()返回子进程的PID值
  - 子进程执行fork()返回0
  - 调用失败返回-1

2.7 Linux中的进程控制

```
asmlinkage int sys_vfork(struct pt_regs regs)
{
    return do_fork(CLONE_VFORK | CLONE_VM | SIGCHLD, regs.esp, &regs, 0);
}
```

- 说明
  - vfork()创建的子进程与父进程共享地址空间
    - 子进程作为父进程的一个单独线程在其地址空间运行
    - 子进程从父进程继承控制终端、信号标志位、可访问的主存区、环境变量和其他资源分配
    - 子进程对虚拟空间任何数据的修改都可为父进程所见
    - 父进程将被阻塞，直到子进程调用execve()或exit()
- 与fork()的关系
  - 功能相同，但vfork()不拷贝父进程的页表项
  - 子进程只执行exec()时，vfork()为首选

2.7 Linux中的进程控制

- copy_process()将task_struct的vfork_done设为NULL

```
p->vfork_done = NULL;
```

- do_fork() 将vfork_done指向一个特殊地址

```
if (clone_flags & CLONE_VFORK) {
  p->vfork_done = &vfork;
  init_completion(&vfork);      }
```

- 父进程阻塞直到子进程向其发送信号

```
if (clone_flags & CLONE_VFORK) {
  freezer_do_not_count();
  wait_for_completion(&vfork);
  freezer_count();
  if (unlikely (current->ptrace & PT_TRACE_VFORK_DONE)) {
    current->ptrace_message = nr;
    ptrace_notify ((PTRACE_EVENT_VFORK_DONE << 8) | SIGTRAP);
  }
}
```

vfork()调用的实现机制说明

2.7 Linux中的进程控制

- 调用mm_release()时，检查vfork_done是否为空
  - 若不为空，则向父进程发送信号

```
struct completion *vfork_done = tsk->vfork_done;
/* Get rid of any cached register state */
deactivate_mm(tsk, mm);
/* notify parent sleeping on vfork() */
if (vfork_done) {
  tsk->vfork_done = NULL;
  complete(vfork_done);
}
```

vfork()调用的实现机制说明

2.7 Linux中的进程控制

- 函数原型
  - int clone(int (*fn)(void *), void *child_stack, int clone_flag, void *arg);
- 参数说明
  - fn：待执行的程序
  - child_stack：进程所使用的堆栈
  - clone_flag：由用户指定，可以是多个标志的组合
  - arg：执行fn所需的参数
- 功能
  - 创建轻量级进程(LWP)的系统调用
  - 通过clone_flag控制

2.7 Linux中的进程控制

```
asmlinkage int sys_clone(struct pt_regs regs)
{
  unsigned long clone_flags;
  unsigned long newsp;
  int __user *parent_tidptr, *child_tidptr;

  clone_flags = regs.bx;
  newsp = regs.cx;
  parent_tidptr = (int __user *)regs.dx;
  child_tidptr = (int __user *)regs.di;
  if (!newsp)
    newsp = regs.sp;
  return do_fork(clone_flags, newsp, &regs, 0,
    parent_tidptr, child_tidptr);
}
```

- fn及arg参数的获取
  - do_fork()返回时,可通过进程切换机制（jmp+ret）使CPU从stack中获取返回地址，并得到fn地址

2.7 Linux中的进程控制

- CLONE_VM：父子进程共享内存描述符及所有页表
- CLONE_FS：父子进程共享文件系统信息
- CLONE_FILES：父子进程共享文件描述符表
- CLONE_SIGHAND：父子进程共享信号描述符
- CLONE_PTRACE：若父进程被跟踪，子进程也被跟踪
- CLONE_PARENT：父进程的real_parent登记为子进程的parent和real_parent
- CLONE_THREAD：子进程加入父进程的线程组
- CLONE_STOPPED：创建新进程但不运行之

clone()函数的常用CLONE标志

2.7 Linux中的进程控制

```
void* func(int arg){
  ......
}
int main(){
  int clone_flag, arg;
  ......
  clone_flag = CLONE_VM | CLONE_SIGHAND | CLONE_FS | CLONE_FILES;
  stack = (char *)malloc(STACK_FRAME);
  stack += STACK_FRAME;
  retval = clone((void *)func, stack, clone_flag, arg);
  ......
}
```

2.7 Linux中的进程控制

- sys_fork()和sys_clone()都通过调用do_fork()函数来完成进程的创建
- sys_fork(): 除了代码段外，父子进程不共享任何东西（各自有一份）
- sys_clone()：有选择地对父进程进行复制，可根据提供的clone_flags决定共享的内容
  - 不提供标志：sys_clone()与sys_fork()相同
  - 提供标志：根据标志进行相应进程的共享，就是传统的线程。如可共享页表、打开的文件列表、信号处理。因此， 利用sys_clone()轻量级进程允许父子进程共享许多数据结构。

2.7 Linux中的进程控制

sys_vfork()：

- 使用vfork()创建的新进程能够共享父进程的内存地址空间。使用复制指针的方法使子进程与父进程的资源实现共享。
- 父进程在这个过程中被阻塞，直到子进程退出或者通过exec()执行一个新的程序(租用父进程的地址空间)。因此，sys_vfork()保证子进程先运行，在它调用exec()或exit之后父进程才可能被调度运行. 可以理解为：vfork创建新进程的目的是exec()一个新进程。

sys_vfork()在写时复制技术出现之后，就基本上不用它了

2.7 Linux中的进程控制

```
sys_fork()
    |-do_fork()
        |-copy_process()
            |-dup_task_struct()
        |-wake_up_new_task()
```

2.7 Linux中的进程控制

◆阅读 do_fork()，了解大致程序流程

◆？？？子进程从哪里开始执行，它的返回值是什么？

2.7 Linux中的进程控制

◆大多情况下，子进程从fork()返回后都调用exec()函数来执行新的程序

- 进程调用exec()函数时，该进程完全由新程序替代，新程序从main()开始执行
- exec()并不创建新进程，前后进程ID不变，但用另外一个程序替代当前进程的正文、数据、堆栈等

2.7 Linux中的进程控制

```
If (result = fork() == 0){
    /* 子进程代码 */
  …
  if (execve("new_program",…)<0)
      perror("execve failed");
      exit(1);
}else if (result<0){
  perror("fork failed")
}
/* result==子进程的pid，父进程将会从这里继续执行*/
```

2.7 Linux中的进程控制

```
#include <unistd.h>

extern char **environ;

int execl( const char *path, const char *arg, ...);
int execlp( const char *file, const char *arg, ...);
int execle( const char *path, const char *arg , ..., char * const envp[]);
int execv( const char *path, char *const argv[]);
int execvp( const char *file, char *const argv[]);
```

指定被执行文件的路径名

- 用一个指定的可执行文件所描述的上下文代替进程的上下文
- ➔系统调用：execve()
  - sys_execve()

2.7 Linux中的进程控制

- 很多情况下，子进程从fork()返回后,很多会调用exec()来开始执行新的程序，从而可以让父进程和子进程执行不同的任务。
- 因此，子进程根本不需要读或者修改父进程拥有的所有资源。所以fork()中地址空间的复制依赖于Copy On Write技术，以降低fork()的开销。
- 当进程调用exec()函数族时，先找到并打开给定的可执行程序文件，装入并运行目标程序。
- 如果exec()成功，调用者进程将被覆盖，从新进程的入口地址开始执行。

2.7 Linux中的进程控制

- exec()并不创建新进程，前后进程ID是不变的。它是用另外一个程序替代了当前进程的正文，数据，堆和栈。
- exec()函数族的函数执行成功后不会返回，因为调用进程的实体，包括代码段，数据段和堆栈等都已经被新的内容取代，只留下进程ID等一些表面上的信息仍保持原样

2.7 Linux中的进程控制

Program A

```
int global;
main() {
  int local;

  if ((child=fork()) == -1) {
    创建失败
  }
  if (child == 0) {
    子进程
    if (execlp("B",...) == -1) {
      加载程序失败
    }
    global=local + 2;
    exit();
  }
  父进程
  global=local + 1;
  exit();
}
```

Program B

```
main() {
  exit();
}
```

2.7 Linux中的进程控制

- 分开这两个系统调用是有好处的
  - 比如服务器可以fork()许多进程,执行同一个程序
  - 有时程序只是简单的exec()，执行一个新程序
  - 在fork和exec之间，子进程可以有选择的执行一系列操作以确保程序以所希望的状态运行
    - 重定向输入输出
    - 关闭不需要的打开文件
    - 改变UID或是进程组
    - 重置信号处理程序
- 若单一的系统调用试图完成所有这些功能将是笨重、低效
  - 现有的fork-exec框架灵活性更强
  - 清晰，模块化强

2.7 Linux中的进程控制

- 创建子进程后，父进程的选择有以下几种：
  - (1) 父进程不理睬子进程，继续执行。如果子进程先于父进程消亡的话，则内核会发送一个信号，通知父进程。
  - (2)父进程暂停，睡眠，等待子进程结束后继续运行
  - (3)父进程自行结束，使用exit（）。

2.7 Linux中的进程控制

- 父进程调用wait()类系统调用检查子进程是否终止
- 父进程调用wait()后进入阻塞队列中。当子进程结束时会产生一个终止状态字，系统内核再向父进程发出SIGCHILD信号。当接收到信号时，父进程提取子进程的终止状态字，从wait()返回到原程序。
- Wait()系统调用的作用
  - 获取子进程终止的消息
  - 清除子进程的所有独占资源

2.7 Linux中的进程控制

- sys_wait4()算法的基本思路：
  - 搜索当前进程的所有子进程，
    - 如果父进程没有子进程，则出错返回。
    - 如果找到一个满足要求的子进程，但都不处于僵死态，则父进程或者睡眠等待，以后由相应信号唤醒，或者立即返回
    - 如果找到一个满足要求的子进程，则返回子进程的进程号，把子进程的CPU使用时间等加到父进程上，释放子进程占用的 task_struct和系统堆栈，以供新进程使用（将其释放）。
- release()的唯一参数是要回收进程的task_struct结构，与该结构相关的大部分资源已经在exit中释放掉了。

2.7 Linux中的进程控制

- 函数原型
  - pid_t wait(int *status);
  - pid_t waitpid(pid_t pid, int *status, int options);
- 说明
  - 均通过wait4()系统调用实现
  - 进程终止时，会向父进程发送SIGCHLD信号
- 调用wait( )和waitpid( )的进程的可能状态
  - 阻塞
    - 如果子进程还在运行
  - 正常返回
    - 返回子进程的终止状态（其中一个子进程终止）
  - 出错返回
    - 没有子进程

2.7 Linux中的进程控制

- 撤销时机
  - 主动撤销：执行完代码，通知内核释放进程的资源
  - 被动撤消：内核强迫杀死进程
    - 进程接收到一个不能处理或忽视的信号
    - 在内核态产生一个不可恢复的CPU异常，而内核此时正代表该进程在运行
- 撤消相关进程状态
  - 进程已死，但必须保存它的描述符，在得到通知后才可以删除
  - 僵死状态，表明进程已死，但需要等待父进程删除（如通过wait()）

2.7 Linux中的进程控制

- 进程终止
  - 释放进程占有的大部分资源
  - 进程终止的一般方式是exit()或_exit()系统调用
    - 该系统调用可由编程者明确地在代码中插入
    - 控制流到达主过程最后一条语句时，自动执行exit()
  - 信号机制
- 进程删除
  - 彻底删除进程的所有数据结构
  - 父进程调用wait()类系统调用获知子进程合法终止后，即可删除该进程。

2.7 Linux中的进程控制

- 进程终止函数原型
  - exit(int status)
  - _exit(int status)
- status说明
  - 正常结束时返回0，否则表示出错信息
- 两函数共性
  - 都通过do_exit() 系统调用

```
asmlinkage long sys_exit(int error_code)
{
    do_exit((error_code&0xff)<<8);
}
```

- 两函数区别
  - _exit()直接使进程停止工作，做清除和销毁工作
  - exit()在调用do_exit()系统调用前检查文件打开情况，清理I/O缓冲

2.7 Linux中的进程控制

- 进程终止时要作的几件事
  - 通知父进程自己将要终止，通常利用信号通知机制，这可以通过exit函数实现
  - 父进程要有相应的机制响应子进程的终止信号。通过信号处理程序和wait4()函数实现
  - 当父进程在子进程前终止时，要为其找一个父进程，如init进程
- 进程的终止分2步：
  - 自己调用函数exit()将自己的状态变为僵死状态，而后向父进程发送终止信号。
  - 由父进程调用wait()函数，回收僵死状态的子进程，将其从内存中彻底清除。

2.7 Linux中的进程控制

- 进程终止的一般方式是exit()系统调用
  - 这个系统调用可能由编程者明确的在代码中插入
  - 另外，在控制流到达主过程[C中的main()函数]的最后一条语句时，执行exit()系统调用
- 内核可以强迫进程终止
  - 当进程接收到一个不能处理或忽视的信号时
  - 当在内核态产生一个不可恢复的CPU异常而内核此时正代表该进程在运行

2.7 Linux中的进程控制

- 进程终止使用exit系统调用

```
asmlinkage long sys_exit(int error_code)
{
    do_exit((error_code&0xff)<<8);
}
```

- 删除进程
  在父进程调用wait()类系统调用检查子进程是否合法终止以后，就可以删除这个进程
- 若子进程包含终止代号，则父进程通过release()释放僵死进程的描述符
  - 释放进程id
  - 从进程链表中删除进程描述符
  - 释放存放进程描述符的内存区

2.7 Linux中的进程控制

- C编译程序总是把exit()插入到main()的最后一条语句之后，exit()调用_exit()系统调用(kernel/exit.c中)
- _exit() 调用do_exit()释放进程所占资源，终止进程
- 函数do_exit()释放进程占用的大部分资源
  - do_exit()调用__exit_mm(),__exit_files(),__exit_fs()以及__exit_sighand()
- 进程进入TASK_ZOMBIE状态（成为僵尸进程），内核调用exit_notify()通知父进程料理后事。当父进程调用了wait（）函数族后，僵尸进程所占有的task_struct空间才被回收。
- 如果父亲先于子进程结束，则由父亲的父亲进程或者init进程来完成回收任务。

2.7 Linux中的进程控制

- do_exit()函数删除内核对终止进程的大部分引用：
  - 信号量队列中的进程描述符
  - 删除进程描述符中与分页、文件系统、打开文件描述符和信号处理相关数据结构
  - 减小进程所用模块的引用计数
- 将进程描述符的exit_code字段设置为终止代号
- 更新父子进程亲属关系，强制将自己的子进程作为其它某个进程的子进程，以等待该父进程来删除
- 调用schedule()进行调度

2.7 Linux中的进程控制

- 进程运行结束时要释放相应的资源，通过exit()调用实现（显式或隐式）
- exit()实现时调用了do_exit()完成以下工作
  - Task_struct中标志成员设为：PF_EXITING
  - 调用__exit_mm()
  - 调用sem__exit()
  - 调用__exit_files(),__exit_fs(),exit__name_space,exit_sighand
  - 退出代码替换为EXIT()提供的代码
  - 调用Exit_notify()向父进程发信号，（标为ZOOMBIE)
  - 调用shedule切换到其他进程

2.7 Linux中的进程控制

- 进程终止时，删除进程描述符
  - 父进程得知子进程终止的消息后才能删除子进程的描述符。（思考：若父进程异常终止，将会发生什么情况）
  - 父进程WAIT()，返回时可以根据代码做相应动作（或者IGNORE），动作完成后真正释放描述符（调用release_task)
    - 调用free_uid:进程记数
    - Unhash_process()：删除pidhash中的项同时删除task_list中的项
    - 若有trace删除ptrace_list对应项
    - 调用put_task_struct释放描述符，（内核栈、threadinfo所占的页、slab高速缓存。

2.8 Linux线程实现机制

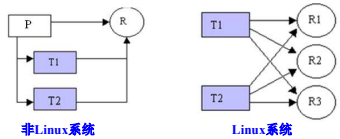
- Linux的线程描述机制
- Linux的线程分类
- Linux线程的创建

2.8 Linux线程实现机制

- 其他多线程操作系统
  - 通过两种数据结构来表示线程和进程：进程和线程表项
  - 每个进程可以指向若干个线程，调度器在进程的时间片内再调度线程
- Linux操作系统
  - 从内核角度，没有线程概念，所有线程当做进程来实现
  - 线程被视为与其他进程共享某些资源的进程，都有自己的进程描述符task_struct
  - 内核没有针对线程的调度算法或数据结构

2.8 Linux线程实现机制

- 举例（包含两个线程的进程）
  - 对于非Linux系统
    - 有一个包含两个不同线程指针的进程描述符
    - 每个线程指针描述其独占资源
  - 对于Linux系统
    - 创建两个进程并分配两个普通task_struct结构
    - 建立两个进程时只要指定它们共享的某些资源



非Linux系统　　Linux系统

2.8 Linux线程实现机制

- Linux内核中没有线程的概念
  - 没有针对所谓线程的调度策略
  - 没有数据结构用来表示一个线程
  - 一般线程的概念在linux中只是表现为一组共享资源的进程(每个这样的进程都有自己的进程描述符)
- 在其他系统中(比如windows)
  - 线程是实实在在的一种运行抽象，提供了比进程更轻更快的调度单元
  - 在Linux中“线程”仅仅是表示多个进程共享资源的一种说法

2.8 Linux线程实现机制

- 用户线程
  - 存在于用户空间中，通过线程库来实现
  - 线程创建和调度都在用户空间进行，以进程为单位调度
  - 特点
    - 同一进程内的线程切换不需要转换到内核
    - 系统调度阻塞问题，不能充分利用多处理器
- 内核线程
  - 在内核空间内执行线程的创建、调度和管理，慢于用户线程的创建和管理
  - 支持多处理器，支持用户进程中的多线程、内核线程切换的速度快
  - 对用户的线程切换来说，系统开销大

2.8 Linux线程实现机制

- Linux线程的创建函数调用形式
  - int clone(int (*fn)(void * arg), void *stack, int flags, void * arg);
  - int pthread_create(pthread_t *restrict tidp, const pthread_attr_t *restrict attr, void *(*start_rtn)(void), void *restrict arg);
    - tidp:新线程的线程描述表指针
    - attr:为新线程定义不同属性(如栈尺寸)默认为NULL
    - 第三个和第四个参数指定执行的函数start_rtn和传递给函数的参数arg
- 两者区别
  - clone()：创建内核支持的用户线程，对内核可见且由内核调度
  - pthread_create()：由基于POSIX标准的线程库创建的用户线程
    - 但在Linux里， pthread_create()最终调用clone()实现

2.8 Linux线程实现机制

- Linux内核线程独立运行在内核空间的标准进程，支持内核在后台执行一些操作
  - 刷新磁盘高速缓存
  - 交换出不用的页框
  - 维护网络链接等待
- 与普通进程的区别
  - 只运行在内核态，内核线程没有独立的地址空间（mm指针被设置为NULL）
  - 每个内核线程执行一个单独的内核函数
  - 只使用大于PAGE_OFFSET的线性地址空间

2.8 Linux线程实现机制

- 没事闲逛的0号进程——从无到有诞生的第一个内核线程，执行cpu_idle()函数（省电又少热）。
- 既是内核线程也是1号用户进程的init。
- Init内核线程被调度到时，执行内核函数init()，将会四次调用kernel_thread，为常规内核任务初始化四个内核线程：
- kflushd (即bdflush)线程：刷新“脏”缓冲区中的内容到磁盘以归还内存。
- Kupdate线程：刷新旧的“脏”缓冲区中的内容到磁盘以减少文件系统不一致的风险。
- Kpiod线程：把属于共享内存映射的页面交换出去。
- Kswapd线程：执行内存回收功能。
- init内核函数调用execve()，装入用户态下的可执行程序sbin/init，用户态下的第1个进程init就产生了。

本章知识点小结

- 理解和掌握Linux的进程/线程描述机制
  - Linux进程描述符
  - Linux进程的系统堆栈结构
- 理解和掌握Linux进程的内核管理机制
  - 内核链表结构
  - Linux的进程家族结构
  - Linux进程系统启动过程
- 理解和掌握Linux进程创建与实现机制
  - fork()、vfork()、clone()的使用场景及实现机制的差别
  - do_fork()的工作机理
- 理解和掌握Linux线程创建与实现机制
  - Linux线程的实现机制
  - Linux用户线程/内核线程的创建方法

作业

- Linux怎样描述进程？Linux有哪几种进程状态？画出Linux进程状态转换图。
- 查阅资料，在Linux2.6中，进程控制块如何存放？为什么？
- 0号进程来自哪里？在系统启动正常后，0号进程在什么情况下才被调度执行？1号进程的主要作用？
- 什么是僵尸进程？内核怎样处理僵尸进程？
- 创建进程的系统函数有哪几个？它们有什么区 别？
- 简述进程终止的大致过程。
- Linux为什么要引入pidhash表？
- 在传统的UNIX系统中，创建子进程时会复制父进程的所有资源，代价比较高，现代UNIX系统中引入了哪几项技术来解决这个问题？
- 进程描述符放在动态内存中，而且和内核态的进程栈放在一个独立的8KB的内存区中。这样做有什么好处？

第3章 Linux进程调度

- Linux进程调度策略
- Linux进程调度依据
- Linux进程调度时机
- Linux进程调度实现
- Linux 2.4进程调度分析
- Linux 2.6进程调度特点

3.1 Linux进程调度策略

- 什么是调度策略（scheduling policy）？它是一组规则，它们决定什么时候以怎样的方式选择一个新进程运行
- 分时技术回顾
  - 允许多个进程“并发”运行
  - CPU的时间被划分成“片”，给每个可运行进程分配一片
  - 在单处理器上，任何时刻只能运行一个进程，当一个并发执行的进程时间片用完时（到期）还没有终止，就可以进行进程调度
- 分时依赖于时钟中断，对进程透明

3.1 Linux进程调度策略

- 如何动态管理和维护就绪进程?
- 进程调度相关数据结构设计?
- 如何设定进程优先级?
  - 进程类型：实时进程 *vs.*交互式进程 *vs.* 普通进程
  - 进程动态/动态优先级的定义
  - 进程动态优先级调整策略及调整时机
- 如何设置/调整进程CPU时间片?
  - 初始值设置原则
  - 时间片修改/调整位置、时机及策略（进程创建/运行/退出）
- 进程调度/切换依据?
  - 如何选择候选进程
  - 与静态/动态优先级、进程类型、时间片等的关系
- 多处理器系统SMP?
  - CPU选择策略
  - 负载均衡与迁移机制
- 进程处理调度流程的设计?

调度策略设计的技术难点

3.1 Linux进程调度策略

- 调度方法
  - 基于分时技术的抢占式调度
    - CPU时间被划分成“时间片”，为每个可运行进程分配一个时间片
    - 分时依赖于时钟中断，对进程透明
- 调度策略
  - 进程分类
    - 普通进程
    - 实时进程
  - 优先级
    - 静态优先级
    - 动态优先级

3.1 Linux进程调度策略

- Linux进程的优先级：计算得到、动态变化
  - 根据特定的算法计算出进程的优先级，是动态变化的，用一个值表示。
  - 调度程序会根据进程的行为周期性调整进程优先级
    - 较长时间未分配到CPU的进程，通常↑
    - 已经在CPU上运行了较长时间的进程，通常↓

3.1 Linux进程调度策略

- Linux既支持普通的分时进程，也支持实时进程，实时进程优先于普通进程。
- 为实时进程赋予远大于普通进程的固定的优先级。普通进程则以保证所有进程能平均占用处理器。
- 普通进程(非实时进程)
  - 动态优先级调度：由调度程序计算出来,周期性地修改进程的优先级（避免饥饿）
- 实时进程
  - 优先级是静态优先级，即由用户预先指定，以后不会改变
- 调度器是以进程的优先级为调度依据的，它总是选择目前就绪队列中优先级最高的进程作为候选进程next。

3.1 Linux进程调度策略

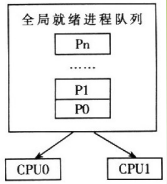
- 静态优先级：由用户确定，只能由用户修改。系统调用sys_setpriority( )改变静态优先级，
  - 实时进程采用静态优先级， 即由用户预先指定，以后不会改变；
- 动态优先级:只要进程占用CPU，它的优先级就随着时间的流失而不断减小。
  - 非实时进程的优先级采用动态优先级， 它由调度程序计算出来。

3.1 Linux进程调度策略

- 实时进程
  - SCHED_FIFO：先进先出。当前实时进程将一直占用CPU 直至自动退出，除非有更紧迫的、优先级更高的实时进程需要运行时，它才会被抢占CPU；每个可执行进程按各自在运行队列中的顺序执行，并且顺序不能变化。
  - SCHED_RR：时间片轮转。它与其它实时进程以RR方式去共同使用CPU。
- 非实时进程
  - 普通进程，SCHED_OTHER，动态优先级。
  - 时间片：用于标识一个进程在被抢占前可以运行的时间
- 用户可以通过系统调用sched_setscheduler()函数改变自己的调度策略。
- 一旦进程变为实时进程，它的子孙进程也是实时进程。

3.1 Linux进程调度策略

- 全局就绪进程队列：由所有就绪态进程构成的全局链表的表头
  - 当前正在运行的进程也在其中（但idle_task除外）
  - 调度器从中选取最适合调度的进程投入运行
- 链表由一个读/写自旋锁保护
  - 支持多处理器并行访问
  - 但对写操作必须互斥访问



3.1 Linux进程调度策略

进程描述符中调度相关成员

变量名	说明
state	进程当前状态
need_resched	布尔值，表示该进程是否需要申请调度
policy	进程调度类型
rt_priority	进程实时优先级
nice	用户控制的进程优先级
priority	静态优先级，实时进程忽略该成员
counter	当前时间片内该进程的剩余运行时间
cpus_allowed	以位向量形式表示可执行该进程的CPU
cpus_runnable	以位向量形式表示当前运行该进程的CPU
processor	本进程当前（或最近）所在的CPU
thread	用于保存进程执行环境（各寄存器的值及IO操作许可权映射表）

3.1 Linux进程调度策略

◆基本思想
- 根据进程价值及CPU时间需求对进程分级
  - 高优先级进程先运行，低优先级后运行
  - 相同优先级进程按轮转或FIFO方式进行调度

◆Linux实现
- 基于动态优先级的调度算法
  - 设置基本优先级，并根据需要动态调整优先级
    - 较长时间未分配到CPU的进程，提高优先级
    - 已经在CPU上运行较长时间的进程，降低优先级
- 两组独立的优先级范围
  - nice（-20~19）：确定分配给进程的时间片大小
  - rt_priority（0~99）：实时优先级，实时进程的优先级高于普通进程

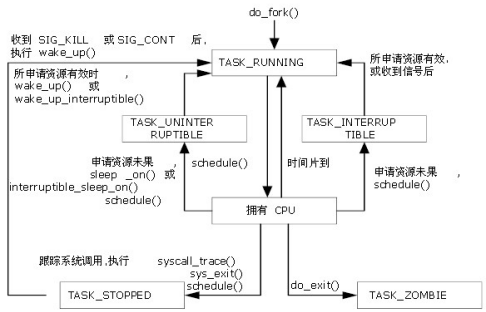
3.1 Linux进程调度策略

◆域成员名
- state

◆状态分类
- 运行态/就绪态
  - TASK_RUNNING：正在运行或处于就绪只等待CPU调度
- 被挂起状态
  - TASK_INTERRUPTIBLE：可被信号或中断唤醒进入就绪队列
  - TASK_UNINTERRUPTIBLE：等待资源，不可被其他进程中断
  - TASK_STOPPED：被调试暂停，或收到SIGSTOP等信号
- 不可运行态
  - TASK_ZOMBIE：退出而暂未被父进程收回资源的“僵尸”进程

◆说明
- 调度器主要处理可运行和被挂起两种状态下的进程

3.1 Linux进程调度策略



3.1 Linux进程调度策略

- 静态优先级：priority
  - 表示分配给进程的时间片
  - 指明在被迫和其他进程竞争CPU之前，进程允许的最大时间片
  - 只能由用户进行修改，不随时间而改变，一般通过nice设定
- 动态优先级：counter
  - 进程拥有CPU时随时间不断减小
  - 指明在这个时间片中所剩余的时间量
  - 当小于0时，标记进程重新调度
- 实时优先级：rt_priority
  - 确定实时进程的调度顺序，较高权值进程优先于较低权值进程
  - 非实时进程的优先级为0，因此实时进程总优先于非实时进程

3.1 Linux进程调度策略

- 普通进程
  - 基本思想：动态优先级调度
    - 通过更新counter值，周期性修改进程优先级（避免饥饿）
  - 基本过程
    - counter变为0时，用priority对counter重新赋值。
    - 所有可运行状态进程的时间片都用完后才对counter重新赋值(*)。
    - 进程运行过程中，counter值减小为其它进程提供运行机会。
    - 该机制相当于优先级在动态变化，所以称之为动态优先调度 。

3.1 Linux进程调度策略

- 实时进程
  - 基本思想：静态优先级策略
    - counter只用来表示该进程的剩余时间片
    - 不作为衡量其是否值得运行的标准（与普通进程的区别）

3.1 Linux进程调度策略

- 动态调整优先级及时间片长度
  - 提高交互式程序的优先级，使其运行得更频繁（*）
  - 调度程序为交互式程序提供较长的默认时间片
  - 进程不一定要一次性使用完所有的时间片，可以分批使用，从而确保尽可能长时间地处于可运行状态
  - 没有时间片的进程不会再投入运行，除非等到其他所有进程都耗尽其时间片（*）

3.1 Linux进程调度策略

- 域成员名
  - policy
- 策略分类
  - 实时进程
    - SCHED_FIFO：先进先出调度，除非有更高优先级进程申请运行，否则该进程保持运行至退出才让出CPU
    - SCHED_RR：轮转式调度，该进程被调度下来后将被放置于运行队列的末尾，以保证其他实施进程有机会运行
  - 普通进程
    - SCHED_OTHER：常规分时调度策略
  - 其他
    - SCHED_YIELD：置位时表示主动放弃CPU

3.1 Linux进程调度策略

- **policy：调度策略,区分实时进程和普通进程，可以通过sched_setscheduler( )来改变调度策略。**

```
/*
 * Scheduling policies
 */
#define SCHED_OTHER    0
#define SCHED_FIFO     1
#define SCHED_RR       2

/*
 * This is an additional bit set when we want to
 * yield the CPU for one re-schedule..
 */
#define SCHED_YIELD    0x10
```

普通的分时进程

先入先出的实时进程

循环轮转的实时进程

当一个进程自动放弃运行时设置

3.2 Linux进程调度依据

- **基本思想**
  - **选择权值（weight）最大的进程**
- **权值计算：goodness()函数**
  - **普通进程权值**
    - 与policy，priority，counter， nice等相关
    - weight = p->counter
  - **实时进程权值**
    - 与policy，rt_priority等相关
    - weight = 1000 + rt_priority

3.2 Linux进程调度依据

权值的计算—goodness()函数

- **功能**
  - **确定就绪态进程的调度顺序**
  - **运行态进程每次执行schedule()时都要调用该函数**
- **返回值类型**
  - **<=1000：只能赋给普通进程**
    - 在单处理器下，普通进程goodness值只使用这个范围底部的一部分
    - 在SMP情况下，调度程序优先照顾等待同一处理器的进程
  - **>1000：只能赋给实时进程**
    - 实时进程goodness值的范围从1001到1099
- **说明**
  - **goodness() 不会返回负值**
    - 由于idle进程的counter值为负，所以如果使用idle进程作为参数调用goodness，将返回负值，但这是不会发生的

3.2 Linux进程调度依据

权值影响因素分析

- **普通进程**
  - **进程当前时间片内所剩的tick数**
    - 既代表进程优先级，又反映进程的“欠运行程度”
  - **进程上次是否在当前CPU上运行**
    - 若是，权值增加一个常量：(weight += PROC_CHANGE_PENALTY)
  - **切换是否需要切换内存**
    - 若不需要，则权值加1(weight += 1)
  - **用户可控优先级nice**
    - nice越小则权值越大：(weight += 20 - p->nice)
- **实时进程**
  - **权值大小仅由rt_priority值决定**
    - weight = 1000 + p->rt_priority
    - 基准值1000使实时进程的权值比所有非实时进程都要大
    - 只要就绪队列存在实时进程，调度器都将优先满足其运行需要

3.2 Linux进程调度依据

goodness()函数分析

```
static inline int goodness(struct task_struct * p, int this_cpu, struct
mm_struct *this_mm) {
    int weight;
    weight = -1;
    if (p→policy & SCHED_YIELD)
        goto out;
    if (p → policy == SCHED_OTHER) {/*普通进程*/
        weight = p → counter;
        if (!weight) /* 进程的时间片已经用完，则直接转到标号out*/
            goto out;

#ifdef CONFIG_SMP
/*在SMP情况下，如果进程将要运行的CPU与进程上次运行的CPU是一
样的，则最有利，因此，假如进程上次运行的CPU与当前CPU一致的话
，权值加上PROC_CHANGE_PENALTY，这个宏定义为20。*/
if (p → processor == this_cpu)
        weight += PROC_CHANGE_PENALTY;
#endif
```

3.2 Linux进程调度依据

goodness()函数分析

```
/*进程p与当前运行进程是同一个进程的不同线程，或者是
共享地址空间的不同进程，优先选择，权值加1*/
        if (p→mm == this_mm || !p → mm)
            weight += 1;
        weight += 20 - p → nice;
        goto out;
    }
    weight = 1000 + p → rt_priority; /*实时进程*/
out:
    return weight;
/* 返回值作为进程调度的唯一依据，谁的权值大，就调度
谁运行*/
}
```

3.2 Linux进程调度依据

- **调度算法将CPU时间划分为时期（epoch）**
  - **在一个单独时期内，每个进程分配一个指定时间片**
    - 一个进程用完其时间片时，将会被抢占
    - 只要进程的时间片没有用完，就可以被多次调度运行
  - **所有进程用完其时间片，一个时期才结束**
  - **此时才重新计算所有进程的时间片，并开始一个新时期**
- **Linux的时间单位**
  - **时钟滴答（10ms）**
    - 时间片即指时间滴答数
    - 例如，若priority为20，则分配给该进程的时间片就为20个时钟滴答，即20*10ms=200ms

3.2 Linux进程调度依据

- **时间片的长短对系统性能非常关键，时间片大小的选择总是一种折衷，既不能太长也不能太短**
  - **太短**
    - 频繁切换会造成系统开销过大
    - 假如切换时间为1ms，时间片设置为1ms，则没空执行进程
  - **太长**
    - 几乎每个进程都一次运行完，并发的概念基本消失
    - 导致系统对交互的响应表现欠佳
    - 普通进程需要等待很长时间才能运行
- **Linux采取单凭经验的方法**
  - **选择尽可能长的时间片，同时能保持良好响应时间**

3.2 Linux进程调度依据

- **为非SCHED_FIFO调度策略的每个进程提供一个运行的时间配额counter**
  - **剩余的时间片**
  - **每个进程的counter初值与nice值有关**
  - **每次时钟中断(tick)发生，时间片都会减1，直到为0（则请求调度）**
  - **新时期（epoch）开始时，将重新计算**
  - **创建新进程时，子进程继承父进程的一半剩余时间片**

```
p->counter = (current->counter + 1) >> 1;
current->counter >>= 1;
```

3.2 Linux进程调度依据

- **除init进程外，所有其他进程都通过do_fork()创建**
- **do_fork()与新进程调度相关的属性设置**
  - **state**
    - TASK_UNINTERRUPTIBLE
  - **cpus_runnable**
    - 全1，未在任何cpu上运行
  - **processor**
    - 与父进程的processor相同
    - 优先在创建子进程的CPU上运行
  - **counter**
    - 父进程counter值加1的一半，父进程counter也减半
    - 子进程结束运行时，剩余时间片也将归还给父进程，以免父进程因创建子进程而遭受时间片的损失

3.2 Linux进程调度依据

counter值的更新

- **每次产生时钟中断时，递减当前进程的counter值**
- **更新函数：update_process_times()**
  - **调度策略为SCHED_RR的进程**
    - 一旦counter降到0，从runqueue中当前的位置移到队列的末尾，同时恢复其最初的时间配额
  - **调度策略为SCHED_OTHER的进程**
    - 当所有就绪进程的counter值降为0时，重新计算并设置每个进程的时间配额。

3.2 Linux进程调度依据

```
void update_process_times(int user_tick)
{
    struct task_struct *p = current;
    int cpu = smp_processor_id(), system = user_tick ^ 1;

    update_one_process(p, user_tick, system, cpu);
    if (p->pid) {
        if (--p->counter <= 0) {
            p->counter = 0;
            /*
             * SCHED_FIFO is priority preemption, so this is
             * not the place to decide whether to reschedule a
             * SCHED_FIFO task or not - Bhavesh Davda
             */
            if (p->policy != SCHED_FIFO) {
                p->need_resched = 1;
            }
        }
        if (p->nice > 0)
            kstat.per_cpu_nice[cpu] += user_tick;
        else
            kstat.per_cpu_user[cpu] += user_tick;
        kstat.per_cpu_system[cpu] += system;
    } else if (local_bh_count(cpu) || local_irq_count(cpu) > 1)
        kstat.per_cpu_system[cpu] += system;
} ? end update_process_times ?
```

3.2 Linux进程调度依据

非实时进程的couter值更新代码

```
repeat_schedule:
    /*
     * Default process to select..
     */
    next = idle_task(this_cpu);
    c = -1000;
    list_for_each(tmp, &runqueue_head) {
        p = list_entry(tmp, struct task_struct, run_list);
        if (can_schedule(p, this_cpu)) {
            int weight = goodness(p, this_cpu, prev->active_mm);
            if (weight > c)
                c = weight, next = p;
        }
    }

    /* Do we need to re-calculate counters? */
    if (unlikely(!c)) {
        struct task_struct *p;

        spin_unlock_irq(&runqueue_lock);
        read_lock(&tasklist_lock);
        for_each_task(p)
            p->counter = (p->counter >> 1) + NICE_TO_TICKS(p->nice);
        read_unlock(&tasklist_lock);
        spin_lock_irq(&runqueue_lock);
        goto repeat_schedule;
    }
```

3.2 Linux进程调度依据

- **当所有就绪进程的动态优先级都为0 时，系统将重新计算进程的动态优先级，这既包括了就绪态进程，也包括了等待态进程。**
- **计算方法为：将每个进程的动态优先级的值右移一位，再加上该进程的静态优先级的值，就得到了该进程的动态优先级的新值。**

nice缺省为0（在-20到19之间选择）

```
p->counter = (p->counter >> 1) + NICE_TO_TICKS(p->nice);

#define NICE_TO_TICKS(nice)  (TICK_SCALE(20-(nice))+1)

#define TICK_SCALE(x)    ((x) >> 2)
```

通常，基本时间片的值 为6，由于时钟中断大约10ms左右，因此基本时间片的长度大约60ms

处于等待态的进程的动态优先级通常会逐渐增加，也可能不变，但不可能减小。

3.2 Linux进程调度依据

- **对于就绪态进程:由公式可看出，由于其动态优先级都为0，计算结果为nice值。**
- **对于等待态进程:此时它们的动态优先级都不为0，所以，计算结果等待态进程的动态优先级会大于静态优先级，但决不会比静态优先级两倍更大。因此，等待态进程会有较高的优先级。**
- **因此，I/O 较多的进程会有较高的优先级(linux偏向I/O密集型进程)；计算较多的进程因其占用CPU 多使优先级会较低。**

3.3 Linux进程调度时机

主动调度

- **当前进程所需资源无法满足而必须立即阻塞时，直接调用schedule()。**
- **步骤**
  - **将current 进程插入到合适的等待队列中。**
  - **将current进程当前状态改变为TASK_INTERRUPTIBLE或TASK_UNINTERRUPTIBLE**
  - **调用schedule()**
  - **检查所需资源是否可用**
    - 若不可用，转到第2 步
  - **一旦所需资源可用，将current 从等待队列中删除。**
- **说明**
  - **长时间执行重复任务的很多设备驱动程序也直接调用调度程序**
  - **在每次反复循环中，驱动程序都检查need_resched 的值，如果必要，调用schedule()主动放弃CPU。**

3.3 Linux进程调度时机

- 将当前进程的need_resched设为1来请求调度
- 执行时机 被动调度
  - 当前进程用完CPU时间片时
    - 通过update_process_times()函数进行
  - 一个进程被唤醒，且优先级高于当前进程时
    - 在wake_up_process()函数中调用reschedule_idle()
    - if (goodness(current, p) > goodness(current, current))
    - current->need_resched = 1;
  - 调用sched_setscheduler()或sched_ yield()系统调用时
  - 每次进入用户态进程前
    - 都会检查need_resched，决定是否调用函数schedule()
  - 系统调用执行结束，控制由核心态返回用户态前
    - 在ret_from_sys_call入口检查当前进程的need_resched

3.3 Linux进程调度时机

被动调度

- ret_from_sys_call入口代码

```
ENTRY(ret_from_sys_call)
cli
cmpl $0, need_resched(%ebx)   #ebx中存放current指针
jne reschedule
……
reschedule:
call SYMBOL_NAME(schedule)
jmp ret_from_sys_call   #反复查询need_resched
```

3.3 Linux进程调度时机

调度器相关函数和宏

- schedule()
  - 进程调度主函数
- switch_to()
  - 由schedule()调用，进行上下文切换的宏
- reschedule_idle()
  - 在SMP系统中
    - 若被切换进程仍可运行，则调用reschedule_idle()重新调度
    - 选择一个空闲或执行低优先级进程的CPU来运行被切换进程
- goodness()
  - 优先级计算函数，选择一个最合适的进程投入运行

3.3 Linux进程调度时机

schedule()函数主要工作

- 从待切换进程的processor域读取CPU标识，存入this_cpu
- 初始化sched_data变量，指向当前处理器schedule_data结构
- 调用goodness()函数选取合适进程
- 若需要，重新计算动态优先权
- 将sched_data → last_schedule值修改为当前时间
- 调用switch_to()，切换进程上下文
- 调用schedule_tail()
  - 若prev所指向进程仍可运行且不是空闲进程
    - schedule_tail()调用reschedule_idle()为其选择合适CPU
- 阅读schedule

3.3 Linux进程调度时机

schedule()主要工作

（1）选择进程：当CPU 空闲时，调度子程序扫描运行队列所有活动进程，从运行队列中选择一个合适进程运行：

- 如果执行队列中有实时进程，则选择一个优先级最高（goodness（）计算的值最大）的进程（可保证实时进程总比普通进程优先执行）。
- 如果运行队列中的进程有相同优先级，则耗时多(counter值较大)的进程比耗时少的进程优先调度。
- 如果优先级相同且counter 值也相等，则位于队列前面的进程被优先调度。

3.3 Linux进程调度时机

schedule()主要工作

（2）切换进程

- 当CPU 从运行队列中选中一个合适的进程后，就按照task_struct 结构中的信息进行进程上下文切换；
- 首先把当前进程放到适当的等待队列里
- 把当前进程的状态设为 TASK_INTERRUPTIBLE或者 TASK_UNINTERRUPTIBLE
- 调用schedule()，让新的进程运行

3.4 Linux进程调度实现

- 实现：具体是通过sched.c文件来完成的。它的大致流程分成：
- (1) 进程的选择和进程的切换。
  - 其中进程的选择就是遍历运行队列，找到一个优先级最高的进程。部分代码如下：

```
repeat_schedule:
    next = init_tasks[this_cpu];
    c = -1000;
    list_for_each(tmp, &runqueue_head) {
        p = list_entry(tmp, struct task_struct, run_list);
        if (p->cpus_runnable & p->cpus_allowed & (1 << this_cpu)) {
            int weight = goodness(p, this_cpu, prev->active_mm);
            if (weight > c)
                c = weight, next = p;
        }
    }
```

3.4 Linux进程调度实现

- 其中的goodness（）是用来计算当前进程的优先级的,用来衡量一个处于可运行状态的进程值得运行的程度。它给每个处于可运行状态的进程赋予一个权值（weight）。
- 调度程序以这个权值作为选择进程的唯一依据。选择一个权值（weight）最大的进程
- 它的第一个参数就是待估进程的描述符，返回值比较真实的反映了待估进程“值得运行的程度”。
- c = 0则表示待估进程的时间片已经用完。

3.4 Linux进程调度实现

进程调度中的prev指针

- 功能
  - 保存当前进程，即可能被切换出去的进程
  - 对实时进程(SCHED_RR)
    - 仅当进程时间片结束后才切换到别的进程，此时将根据nice值重置counter，并将该进程置于就绪队列的末尾
  - 对处于TASK_INTERRUPTIBLE、有信号需处理的进程
    - 不立即执行该进程，而是将该进程置为就绪态，参与goodness选择
  - prev不处于就绪态，也不处于有信号等待处理的挂起态
    - 将其从就绪队列中删除
    - 此后，除非有唤醒操作将其重新放回就绪队列，否则不参与调度
  - 被动方式启动调度器工作时，当前进程的need_resched属性置位
    - schedule()将清除该位，表示该进程已在调度器中得到处理

3.4 Linux进程调度实现

进程调度中的next指针

- 功能
  - 保存待切换入的新进程
  - 新进程刚好是需切换出去的老进程，不需进行进程切换
- 新进程运行环境实际上主要指内存
  - task_struct与调度相关的内存属性：mm和active_mm
    - 前者指向进程所拥有的内存区域，后者指向进程实际使用的内存
  - 对于大多数进程，mm和active_mm相同
    - 但内核线程没有自主内存，其mm指针永远为NULL
  - 调度器判断next→mm是否为空，即可判断该进程是否为核心线程
    - 若是，则继续使用prev的active_mm，并将cpu_tlbstate[cpu].state设置为TLBSTATE_LAZY，表示内存管理部件不要刷新TLB
    - 否则，调用switch_mm()函数进行内存的切换
  - 设置好next内存环境后，可调用mmdrop()释放prev的内存结构
    - 所有不在运行中的进程，其active_mm属性都应该为空

3.4 Linux进程调度实现

```
asmlinkage void schedule(void)
{
    struct schedule_data * sched_data;
    struct task_struct *prev, *next, *p;
    struct list_head *tmp;
    int this_cpu, c;

    spin_lock_prefetch(&runqueue_lock);

    BUG_ON(!current->active_mm);
need_resched_back:
    prev = current;
    this_cpu = prev->processor;

    if (unlikely(in_interrupt())) {
        printk("Scheduling in interrupt\n");
        BUG();
    }

    release_kernel_lock(prev, this_cpu);
```

3.4 Linux进程调度实现

```
/*
 * 'sched_data' is protected by the fact that we can run
 * only one process per CPU.
 */
sched_data = & aligned_data[this_cpu].schedule_data;

spin_lock_irq(&runqueue_lock);

/* move an exhausted RR process to be last.. */
if (unlikely(prev->policy == SCHED_RR))
    if (!prev->counter) {
        prev->counter = NICE_TO_TICKS(prev->nice);
        move_last_runqueue(prev);
    }

switch (prev->state) {
    case TASK_INTERRUPTIBLE:
        if (signal_pending(prev)) {
            prev->state = TASK_RUNNING;
            break;
        }
    default:
        del_from_runqueue(prev);
    case TASK_RUNNING:;
}
prev->need_resched = 0;
```

3.4 Linux进程调度实现

switch_to()主要工作

- 将esi，edi，ebp压入堆栈
- 堆栈指针esp保存到prev→thread.esp
- 将esp恢复为next → thread.esp
- 将标号1:的地址保存到prev → thread.eip
- 将next → eip压入堆栈
- 无条件跳转到__switch_to()函数
  - 切换LDT和FS，GS等寄存器
  - 函数返回时已经切换到next上下文
- switch_to()中jmp指令以后代码（标号1:的代码）
  - 工作过程与第一步相反，即从堆栈弹出ebp，edi和esi
  - 该代码是下次该进程运行时最先执行的代码

3.4 Linux进程调度实现

Linux 2.4进程调度机制小结

- 进程就绪队列管理
  - 全局共享队列：runqueue
- 进程类型
  - 实时进程：SCHED_FIFO/SCHED_RR
  - 普通进程：SCHED_OTHER
  - 其他：SCHED_YIELD
- 进程优先级
  - 静态优先级：priority
    - 可通过nice调整，决定进程的初始时间片
  - 实时优先级：rt_priority
    - 只对实时进程有效
  - 动态优先级：通过goodness()完成，决定进程调度顺序
    - 与policy，priority，rt_priority，counter，nice等相关
    - 动态优先级在goodness()中集中式完成

3.4 Linux进程调度实现

Linux 2.4进程调度机制小结

- 进程CPU时间片：counter
  - 初始值与nice有关
  - 时间片调整策略：进程创建/运行/退出
- 进程调度/切换时机
  - 切换依据
    - 实时进程：rt_priority
    - 普通进程： goodness()计算结果
  - 切换方式
    - 主动式：直接调用schedule()
    - 被动式： need_resched设为1
      - 使用完CPU时间片
      - 高优先级进程被唤醒（SMP）：reschedule_idle()
      - 主动出让CPU： sched_yield()
      - 每次进入用户态前，检查need_resched，决定是否执行schedule ()
- 与SMP相关处理
  - reschedule_idle()
  - schedule_tail()

3.4 Linux进程调度实现

counter小结

- 在每次时钟中断时都要递减当前进程的counter值
  - update_process_times()
  - 对于调度策略为SCHED_RR的进程，一旦counter降到0，就要从runqueue中当前的位置移到队列的末尾，同时恢复其最初的时间配额
  - 对于调度策略为SCHED_OTHER的进程，当所有进程的counter值降为0时，重新计算并设置每个进程的时间配额

3.4 Linux2.4调度算法分析

- 在Linux 2.4 内核里，系统中并没有根据进程的调度策略分别管理进程，而是把将可运行进程都集中于同一队列中，三种调度算法在实现过程中合为一种，使用同一段程序。
- 在 Linux 2.4内核里，查找最佳候选就绪进程的过程是在调度器 schedule() 中进行的，每一次调度都要进行一次（在 for 循环中调用 goodness()），在所有就绪进程的时间片都耗完以后在调度器中一次性重算。这种查找过程与当前就绪进程的个数相关，因此，查找所耗费的时间是 O(n) 级的，n 是当前就绪进程个数。
- 调度动作的执行时间就和当前系统负载相关，无法给定一个上限，这与实时性的要求相违背。

3.4 Linux2.4调度算法分析

- 每次调度时，调度器都要线性遍历这个队列，另外每次重新计算 counter 所花费的时间也会随着系统中进程数的增加而线性增长
  - 系统中调度算法属于O(n)，开销线性增长
- 当大多数就绪进程的时间片都用完，且未重新分配时间片时
  - SMP系统中将可能有部分处理器处于空闲状态
  - 全局就绪进程队列，对多处理器伸缩性支持不好
  - 时间片的重算循环制约多处理器的效率

3.4 Linux2.4调度算法分析

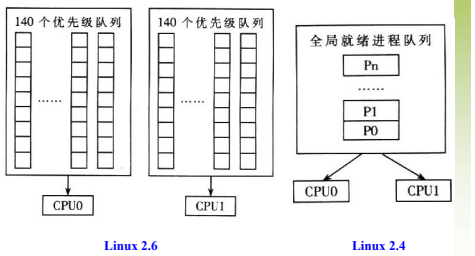
- 空闲处理器执行时间片尚未用尽、而处于等待状态的进程时，会导致进程开始在处理器之间“跳跃”
  - 处理器的亲和性不好
  - 实时进程或者占用内存大的进程在处理器之间跳跃会严重影响系统的性能
- 交互式进程支持不够,不支持内核抢占

3.5 Linux 2.6进程调度

优化目标

- 提供完全O(1)调度算法
  - 不管系统有多少进程，调度算法都必须在常数时间内完成调度
- 对SMP有良好可伸缩性
  - 每个处理器应有独立的可执行进程队列和锁机制
- 提高SMP处理器的亲和性
  - 出现负载不均衡时，应具备在处理器间迁移进程的能力

3.5 Linux 2.6进程调度



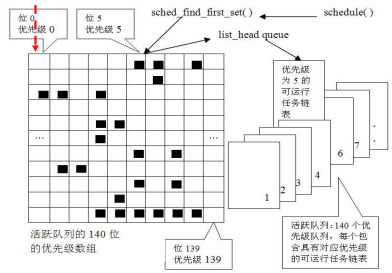
3.5 Linux 2.6进程调度

- 基于每个CPU分配时间片，取消全局同步和重算循环
- 每个处理器有两个数组：活动就绪进程队列和不活跃就绪进程队列
  - 进程消耗完其“时间片”后，进入不活跃就绪进程数组中相应队列的队尾
  - 当所有进程都“耗尽”其“时间片”后，交换活跃与不活跃就绪进程队列数组，不需要任何其他的开销
- 每个数组中有140个就绪进程队列(runqueue)，每个队列对应于140个优先级的一个
  - 通过位图标记队列状态
  - 调度时，通过find_first_bit()找到第一个非空的队列，并取队首进程
  - 不管队列中有多少就绪进程，挑选就绪程的速度恒定，因此称为O(1)算法

3.5 Linux 2.6进程调度

O(1)级调度算法结构

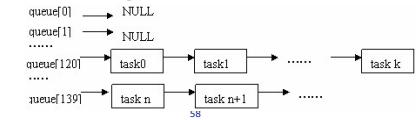
prio_array_t arrays[2];



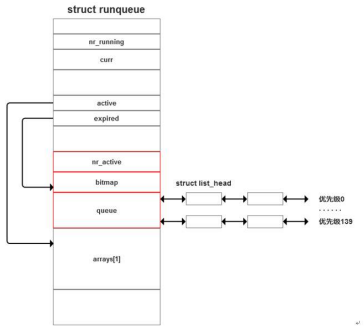
3.5 Linux 2.6进程调度

优先级数组数

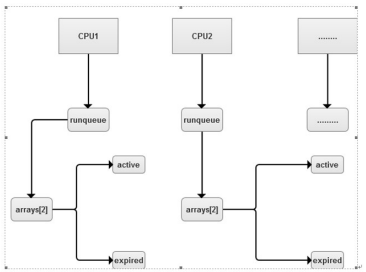
- prio_array_t *active, *expired, arrays[2];
  - 根据时间片是否被用完将就绪队列分成两类
    - active：时间片没有用完，当前可被调度的就绪进程
    - expired：时间片已用完的就绪进程
  - arrays是两类就绪队列的容器，active 和 expired 分别指向其中一个
    - active中的进程一旦用完自身时间片，就被转移到expired中，并重新设置新的初始时间片
    - active为空时，表示当前所有进程的时间片都消耗完
      - active 和 expired 对调，重新开始下一轮的时间片递减过程



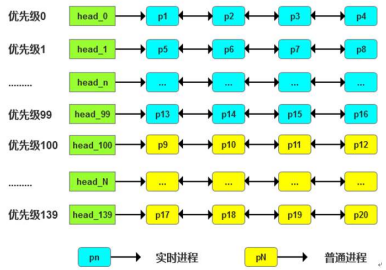
3.5 Linux 2.6进程调度



3.5 Linux 2.6进程调度

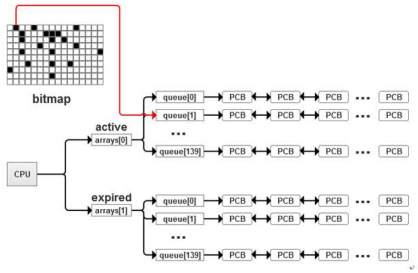


3.5 Linux 2.6进程调度



实时进程的优先级为0～99,普通进程的优先级为100～139

3.5 Linux 2.6进程调度



通过bitmap来选择优先级队列的示意图

3.5 Linux 2.6进程调度

就绪队列状态的交换

- 交换时机
  - active数组的可执行队列上的进程为空
- 时间片计算时机
  - 进程由active数组移至expired数组之前
- 交换方法（在schedule()中）

```
struct prio_array * array = rq->active;
if (!array->nr_active) {
        rq->active = rq->expired;
        rq->expired = array;
}
```


3.5 Linux 2.6进程调度

优先级数组定义

- struct prio_array

```
typedef struct runqueue runqueue_t;

struct prio_array {
    unsigned int nr_active;/*本进程组中的进程数*/
    unsigned long bitmap[BITMAP_SIZE];/*加速HASH表访问的映射*/
    struct list_head queue[MAX_PRIO];/*以优先级为索引的HASH表*/
};
```

  - queue：指定优先级进程链表的指针
    - 如queue[i]是priority为i的进程的指针
  - bitmap：优先级位图
    - 每一位代表一个优先级
  - MAX_PRIO：优先级数量
- BITMAP_SIZE的值为5
  - 通过5个四字节（long型）的整数位(160位)作为运行队列queue[MAX_PRIO]的位图掩码

3.5 Linux 2.6进程调度

```
static inline int sched_find_first_bit(const unsigned long *b)
{
    if (unlikely(b[0]))
        return __ffs(b[0]);
    if (unlikely(b[1]))
        return __ffs(b[1]) + 32;
    if (unlikely(b[2]))
        return __ffs(b[2]) + 64;
    if (b[3])
        return __ffs(b[3]) + 96;
    return __ffs(b[4]) + 128;
}
```

- _ffs()：确定long类型变量最右边的1的所在位

3.5 Linux 2.6进程调度

O(1)级调度优先级数据结构小结

- 生成160位的位数组
  - 5个long类型变量，构成一张表
  - 每一位对应一个优先级
    - 该优先级若有active状态进程，则相应位置1
    - 否则置0
  - 从右向左位扫描1，得到优先级最高的优先级号k
    - 进入queue[k]数组
  - sched_find_first_bit()函数
    - 找到位图中第1个置位的位，得到优先级最高的优先级号k

3.5 Linux 2.6进程调度

O(1) 调度算法执行过程

- 查找过程分解为 n 步，每步耗费时间均为 O(1) 级
- prio_array 包含一个就绪队列数组
  - 数组索引代表优先级（共 140 级）
  - 相同优先级进程放置在相应数组元素的链表queue中
- 调度时直接选择active就绪队列中具有最高优先级的链表中的第一项作为候选进程
- 优先级计算过程分布到各个进程的执行过程中进行
- 位映射数组用于加速寻找存在就绪进程的链表，其中每位对应一个优先级链表
  - 如果该优先级链表非空，则对应位为 1，否则为 0
  - 核心还构造了一个 sched_find_first_bit() 函数，快速定位第一个非空的就绪进程链表

3.5 Linux 2.6进程调度

```
struct task_struct *prev, *next;
struct list_head *queue;
struct prio_array *array;
int idx;

prev = current;
array = rq->active;
idx = sched_find_first_bit(array->bitmap);
queue = array->queue + idx;
next = list_entry(queue->next, struct task_struct, run_list);
```


3.5 Linux 2.6进程调度

Linux 2.6调度算法特点

- 每个处理器都有独立就绪进程队列
- 各处理器可并行运行调度程序
- 不同处理器上的进程可以完全并行地休眠、唤醒和上下文切换
- 进程只映射到一个处理器的就绪进程队列中
  - 不会被其他处理器选中
  - 也不会在不同处理器之间跳跃

3.5 Linux 2.6进程调度

对比分析

- Linux 2.4的调度算法
  - 需要遍历可运行队列，算法O(n)
  - Epoch，基本时间片，动态优先级
- Linux 2.6.17的调度算法（2.6.23之前）
  - 采用双队列（Active；expire），按照优先级组队，O(1)
- Linux 2.6.26的调度算法
  - 非实时：CFS，vruntime，红黑树
  - 实时：优先级队列
- Linux进程可以指定该进程所采用的调度策略
- 调度算法根据进程的调度策略，采用不同调度算法

3.5 Linux 2.6进程调度

- Linux 2.4的调度算法
  - 需要遍历可运行队列，算法O(n)
  - Epoch，基本时间片，动态优先级
- Linux 2.6.17的调度算法（2.6.23之前）
  - 采用双队列（Active；expire），按照优先级组队，O(1)
- Linux 2.6.26的调度算法
  - 非实时：CFS，vruntime，红黑树
  - 实时：优先级队列
- Linux进程可以指定该进程所采用的调度策略
- 调度算法根据进程的调度策略，采用不同的调度算法

3.5 Linux 2.6进程调度

- time_slice
  - 2.6 中，time_slice 变量代替了 2.4 中的 counter 变量来表示进程剩余运行时间片，但不再直接影响进程的动态优先级。
- first_time_slice
  - 0 或 1，表示是否是第一次拥有时间片（刚创建的进程）。这一变量用来判断进程结束时是否应当将自己的剩余时间片返还给父进程

3.5 Linux 2.6进程调度

- time_slice 对调度的影响
  - 在 2.4 中，进程剩余时间片是除 nice 值以外对动态优先级影响最大的因素，并且休眠次数多的进程，它的时间片会不断叠加，从而算出的优先级也更大，调度器正是用这种方式来体现对交互式进程的优先策略。
  - 但实际上休眠次数多并不表示该进程就是交互式的，只能说明它是 I/O 密集型的，因此，这种方法精度很低，有时因为误将频繁访问磁盘的数据库应用当作交互式进程，反而造成真正的用户终端响应迟缓。

3.5 Linux 2.6进程调度

- time_slice 对调度的影响
  - 2.6 的调度器以时间片是否耗尽为标准将就绪进程分成 active、expired 两大类，分别对应不同的就绪队列，前者相对于后者拥有绝对的调度优先权-----仅当active 进程时间片都耗尽，expired 进程才有机会运行。
  - 但在 active 中挑选进程时，调度器不再将进程剩余时间片作为影响调度优先级的一个因素，

本章小结

- Linux进程调度策略
  - 普通进程、实时进程
- Linux进程调度依据
  - counter值的变化，counter值重新被赋值的时机
- Linux进程调度时机
  - 主动调度、被动调度
- Linux进程调度实现
  - goodness()，weight
- Linux 2.4进程调度分析
- Linux 2.6进程调度特点

第4章 Linux系统调用

4.1 Linux系统调用功能
4.2 Linux系统调用处理过程
4.3 Linux系统调用的实例分析
4.4 Linux系统调用的参数传递
4.5 Linux中如何增加一个系统调用
4.6 Linux的内核升级与模块机制

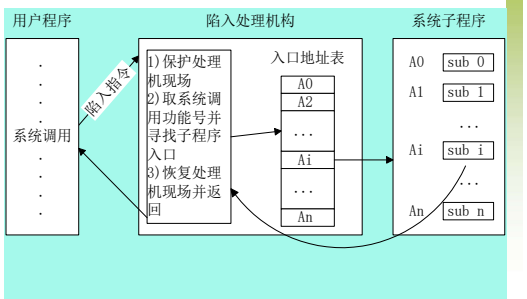
知识回顾：系统调用概述

- 系统调用(SYSTEM CALL)：OS内核中都有一组实现系统功能的过程，系统调用就是对上述过程的调用。编程人员利用系统调用，向OS提出服务请求，由OS代为完成。
- 一般情况下，进程是不能够存取系统内核的。它不能存取内核使用的内存段，也不能调用内核函数，CPU的硬件结构保证了这一点。只有系统调用是一个例外。

4.1 Linux系统调用功能

- 系统调用是用户态进入内核态的唯一入口：一夫当关，万夫莫开。常用系统调用：
  - 控制硬件:如write/read调用。
  - 设置系统状态或读取内核数据——getpid()、getpriority()、setpriority()、sethostname()
  - 进程管理:如 fork()、clone()、execve()、exit()等
- 优点
  - 编程容易，从硬件设备的低级编程中解脱出来
  - 提高了系统的安全性，可以先检查请求的正确性

4.1 Linux系统调用功能

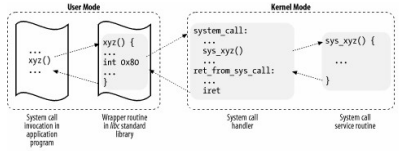


4.1 Linux系统调用功能

- Linux中实现系统调用利用了i386体系结构中的软件中断，即调用了int $0x80汇编指令。
- 这条汇编指令将产生向量为128的编程异常，CPU便被切换到内核态执行内核函数，转到了系统调用处理程序的入口：system_call()。
- int $0x80指令将用户态的执行模式转变为内核态，并将控制权交给系统调用过程的起点system_call()处理函数。

4.1 Linux系统调用功能

- system_call()检查系统调用号，该号码告诉内核进程请求哪种服务。
- 内核进程查看系统调用表(sys_call_table),找到所调用的内核函数入口地址。
- 接着调用相应的函数，在返回后做一些系统检查，最后返回到进程。



4.1 Linux系统调用功能

系统调用和普通函数调用

- API是用于某种特定目的的函数，供应用程序调用，而系统调用供应用程序直接进入系统内核。
- Linux内核提供了一些C语言函数库，这些库对系统调用进行了包装和扩展，因为这些库函数与系统调用的关系非常紧密，所以习惯上把这些函数也称为系统调用。
- 有的API函数在用户空间就可以完成工作，如一些用于数学计算的函数，因此不需要使用系统调用。
- 有的API函数可能会进行多次系统调用。
- 不同的API 函数也可能会有相同的系统调用。比如malloc()，calloc()，free()等函数都使用相同的方法分配和释放内存。

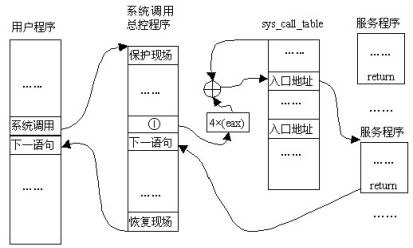
4.1 Linux系统调用功能

- **系统调用与系统命令**
  - 系统命令相对API来说，更高一层。每个系统命令都是一个执行程序，如ls命令等。这些命令的实现调用了系统调用。
- **系统调用与内核函数**
  - 系统调用是用户进入内核的接口层，它本身并非内核函数，但是它由内核函数实现。
  - 进入内核后，不同的系统调用会找到各自对应的内核函数，这些内核函数被称为系统调用的“服务例程”。如系统调用getpid()实际调用的服务例程为sys_getpid()，或者说系统调用getpid()是服务例程sys_getpid()的封装例程。

4.1 Linux系统调用功能

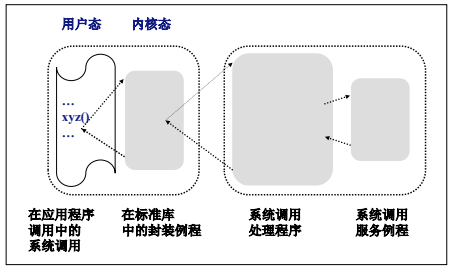
- **封装例程(wrapper routine)**
- 由于陷入指令（trap）是一条特殊指令，依赖操作系统实现的平台，如在i386体系结构中，这条指令是int $0x80(陷入指令)，不是用户在编程时应该使用的语句，因为这将使得用户程序难于移植。
- 在标准C库函数中，为每个系统调用设置了一个封装例程，当一个用户程序执行了一个系统调用时，就会调用到C函数库中的相对应的封装例程。

4.2 Linux系统调用处理过程



注①：此处语句为： call * SYMBOL_NAME(sys_call_table)(,%eax,4)；eax 中为系统调用号

4.2 Linux系统调用处理过程



4.2 Linux系统调用处理过程

system_call()片段

```
...
pushl  %eax    /*将系统调用号压栈*/
SAVE_ALL
...
cmpl $(NR_syscalls), %eax    /*检查系统调用号
JB nobadsys
movl  $(-ENOSYS), 24(%esp)   /*堆栈中的eax设置为-ENOSYS，作为返回值
jmp ret_from_sys_call
```

4.2 Linux系统调用处理过程

system_call()片段(续)

```
nobadsys:

...
call  *sys_call_table(%eax,4)
 //调用系统调用表中调用号为eax的系统调用例程
movl  %eax,EAX(%esp)
//将返回值存入堆栈中
Jmp ret_from_sys_call
```

4.2 Linux系统调用处理过程

system_call()函数执行过程

- 首先将系统调用号(eax)和可以用到的所有CPU寄存器保存到相应的堆栈中（由SAVE_ALL完成）；
- 对用户态进程传递过来的系统调用号进行有效性检查(eax是系统调用号，它应该小于 NR_syscalls)
- 若为合法系统调用，再进一步检测该系统调用是否正被跟踪；
- 根据eax中的系统调用号调用相应的服务例程。
- 服务例程结束后，从eax寄存器获得其返回值，并把这个返回值存放在堆栈中，让其位于用户态eax寄存器曾存放位置。
- 然后跳转到ret_from_sys_call()，终止系统调用程序的执行。

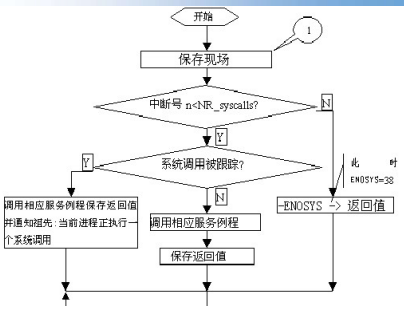
4.2 Linux系统调用处理过程

SAVE_ALL宏定义

```
#define SAVE_ALL
cld;
pushl %es;
pushl %ds;
pushl %eax;
pushl %ebp;
pushl %edi;
pushl %esi;
pushl %edx;
pushl %ecx;
pushl %ebx;
movl
   $(__KERNEL_DS),%edx;
movl %edx,%ds;
movl %edx,%es;
```

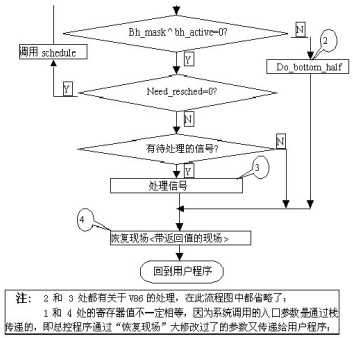
- 将寄存器中的参数压入到核心栈中(这样内核才能使用用户传入的参数。)
- 因为在不同特权级之间控制转换时，INT指令不同于CALL指令，它不会将外层堆栈的参数自动拷贝到内层堆栈中,所以在调用系统调用时，必须把参数指定到各个寄存器中

4.2 Linux系统调用处理过程



调用总控程序(system_call)执行流程图

4.2 Linux系统调用处理过程



注： 2 和 3 处都有关于 vm86 的处理，在此流程图中都省略了；
1 和 4 处的寄存器值不一定相等，因为系统调用的入口参数是通过栈传递的，即总控程序通过“恢复现场”大都改过了的参数又传递给用户程序。

4.2 Linux系统调用处理过程

- **系统调用表与调用号**
  - 系统调用处理程序一旦运行，就可以从EAX中得到系统调用号，然后再去系统调用表中寻找相应服务例程。
  - 一个应用程序调用fork()封装例程，那么在执行int $0x80之前就把EAX寄存器的值置为2(即__NR_fork)。
  - 这个寄存器的设置是libc库中的封装例程进行的，因此用户一般不关心系统调用号

4.2 Linux系统调用处理过程

- **系统调用表与调用号**
  - 核心中为每个系统调用定义了一个唯一的编号，这个编号的定义在linux/include/asm/unistd.h中（最大为NR_syscall）
  - 同时在内核中保存了一张系统调用表，该表中保存了系统调用编号和其对应的服务例程地址。第n个表项包含系统调用号为n的服务例程的地址。
  - 系统调用陷入内核前，需要把系统调用号一起传入内核。而该标号实际上是系统调用表(sys_call_table)的下标
  - 在i386上，这个传递动作是通过在执行int $0x80前把调用号装入eax寄存器实现。
  - 这样系统调用处理程序一旦运行，就可以从eax中得到系统调用号，然后再去系统调用表中寻找相应服务例程。

4.2 Linux系统调用处理过程

```
#define __NR_exit     1
#define __NR_fork     2
#define __NR_read     3
#define __NR_write    4
#define __NR_open     5
#define __NR_close    6
#define __NR_waitpid  7
#define __NR_creat    8
#define __NR_link     9
#define __NR_unlink   10
#define __NR_execve   11
#define __NR_chdir    12
#define __NR_time     13
```

系统调用号

4.2 Linux系统调用处理过程

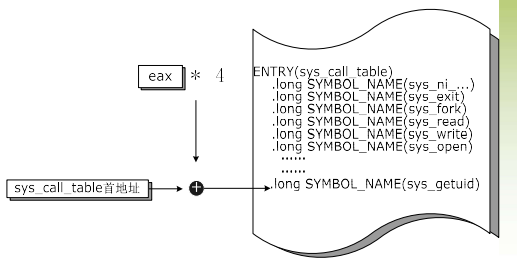
系统调用表 (arch/i386/kernel/entry.s)

```
data
ENTRY(sys_call_table)
.long SYMBOL_NAME(sys_ni_syscall)
.long SYMBOL_NAME(sys_exit)
.long SYMBOL_NAME(sys_fork)
.long SYMBOL_NAME(sys_read)
.long SYMBOL_NAME(sys_write)
.long SYMBOL_NAME(sys_open)
.long SYMBOL_NAME(sys_close)
.long SYMBOL_NAME(sys_waitpid)
.long SYMBOL_NAME(sys_creat)
.long SYMBOL_NAME(sys_link)
.long SYMBOL_NAME(sys_unlink)
.long SYMBOL_NAME(sys_execve)
.long SYMBOL_NAME(sys_chdir)
.long SYMBOL_NAME(sys_time)
.long SYMBOL_NAME(sys_mknod)
```

1. 系统调用表记录了各个系统调用的服务例程的入口地址。
2. 以系统调用号为偏移量能够在该表中找到对应处理函数地址。
3. 在linux/include/linux/sys.h中定义的NR_syscalls表示该表能容纳的最大系统调用数，一般NR_syscalls = 256。

4.2 Linux系统调用处理过程

系统调用表 (arch/i386/kernel/entry.s)



4.2 Linux系统调用处理过程

- **系统调用的返回**
  - 当服务例程结束时，system_call( ) 从eax寄存器获得系统调用的返回值，并把这个返回值存放在曾保存用户态eax寄存器栈单元的那个位置上，然后跳转到ret_from_sys_call( )，终止系统调用处理程序的执行。
  - 当进程恢复它在用户态的执行前，RESTORE_ALL宏会恢复用户进入内核前被保留到堆栈中的寄存器值。其中eax返回时会带回系统调用的返回码（负数说明调用错误，0或正数说明正常完成）

4.2 Linux系统调用处理过程

ret_from_sys_call

```
cli          /* 关中断*/
cmpl $0,need_resched(%ebx)
jne   reschedule
/*若进程描述符中的 need_resched位不为0，重新调度*/
cmpl   $0,sigpending(%ebx)
jne   signal_return   /*若有未处理完的信号，则处理*/
restore_all:
RESTORE_ALL    /*堆栈弹栈，返回用户态*/
```

4.2 Linux系统调用处理过程

- **系统调用的返回值**
  - **所有的系统调用返回一个整数值。**
    - 正数或0表示系统调用成功结束
    - 负数表示一个出错条件
  - **这里的返回值与封装例程返回值的约定不同**
    - 内核没有设置或使用errno变量
    - 封装例程在系统调用返回取得返回值之后设置这个变量
    - 当系统调用出错时，返回的那个负值将要存放在errno变量中返回给应用程序

4.3 系统调用-实例分析

假设源文件名为getpid.c，内容是：

```
#include <syscall.h>
#include <unistd.h>
#include <stdio.h>
#include <sys/types.h>
int main(void) {
  long ID;
  ID = getpid();
  printf ("getpid()=%ld\n", ID);
  return(0);
}
```

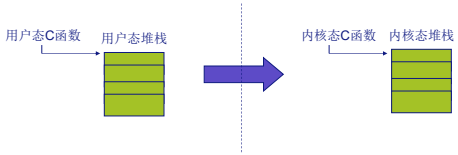
4.3 系统调用-实例分析

1. 该程序调用封装例程getpid()。该封装例程将系统调用号_NR_getpid（第20个）压入EAX寄存器
2. CPU通过int $0x80 进入内核，找到system_call()，并调用它
   (以下进入内核态)
3. 在内核中首先执行system_call()，接着执行根据系统调用号在调用表中查找到的对应的系统调用服务例程sys_getpid()。
4. 执行sys getpid()服务例程。
5. 执行完毕后，转入ret_from_sys_call()例程，系统调用返回到用户态。

4.4 系统调用的参数传递

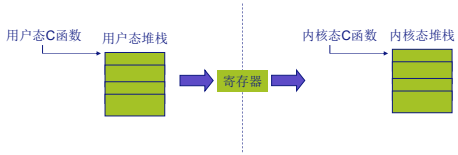
◆很多系统调用需要不止一个参数

- **普通C函数的参数传递是通过把参数值写入堆栈(用户态堆栈或内核态堆栈)来实现的。但因为系统调用是一种特殊函数，它由用户态进入了内核态，所以既不能使用用户态的堆栈也不能直接使用内核态堆栈**



4.4 系统调用的参数传递

- **在int $0x80汇编指令之前，系统调用的参数被写入CPU的寄存器。然后，在进入内核态调用系统调用服务例程之前，内核再把存放在CPU寄存器中的参数拷贝到内核态堆栈中。因为毕竟服务例程是C函数，它还是要到堆栈中去寻找参数的.**



4.4 系统调用的参数传递

系统调用**使用寄存器来传递参数**，要传递的参数有：

–系统调用号

–系统调用所需的参数

用于传递参数的寄存器有：

–eax：用于保存系统调用号和系统调用返回值

–系统调用参数保存在：ebx,ecx,edx,esi和edi中

进入内核态后，system_call() 通过使用SAVE_ALL宏把这些寄存器的值保存在内核态堆栈中。

4.4 系统调用的参数传递

- **用寄存器传递参数必须满足两个条件**
  - 每个参数的长度不能超过寄存器的长度
  - 参数的个数不能超过6个(包括eax中传递的系统调用号);否则，需要用一个单独的寄存器(ebx)指向进程地址空间中这些参数值所在的一个内存区即可
- 返回值必须写到eax寄存器中

4.4 系统调用的参数传递

- 当一个系统调用所需的参数个数大于 5 时，执行int 0x80 指令时仍需将系统调用功能号保存在寄存器 eax 中，所不同的只是全部参数应该依次放在一块连续的内存区域里，同时在寄存器 ebx 中保存指向该内存区域的指针。系统调用完成之后，返回值仍将保存在寄存器 eax 中。
- 由于只是需要一块连续的内存区域来保存系统调用的参数，因此完全可以像普通的函数调用一样使用栈(stack)来传递系统调用所需的参数。

4.4 系统调用的参数传递

- **要注意一点，**Linux 采用的是 C 语言的调用模式，意味着所有参数必须以相反的顺序进栈，即最后一个参数先入栈，而第一个参数则最后入栈。
- 如果采用栈来传递系统调用所需的参数，在执行int 0x80 指令时还应该将栈指针的当前值复制到寄存器 ebx中。

4.4 系统调用的参数传递

- 处理write系统调用的sys_write服务例程声明如下

```
asmlinkage ssize_t sys_write(unsigned int fd, const char * buf, size_t count)
```

- 该函数期望在栈顶找到fd，buf和count参数
  在封装sys_write()的封装例程中，将会在ebx、ecx和edx寄存器中分别填入这些参数的值，然后在进入system_call时，SAVE_ALL会把这些寄存器保存在堆栈中，进入sys_write服务例程后，就可以在相应的位置找到这些参数

asmlinkage使得编译器不通过寄存器(x=0)而使用堆栈传递参数

4.4 系统调用的参数传递

SAVE_ALL

```
#define SAVE_ALL \
        cld; \
        pushl %es; \
        pushl %ds; \
        pushl %eax; \
        pushl %ebp; \
        pushl %edi; \
        pushl %esi; \
        pushl %edx; \
        pushl %ecx; \
        pushl %ebx; \
        movl $(__KERNEL_DS),%edx; \
        movl %edx,%ds; \
        movl %edx,%es;
```

Sys_write需要的参数

4.4 系统调用的参数传递

系统调用的参数传递-举例

设C库中封装的系统调用号为3的函数原形如下：

```
int sys_func(int para1, int para2)
C编译器产生的汇编伪码如:
...movl  0x8(%esp),%ecx
  /*将用户态堆栈中的para2放入ecx
Movl 0x4(%esp),%ebx
  /*#将用户态堆栈中的para1放入ebx
Movl $0x3,%eax
  /*系统调用号保存在eax中int$0x80 #引发系统调用
...
Movl %eax,errno /*将结果存入全局变量errno中
Movl $-1,%eax   /*eax置为-1，表示出错注
```

4.5 练习：添加一个系统调用mysyscall

- **功能要求**
  - 首先，自定义一个系统调用mysyscall ，它的功能是使用户的uid等于0 。然后，编写一段测试程序进行该系统调用的使用。
- **执行步骤如下**
  - 添加系统调用号
  - 在系统调用表中添加相应的表项
  - 实现系统调用服务例程
  - 重新编译内核，启动新内核
  - 编写一段测试程序检验实验结果

4.5 练习：添加一个系统调用mysyscall

（1）添加系统调用号：它位于unistd.h，每个系统调用号都以“_NR_开头”，

◆系统调用的编号命名为 __NR_mysyscall

◆改写/usr/include/asm/unistd.h

```
240 #define __NR_llistxattr        233
241 #define __NR_flistxattr        234
242 #define __NR_removexattr       235
243 #define __NR_lremovexattr      236
244 #define __NR_fremovexattr      237
245 #define __NR_mysyscall         238
```

4.5 练习：添加一个系统调用mysyscall

（2）在系统调用表中添加相应的表项

- 内核中实现该系统调用的例程的名字　sys_mysyscall
- 改写arch/i386/kernel/entry.S

```
398 ENTRY(sys_call_table)
399         .long SYMBOL_NAME(sys_ni_syscall)
            ......
636         .long SYMBOL_NAME(sys_ni_syscall)
637         .long SYMBOL_NAME(sys_mysyscall)
638
639         .rept NR_syscalls-(.-sys_call_table)/4
640                 .long SYMBOL_NAME(sys_ni_syscall)
641         .endr
```

4.5 练习：添加一个系统调用mysyscall

（3）实现系统调用服务例程

把一小段程序添加在kernel/sys.c

```
asmlinkage int sys_mysyscall(void)
{
        current->uid = current->euid = current->suid =
  current->fsuid = 0;
        return 0;
}
```

（4）重新编译内核，启动新内核

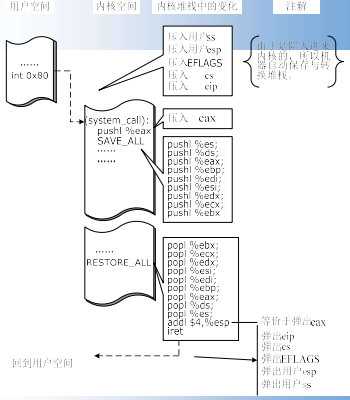
asmlinkage标志:可以告诉编译器该函数不需要从寄存器中获得任何参数，而是从堆栈中取得参数

4.5 练习：添加一个系统调用mysyscall

（5）编写一段测试程序检验实验结果

```
#include <linux/unistd.h>
_syscall0(int,mysyscall)/* 注意这里没有分号 */
int main()
{  mysyscall();
   printf("This is my uid: %d. \n", getuid());
}
```

- _syscall1(int,print_info,int,testflag)
- 如果要在用户程序中使用系统调用函数，那么在主函数main前必须申明调用_syscall，其中1 表示该系统调用只有一个入口参数，第一个int 表示系统调用的返回值为整型，print_info为系统调用函数名，第二个int 表示入口参数的类型为整型，testflag为入口参数名。



系统调用执行-小结

1、程序调用库的封装函数
2、调用软中断 int $0x80 进入内核。
3、在内核中首先执行system_call()函数，接着根据系统调用号在系统调用表中查找到对应的系统调用服务例程
4、执行该服务例程
5、执行完毕后，转入ret_from_sys_call例程，从系统调用返回

4.6 升级内核与模块机制

Linux内核源代码公开、漏洞修补、优秀代码加入、最新软件技术的加入，会导致内核快速升级。

- 为什么要升级内核
  - 内核承担操作系统最核心的任务，是其他程序和硬件运行过程中的仲裁者。
  - 内核要管理所有进程的内存，保证进程都能平等地得到CPU的时隙。
  - 更新的内核会支持更多的硬件，具备更好的进程管理能力，运行速度更快且更稳定。且能修复旧版本中的漏洞。
  - 经常升级内核是用户需要掌握的操作内容。

4.6 升级内核与模块机制

内核的配置与编译

- Linux支持的CPU
  - X86、ARM、DEC Alpha、SUN Sparc 、M68000、MIPS、PowerPC
  - 安装过程中能自动识别这些CPU
- 将获得的内核放置在正确位置(x86型)
  - /usr/src/linux/arch/i386/boot目录
- 检查源代码
  - #cd /usr/src/linux;make mrproper
    - 该命令确保源代码目录下没有不正确的.o文件以及文件的互相依赖。

4.6 升级内核与模块机制

配置内核

- 配置命令(任选一种)
  - #make config
    - (基于文本的最传统的配置界面)
  - #make menuconfig
    - (基于文本选单的配置界面)
  - #make xconfig
    - (基于图形窗口模式的配置界面)
  - #make oldconfig
    - (只想在原来内核配置的基础上上修改一些小地方，则用这个命令会省去不少麻烦)

4.6 升级内核与模块机制

配置内核（选项）

- 大部分可使用默认值，只有小部分需要根据用户不同的需要进行选择。
- 选择相应的配置
  - Y 表示将该功能编译进内核
  - N 表示不将该功能编译进内核
  - M 将该功能编译成可以在需要时动态插入到内核中的模块
- 将与内核其他部分关系较远且不经常使用的部分功能代码编译成为可加载模块有利于减少内核的长度、减少内核消耗的内存。

4.6 升级内核与模块机制

内核源代码文件的检验与编译

- 对内核源代码文件的依赖性和完整性进行检验后进行编译
  - #make dep (确保关键文件在正确的位置)
  - #make clean(确保所有有关文件都处于最新版本状态)
  - #make zImage (编译压缩形式的内核)
  - #make bzImage(编译内核)
- 若选择了可加载模块，则编译完内核后，要对选择的模块进行编译
  - #make module (编译选择的模块)
  - #make module-install(将编译后的模块转移到系统标准位置)
  - 模块的标准位置/lib/modules/x.y.z(版本号)
  - #lsmod 列出内核的当前模块

4.6 升级内核与模块机制

利用内核模块添加系统调用

- 模块是内核的一部分，但是并没有被编译到内核里面去。它们被分别编译并连接成一组目标文件， 这些文件能被插入到正在运行的内核，或者从正在运行的内核中移走。内核模块至少必须有2个函数：
- int_module和cleanup_module。第一个函数是在把模块插入内核时调用的；第二个函数则在删除该模块时调用。由于内核模块是内核的一部分，所以能访问所有内核资源。
- 根据对linux系统调用机制的分析，如果要增加系统调用，可以编写自己的函数来实现，然后在sys_call_table表中增加一项，使该项中的指针指向自己编写的函数，就可以实现系统调用。

4.6 升级内核与模块机制

- 具体实现实例通过本课程的实验完成

4.6 升级内核与模块机制

- 编译内核模块并插入内核
- 编译内核的命令为：gcc -Wall -02 -DMODULE -D_KERNEL_-C syscall.c
- -Wall通知编译程序显示警告信息；参数-02 是关于代码优化的设置， 内核模块必须优化；
- 参数-DMODULE通知头文件向内核模块提供正确的定义； 参数-D_KERNEL_通知头文件，这个程序代码将在内核模式下运行。
- 编译成功后将生成 syscall.o文件。最后使用insmod syscall.o命令将模块插入内核后即可使用增加的系统调用。

4.6 升级内核与模块机制

- 比较以上两种方法，采用内核模块的方法较好。因为这种方法可省去编译新内核并用新内核重新启动的麻烦，这一优点对于代码的调试是非常有价值的， 可以节省大量时间。

本章小结

- Linux系统调用功能的实现机制
- Linux系统调用的处理过程
- 通过实例分析Linux系统调用的过程
- 通过实验强化如何增加一个系统调用
- 通过实验强化Linux的模块机制

作业

◆什么是系统调用？它与普通函数调用有何区别？
◆什么是系统调用号和系统调用表？
◆系统调用的参数如何传递到内核？
◆简述添加系统调用的大致流程。

第5章 Linux中断处理

- Linux中断处理
  - 快中断与慢中断
  - 下半部处理
  - 下半部处理的相关数据结构和函数
  - 下半部处理的3种实现方法
- 定时器中断
  - 定时器向量
  - 动态内核定时器

5.1 Linux的中断处理

- 在中断响应时，OS规定要关闭中断允许位，直到中断处理程序中调用了开中断指令STI为止。
- 问题分析：
- 如果一个中断的处理时间较长（比如一些硬件中断，可能会一直等待I/O），则中断将长时间关闭，这时将会失去对外部发来的其它中断的响应能力（出现“死机”状态）。
- 如果开中断的时间太长，也会造成中断不断进行：一种“不安定因素”。
- 所以采用一种折中的办法：**快中断与慢中断/上半部和下半部**

快 or 慢 分析

- 例如：外部设备和处理器之间的**数据传递**工作通常是比较紧急的，且必须在关中断状态下先完成。
- 但是，对这些**数据的分析和处理**工作一般就不会那么紧急，而且不一定非得与紧急事务连着作。
- 因此，为了防止处理器因为处理不太紧急事务而耽误其它紧急中断，在上述数据处理工作期间通常都使处理器处于开中断状态，利用中断嵌套来防止其它紧急中断的丢失.

快中断与慢中断

- Linux中，将中断事件分为**快中断**（中断事件简单、时间短）和**慢中断**（事件复杂、需时长）两类。
  - 处理慢中断前：需**保存所有寄存器**的内容,而快中断处理仅要**保存部分被修改的寄存器**。
  - 慢中断处理时：**不屏蔽其它中断信号（开中断）**,而快中断处理时会屏蔽所有其它中断**（关中断）**。
  - 一般把较频繁的中断设置成快中断,减少系统额外开销。

快中断与慢中断

- **慢中断处理完毕后**，通常不立即返回被中断的进程，而是进入调度程序重新调度，调度结果未必是被中断的原进程运行(是抢先式调度)。
- **快中断处理完毕后**，通常恢复现场返回被中断的进程继续执行(是非抢先式调度)。

上半部和

- Linux将那些执行时间可
  程序一分为二，称作to
  - top half（上半部
    备紧密相关的程序，这
    行。来处理硬件发出的请求，它必须在一个新的中断产生之前终止。一般用于检测设备状态，很少做其它工作。
  - bottom half （下半部）：为其余的一些处理程序，这些程序不怕被中断，可以稍后执行

可以将上半部看做是一个用来实现“登记中断”功能的函数，即当一个中断发生时，它就把中断例程的下半部分挂接到该设备的下半部分执行队列中，然后不再做其他事情

下半部举例

- 举例说明：
  - 假设一个数据块已经到达了网线，当中断控制器接收到这个中断请求后，Linux内核只是简单地标记数据到来了，然后CPU恢复到它以前的运行状态，其余的处理稍后再说（如将数据移入一个缓冲区，需要数据的进程就可以在缓冲区找到数据）。
  - 这些稍后处理的程序包括：唤醒进程、启动其他的I/O操作等。

执行过程

- **发生中断时**：上半部分立即得到执行，它将待处理的数据存放到一个设备指定的缓冲区，然后标记它的下半部再退出。
- 由上半部**决定**下半部是否需要执行
- 有些中断处理程序下半部没有必要在每次中断时都执行，上半部通过**设置标记**标识下半部是否需要执行。
- 有时上半部运行多次，下半部才运行一次。

如何识别快中断 or 慢中断?

- 用户在注册时,将一个中断处理程序设置为快速中断或者慢速中断。 通过向中断注册函数（用于注册中断服务程序)request_irq （）中传递参数irqflags来标识：
  - SA_Interrupt=1:告诉内核这是一个“快速”中断处理程序
  - SA_Interrupt=0:告诉内核这是一个“慢速”中断处理程序
- 由于Intel公司保留0-31号中断向量由于处理异常事件。因此，硬件中断32-255（244个）可供使用。244个如何分配呢？除了0x80作为系统调用之外，其它都用在外部硬件中断源上。其中包括8259A的15个irq。

管理用的数据结构

- 有三个主要的数据结构包含了与 IRQ 相关的所有信息：
  - irqaction：处理中断的相关信息
  - irq_desc_t/irq_desc:中断请求队列数组
  - hw_interrupt_type:描述中断控制器

管理用的数据结构

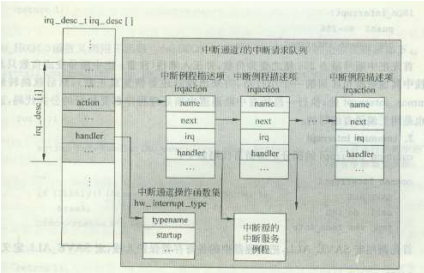


图 7-11 中断请求队列的总体结构

(1)中断请求队列结构

- 由于中断源的数目比处理器提供的中断通道数目大得多，就使得多个中断源必须共同使用同一个中断通道。于是，就必须为**每个中断通道设置一个中断请求队列**。
- 为了记录同一个通道的中断请求，Linux组织了一个链表，链表中的每个节点都是一个**irqaction**，实现了irq的共享。所有共享一个irq请求的链接在一个**action**表中，由中断描述符中的action指针指向。

(1)中断请求队列结构

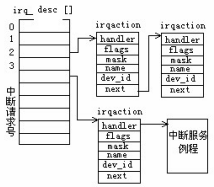
- **Irqaction[]**指向设备的中断响应函数，它在系统初始化时被设置。当中断发生时，系统自动调用该函数。
- 每个irqaction[] 中包含了处理一种中断的相关信息，包括中断处理程序的地址。
  - handler指向具体的中断服务程序的入口
  - next 指向下一个irqaction项

```
struct irqaction {
    void (*handler)(int, void *, struct pt_regs *);
    unsigned long flags;
    unsigned long mask;
    const char *name;
    void *dev_id;
    struct irqaction *next;
};
```

指向具体的中断处理程序
中断行为标志
设备名称
用户决定其用途
链表，指向下一个irqaction项

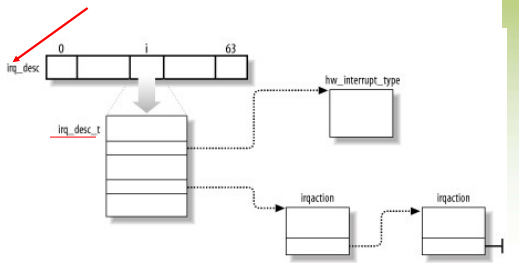
(1)中断请求队列结构

- **flags**：取值只有SA_INTERRUPT（中断处理时必须关闭中断），SA_SAMPLE_RANDOM（这个中断是源于物理随机性的，利用本设备可以作为随机数发生器），和SA_SHIRQ（这个IRQ和其它struct irqaction共享本通道）。



(2)结构体iqr_desc_t

- 在include/linux/irq.h文件中，定义了一个中断描述数组**iqr_desc[NR_IRQS]**，每一个中断向量都与它的一个元素相关联，对应一个中断源：



irq_desc_t和irq_desc:中断请求队列数组

```
typedef struct {
    unsigned int status;            /* IRQ status */
    hw_irq_controller *handler;
    struct irqaction *action;       /* IRQ action list */
    unsigned int depth;             /* nested irq disables */
    spinlock_t lock;
} ____cacheline_aligned irq_desc_t;

extern irq_desc_t irq_desc [NR_IRQS];

#define NR_IRQS 224

irq_desc_t irq_desc[NR_IRQS] __cacheline_aligned =
    { [0 ... NR_IRQS-1] = { 0, &no_irq_type, NULL, 0, SPIN_LOCK_UNLOCKED}};
```

指向hw_irq_controller中断
指向irqaction队列的头部。每一个中断号具有一个描述符，使用action链表连接共享。同一个中断号的多个设备和中断
中断描述符数组

(3) hw_interrupt_type数据结构

- hw_interrupt_type结构是一个抽象的中断控制器描述,定义在/arch/i386/kernel/irq.h中

```
/*
 * Interrupt controller descriptor. This is all we need
 * to describe about the low-level hardware.
 */
struct hw_interrupt_type {
    const char * typename;
    unsigned int (*startup)(unsigned int irq);
    void (*shutdown)(unsigned int irq);
    void (*enable)(unsigned int irq);
    void (*disable)(unsigned int irq);
    void (*ack)(unsigned int irq);
    void (*end)(unsigned int irq);
    void (*set_affinity)(unsigned int irq, unsigned long mask)
};
```

允许从给定的控制器的IRQ所产生的事件
禁止从给定的控制器的IRQ所产生的事件发生

(3) hw_interrupt_type数据结构

- hw_interrupt_type结构是一个抽象的中断控制器描述,定义在/arch/i386/kernel/irq.h中。
- 该结构中的域都是**函数指针**，它们指向了一些用来操作中断服务例程的**函数**，采用的是**面向对象的方法**。

8259A的hw_interrupt_type

- 8259A的中断控制操作

```
static struct hw_interrupt_type i8259A_irq_type = {
    "XT-PIC",
    startup_8259A_irq,
    shutdown_8259A_irq,
    enable_8259A_irq,
    disable_8259A_irq,
    mask_and_ack_8259A,
    end_8259A_irq,
    NULL
};
```

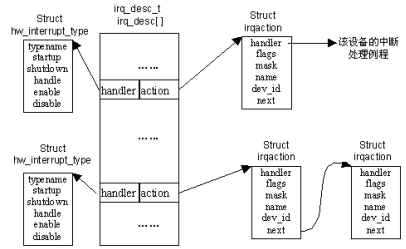
(3) hw_interrupt_type数据结构

- 例如：

```
void make_8259A_irq(unsigned int irq)
{
    disable_irq_nosync(irq);
    io_apic_irqs &= ~(1<<irq);
    irq_desc[irq].handler = &i8259A_irq_type;
    enable_irq(irq);
}
```

```
if (irq < 16)
    make_8259A_irq(irq);
```

几个数据结构之间的关系



中断服务例程的注册与安装

◆在中断源申请中断之前，该中断源的中断服务例程必须已挂接（安装）在所在通道的中断请求队列中。

(1) 中断服务例程在安装之前，需先用函数request_irq()进程注册，以向系统申请一个中断描述符结构空间和一个中断号。

(2) 调用函数setup_irq()把中断服务例程安装到中断请求队列中。该函数的主要作用是为中断服务例程分配一个irqaction结构，并把指针handler指向中断服务例程的函数。

(3) 在中断结束之后，关闭申请中断的设备时，调用free_irq()释放所申请的中断号。

注册中断处理程序request_irq()

- 一个设备在使用一个中断请求号时，需要首先通过 request_irq （）申请使用该中断请求。
- 在中断源申请中断之前，该中断源的中断服务器例程必须已经挂接在所在通道的中断请求队列中。
- 主要工作是将中断信息添加到irqaction中，从而使中断服务子程序联系起来。(arch/kernel/irq.c)
- Linux支持中断的共享，即中断信号线能够被多个中断源驱动激活。

注册中断处理程序request_irq()

```
int request_irq(
    unsigned int irq,          //要使用的中断号
    void (*handler)(int, void *, struct pt_regs *), //中断处理函数
    unsigned long irqflags,    //中断类型的标志
    const char *devname,       //设备名
    void *dev_id);  //指向共享的中断信号线的指针。
```

调用函数setup_irq()

- 调用函数setup_irq()把中断服务例程安装到中断请求队列中
- int setup_irq(unsigned int irq, //中断号
- struct irqaction *new);//中断服务程序描述
- 该函数的主要作用是为中断服务程序生成一个irqaction，设置相应的值，然后挂在irq_desc[<irq>].action队列中，如：request_irq(USB_IRQ,USB_interrupt,SA_INTERRUPT,"USB",NULL)

◆irqflags来标识中断的快 / 慢：
- SA_Interrupt=1:告诉内核这是一个“快”中断处理程序
- SA_Interrupt=0:告诉内核这是一个“慢”中断处理程序

释放中断处理程序free_irq()

- free_irq(unsigned int irq,void * dev_id)
- 该函数将已经使用完的IRQ线释放掉，以便供其它设备使用

与中断处理相关的主要函数

- Init_ISA_irqs()
  arch/i386/kernel/i8259.c中，负责初始化数组irq_desc[]
- Init_IRQ()
  arch/i386/kernel/i8259.c中，负责初始化IDT中与硬件中断相对应的那些表项。
- do_IRQ()：该函数负责处理所有的外部设备中断

中断处理过程

- 在调用do_IRQ()之前，要为中断处理程序保存寄存器
- 对所有的中断处理程序都执行相同的代码

```
common_interrupt:
        SAVE_ALL
        call do_IRQ
        jmp $ret_from_intr
```

- SAVE_ALL宏定义在entry.S中，负责保存寄存器，再将%esp寄存器移送到%eax中，调用do_IRQ()函数（arch/i386/kernel/irq.c）：

```
#define SAVE_ALL \
        cld; \
        pushl %es; \
        pushl %ds; \
        pushl %eax; \
        pushl %ebp; \
        pushl %edi; \
        pushl %esi; \
        pushl %edx; \
        pushl %ecx; \
        pushl %ebx; \
        movl $(__KERNEL_DS),%edx; \
        movl %edx,%ds; \
        movl %edx,%es;
```

do_IRQ---_do_IRQ----handle_IRQ_event

通过循环依次查询中断请求队列的所有中断，并执行。

在do_IRQ的末尾，调用iqr_exit()。它的一项重要工作中就使启动中断的下半部分，使系统可以在适当的时候执行中断服务例程的后半部分，即在向下半部分发出中断请求之后便中断返回。（invoke_softirq());

至此，计算机硬件中断系统系统负责的工作以及软中断的启动就告结束。

iqr_exit()函数

◆/usr/src/linux/kernel/softirq.c

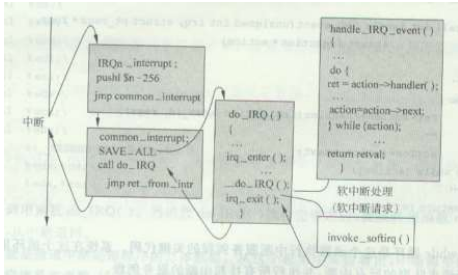
```
void irq_exit(void)
{
    account_system_vtime(current);
    trace_hardirq_exit();
    sub_preempt_count(IRQ_EXIT_OFFSET);
    if (!in_interrupt() &&
local_softirq_pending())
        invoke_softirq();//启动软中断
    preempt_enable_no_resched();
}
```

- 函数中的宏invoke_softirq(),在文件/usr/src/linux/kernel/softirq.c 中的定义为：

```
# define invoke_softirq()    do_softirq()
```

- 至此，前半部分或者说由计算机硬件中断系统负责的工作以及软中断的启动就结束了。

- 中断上半部分执行过程



下半部机制实现的几种方法

- **下半部可以通过多种机制实现**，分别由不同的接口和子系统组成。不像上半部分只能通过中断处理程序实现。

下半部机制	状态
BH(Bottom Half)	在2.6中去除
任务队列（Task queues)	在2.6中去除
软中断(Softirq)	从2.4开始引入
Tasklet	从2.4开始引入
工作队列(Work queues)	从2.4开始引入

5.2 Linux的时间系统

- **Linux的时钟**：它以读取的硬件时钟为计时器起点，根据系统启动后的时钟滴答数（节拍）来计算时间。所有的系统时钟都基于这种量度。
- **Jiffies**：是一个全局变量，用于存放自系统启动以来产生的总的节拍数。在<include/linux/jiffies.h>中定义。在内核启动时初始化为0，每产生一次中断加1。

Linux的计时体系结构

- Linux时钟每一个节拍到来时都会引发一次中断，每次中断都要完成如下工作：
  - 更新系统自启动以来所经过的时间
  - 更新时间和日期
  - 确定进程运行了多久
  - 检查每个定时器是否已经到期
- 在单处理器系统中，所有定时活动都由IRQ_0上的时钟中断触发，包括：
  - 在中断中立即执行的部分
  - 作为下半部分延迟执行的部分

- 显然，给全局变量jiffies加1这项工作是紧急的，因此它由上半部来完成；而其余各项，则因它们不那么紧急，由下半部分完成。

高精度

```
struct timespec
 { long tv_sec;
   long tv_nsec;
 }
```

普通精度

```
struct timespec
 { long tv_sec;
   long tv_usec;
 }
```

时钟中断处理例程

- 定时器中断(IRQ_0) 由top half和bottom half组成。
- 在系统初启时，由init_IRQ()函数对它初始化，设置IRQ0对应的中断门，一旦IRQ0被初始化，就可以调用上半部分。
- 定时器中断的
  - 上半部函数：timer_interrupt()(arch/i386/kernel/timer.c)
  - 下半部函数：timer_bh()(kernel/sched.c)。
- timer_interrupt()调用了do_timer()，该函数的主要任务是为jiffies+1。

时钟中断处理例程

- Linux初始化时由time_init()建立IRQ0对应的中断处理函数，它调用setup_irq()函数。

```
setup_irq(0, &irq0);
```

将irq0作为irq_desc的第一项的中断处理函数

```
static struct irqaction irq0 = { timer_interrupt, SA_INTERRUPT, 0, "timer", NULL, NULL};
```

```
struct irqaction {
    void (*handler)(int, void *, struct pt_regs *);
    unsigned long flags;
    unsigned long mask;
    const char *name;
    void *dev_id;
    struct irqaction *next;
};
```

指向具体的中断处理程序

链表，指向下一个irqaction项

定时器中断 - 上半部

timer_interrupt()读取时间的计数值。它调用do_timer_interrupt(),该函数的主要任务是调用do_timer()，它用来获取机器自启动以来时钟滴答的次数，并在do_timer()中标记需要运行的下半部

timer_interrupt( )->do_timer_interrupt()->do_timer()

timer_interrupt()

```
static void timer_interrupt(int irq, void *dev_id, struct pt_regs *regs)
{
    int count;
    write_lock(&xtime_lock);
        ……
    do_timer_interrupt(irq, NULL, regs);
    write_unlock(&xtime_lock);
```

该函数会调用do_timer进一步处理

do_timer ()

```
void do_timer(struct pt_regs *regs)
{
(*(unsigned long *)&jiffies)++;
Lost_ticks++;//记录time_bh()执行前时钟中断发生的次数
mark_bh(TIMER_BH); //标记时钟中断程序的下半部以便及时得到执行
If(!user_mode(regs))
Lost_ticks_system++;
If(tq_timer)        //如果定时器队列有任务在等待，则标记定时器队列下半部
 mark_bh(TQUEUE_BH);
}
```

最后，当tq_timer任务队列不为空时，激活下半部分TQUEUE_BH。TQUEUE_BH指向的下半部分timer_bh()，该函数在关中断的情况下调用函数update_times（）.该函数将更新xtime,并统计一些与时间相关的信息。

do_timer

```
void do_timer(struct pt_regs *regs)
{
    (*(unsigned long *)&jiffies)++;
#ifndef CONFIG_SMP
    /* SMP process accounting uses the local APIC tim
    update_process_times(user_mode(regs));
#endif
    mark_bh(TIMER_BH);
    if (TQ_ACTIVE(tq_timer))
        mark_bh(TQUEUE_BH);
}
```

全局变量，存放自系统启动以来的时钟节拍数 32位 约497天会溢出（回归为0）

检查当前进程对时间片的使用情况

激活下半部分

如果tq_timer非空，还要激活相关的下半部分处理

update_process_times

```
void update_process_times(int user_tick)
{
    struct task_struct *p = current;
    int cpu = smp_processor_id(), system = user_tick ^ 1;

    update_one_process(p, user_tick, system, cpu);
    If (p->pid) {
        if (--p->counter <= 0) {
            p->counter = 0;
            p->need_resched = 1;
        }
        if (p->nice > 0)
        ……

}

} ? end timer_interrupt ?
```

统计当前进程对CPU时间的使用情况

更新时间片 视需要进行调度

定时器中断 - 下半部TIMER_BH

- 当定时器中断处理程序的上半部timer_interrupt()执行结束后,因为在do_timer()中已经标记了需要运行的下半部，所以在do_IRQ结束之前，就可以检测到这个这个标记。
- timer_bh调用init_bh(TIMER_BH, timer-bh)将下半部timer_bh()注册为定时器下半部
- timer_bh()继续完成定时器上半部函数没有完成的统计工作，继续调用其它函数为进程和内核更新有关的时间计数值，为系统调度等各项管理提供依据。

TIMER_BH

- 当时钟中断处理例程运行结束并返回时，会立即处理下半部分

```
void timer_bh(void)
{
    update_times();
    run_timer_list();
}
```

更新系统日期和时间，计算当前的系统负载

维护定时器处理

更新时间和日期

- 用户程序从下面这个变量中获得当前时间和日期

```
struct timeval {
    time_t      tv_sec;      /* seconds */
    suseconds_t tv_usec; /* microseconds */
};

extern struct timeval xtime;
```

存放从1970年1月1日凌晨0点以来经过的所有秒数

最后一秒已经过去的微秒数 取值范围：0~999999

- 系统初始化时 time_init()初始化时间和日期

```
void __init time_init(void)
{
    extern int x86_udelay_tsc;

    xtime.tv_sec = get_cmos_time();
    xtime.tv_usec = 0;
```

获得coms时间 一旦完成，Linux不再需要RTC，依靠下半部分维护xtime

- 观察 get_cmos_time()

```
static inline void update_times(void)
{
    unsigned long ticks;

    /*
     * update_times() is run from the raw timer_bh handler so we
     * just know that the irqs are locally enabled and so we don't
     * need to save/restore the flags of the local CPU here. -arca
     */
    write_lock_irq(&xtime_lock);

    ticks = jiffies - wall_jiffies;
    if (ticks) {
        wall_jiffies += ticks;
        update_wall_time(ticks);
    }
    write_unlock_irq(&xtime_lock);
    calc_load(ticks);
}
```

上一次xtime更新后的jiffies

更新xtime

内核定时器

- 内核定时器（也称为动态定时器）是内核在以后某一个时刻运行一段程序或进程的基础。
- 使用一个定时器很简单，只需做一些初始化工作，设置一个相对于当前时刻的**超时时间**和**超时处理函数**，将其插入到内核定时器队列中即可，设置的超时处理函数会在定时器超时时自动运行。
- 内核定时器由数据结构**timer_list**表示，该结构表示了一个待处理的延迟任务。

内核定时器

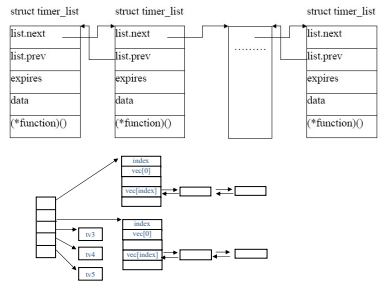
- **定时器向量：双向循环定时器链表，其中每个元素都是一个timer_list结构，队列中的所有定时器都在同一个时刻到期**<include/linux/timer.h>
- 可以被动态的创建和撤销，当前活动的定时器个数没有限制

```
struct timer_list {
    struct list_head list;
    unsigned long expires;
    unsigned long data;
    void (*function)(unsigned long);
};
```

到期时间,用于和系统核心变量jiffie

函数使用的参数

定时器到期时要执行的函数



实例：创建并激活一个内核定时器

- **如何在自己的代码中使用一个内核定时器，实现在一段时间后执行一个延迟处理任务?**
- 要求：创建一个内核定时器my_timer。该内核定时器在当前时刻以后delay个时钟中断后超时，执行超时处理函数my_function，传给超时处理函数的参数为0。
- **使用定时器的一般流程为：**

(1) 创建timer对象，编写超时定时器处理函数function；
(2) 为timer的expires、data、function赋值；
(3) 调用add_timer将timer_list加入列表；
(4) 在定时器到期时，function被执行；
(5) 在程序中涉及timer控制的地方适当地调用del_timer、mod_timer删除timer或修改timer的expires。

实现步骤

① 首先，使用下面语句声明一个内核定时器数据结构。

```
struct timer_list my_timer;
```

② 使用函数init_timer()对上一步声明的内核定时器结构进行初始化。函数init_timer()主要设置该内核定时器归属系统中哪一个处理，并初始化内核定时器链表指针的next域为NULL。

```
init_timer(&my_timer);
```

③ 使用下面的语句来设置内核定时器的超时时间expires、超时处理函数function、超时处理函数所使用的参数data。

```
my_timer.expires = jiffies + delay;
my_timer.data = 0;
my_timer.function = my_function;
```

④ 最后一步，通过函数add_timer()来激活内核定时器：

```
add_timer(&my_timer);
```

将一个定时器插入到链表中

◆函数 add_timer()用来将参数timer指针所指向的定时器插入到一个合适的定时器链表中。

◆它首先调用timer_pending()函数判断所指定的定时器是否已经位于在某个定时器向量中等待执行。

- 如果是，则不进行任何操作，只是打印一条内核告警信息就返回了；
- 如果不是，则调用 internal_add_timer()函数完成实际的插入操作。（认真分析）
- 其源码见（kernel/timer.c）

修改和修改定时器

- **修改一个定时器：**当一个定时器已经被插入到定时器链表中后，还可以修改该定时器的expires值。函数mod_timer()实现这一点（kernel/timer.c）
- **删除一个定时器：**函数del_timer()用来将一个定时器从相应的定时器队列中删除。（kernel/timer.c）
- **初始化动态定时器**

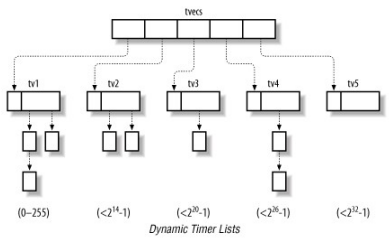
```
init_timer(struct timer_list* timer){
 (timer->list.next = NULL;
 timer->list.prev = NULL;}
```

动态定时器的提出

- 定时器的expires的值与jiffies的差值决定了一个定时器在多长时间后到期。在32位的系统中为0xffffffff。
- 因此，内核至少要维护0xffffffff个timer_list的队列。
- 数量太大！

动态定时器的处理

- Linux根据expires的值，维护这样的数据结构



动态内核定时器机制的原理

- **内核仅关心那些当前已经到期或者马上就要到期的定时器。**
- 基于上述考虑，并假定一个定时器要经过interval个时钟滴答后才到期（interval = expires - jiffies），则Linux采用了下列思想来实现其动态内核定时器机制：

动态内核定时器机制的原理

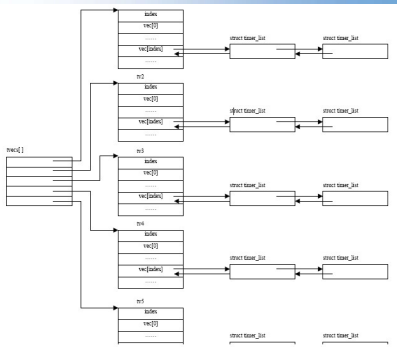
- **对于那些0≤interval≤255的定时器：**是Linux内核最关心的，因此将它们按照各自不同的expires值组织成256个定时器向量。
- 而对于那些256≤interval≤0xffffffff的定时器，由于它们离到期还有一段时间，因此内核并不关心它们，而是将它们以一种扩展的定时器向量语义（或“松散的定时器向量语义”）进行组织。
  - “松散的定时器向量语义”就是指：各定时器的expires值可以互不相同的一个定时器队列。
- 具体的组织方案可以分为两大部分：
  - tv1
  - tv2-tv5

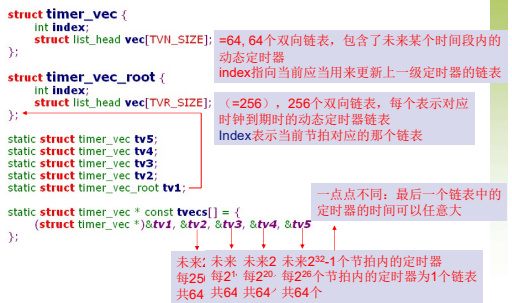
tv1

- 基于数据结构timer_vec_root，Linux定义了一个全局变量**tv1**，以表示内核所关心的**前256个定时器向量**。这样内核在处理是否有到期定时器时，它就只从定时器向量数组tv1.vec［256］中的某个定时器向量内进行扫描。
- **tv1的index字段所指定的定时器向量tv1.vec［index］中包含了当前时钟节拍内已经到期的所有动态定时器。**
  - 如：定时器向量tv1.vec［index＋k］则包含了接下来第k个时钟节拍时刻将到期的所有动态定时器。每个时钟节拍时index字段都会加1。
- 当index值又重新变为0时，就意味着内核已经扫描了tv1变量中的所有256个定时器向量。在这种情况下就必须将那些以松散定时器向量语义来组织的定时器向量**补充**到tv1中来。

tv2～tv5

- 而对于内核不关心的、interval值在［0xff，0xffffffff］之间的定时器，具有**相同值的定时器**都将被组织在同一个松散定时器向量中。
- 这些松散定时器向量也放在一起形成数组，并作为数据结构timer_vec的一部分。
- 基于数据结构timer_vec，Linux定义了全局变量tv2 ～tv5 ，来表示这些松散定时器向量。





run_timer_list

- 下半部分timer_bh()调用run_timer_list()检查到期的动态定时器，包括：
  - 执行动态定时器
  - 更新链表
- 时钟中断的每个下半部分都要执行该函数。它的作用是执行到期的定时器上注册的操作。
  - 观察 run_timer_list()

```
void timer_bh(void)
{
    update_times();
    run_timer_list();
}
```

动态定时器机制的初始化

- 函数init_timervecs()实现对动态定时器机制的初始化,主要任务就是将tv1、tv2、…、tv5这5个结构变量中的定时器向量指针数组vec［］初始化为NULL。如下所示（kernel/timer.c）：

```
void init_timervecs (void)
    {
    int i;
    for (i = 0; i < TVN_SIZE; i++) {
    INIT_LIST_HEAD(tv5.vec + i);
    INIT_LIST_HEAD(tv4.vec + i);
    INIT_LIST_HEAD(tv3.vec + i);
    INIT_LIST_HEAD(tv2.vec + i);
    }
    for (i = 0; i < TVR_SIZE; i++)
    INIT_LIST_HEAD(tv1.vec + i);
    }
```

动态内核定时器机制的实现

- 内核动态定时器机制的实现中三个重要操作：
  （1）将一个定时器插入到它应该所处的定时器向量中。Add_timer(…);
  （2）定时器的迁移，也即将一个定时器从它原来所处的定时器向量迁移到另一个定时器向量中。
  cascade_timers(…) //定时器迁移
  （3）扫描并执行当前已经到期的定时器。下半部分timer_bh()调用run_timer_list()检查到期的动态定时器，包括执行动态定时器，更新链表。

作业

- Linux为何要将中断分为快中断和慢中断两种类型？各有什么特点？
- Linux中提供了哪些机制来实现中断处理下半部分？简单描述每种的实现原理。
- Linux中管理软中断的数据结构有哪些？描述它们之间的关系。
- 如何注册、释放中断处理程序？
- 简述linux动态内核定时器机制的原理。
- 为什么动态内核定时器要进行迁移？
- 在从中断和异常返回之前内核还要做完哪些事情才能安全返回？

第6章 内存管理

6.1 Linux内存寻址
- I386内存寻址的硬件支持
- 硬件及Linux的分段机制
- 硬件及Linux的分页机制

6.2 Linux页框管理
- 内存区管理
- 非连续存储区管理

6.3 Linux进程地址空间管理
- 内存描述符
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

i386中的段寄存器

◆共6个段寄存器，每个寄存器16位
- CS：代码段寄存器，指向存放程序指令的段
  - CS寄存器包含一个两位的域，指明CPU的当前特权级CPL，值为0代表最高优先级，值为3代表最低优先级
- SS：堆栈段寄存器，指向存放当前堆栈的段
- DS：数据段寄存器，指向存放数据的段
- ES、FS及GS：附加数据段寄存器

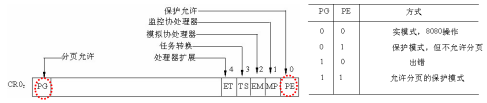
◆在保护模式下，16位的寄存器无法存放32位的段基地址，段寄存器中存放的不是某个段的基地址，而是某个段的选择子（Selector）
- 段基地址存放在段描述符表中

i386中的状态和控制寄存器

◆由标志寄存器EFLAGS、指令指针EIP和4个控制寄存器组成

CR0控制寄存器

◆机器状态字
- PE(Protedted Enable)：用于启动保护模式
  - PE=1，保护模式启动
  - PE=0，实模式下运行
- PG(Paging Enable)：分页允许位
  - 表示芯片上的分页部件是否允许工作



其他控制寄存器

◆CR1：未定义的控制寄存器

◆CR2：页故障线性地址寄存器
- 保存最后一次出现页故障的全32位线性地址

◆CR3：页目录基址寄存器
- 保存页目录表的物理地址，页目录表总是放在以4K字节为单位的存储器边界上
- 地址的低12位总为0，不起作用，即使写上内容，也不会被使用

i386中的系统地址寄存器

◆全局描述符表寄存器GDTR
- 48位寄存器，保存GDT的32位基地址和16位GDT的界限

◆中断描述符表寄存器IDTR
- 48位寄存器，保存IDT的32位基地址和16位IDT的界限

◆局部描述符表寄存器LDTR
- 16位寄存器，保存局部描述符表LDT段的选择子

◆任务状态寄存器TSR
- 16位寄存器，保存任务状态段TSS段的16位选择子

主要内容

◆I386内存寻址的硬件支持

◆硬件及Linux的分段机制

◆硬件及Linux的分页机制

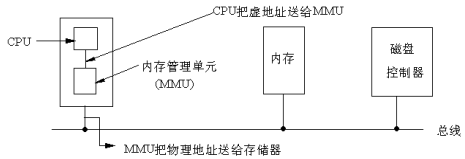
内存寻址概述

◆功能
- 保护内核不受恶意或者无意的破坏
- 隔离各个用户进程
- 方便程序的编写，使程序员可以抛开对物理内存的考虑，而且理论上可以使用任意大小的空间

◆主要内容
- X86微处理器的存储器寻址
- Linux在X86上的寻址实现

内存寻址的一般过程

◆虚实转换



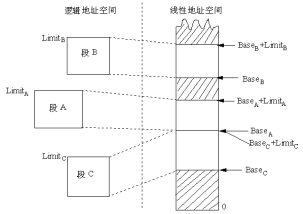
内存地址

◆Intel x86处理器的地址形式
- 逻辑地址：每个逻辑地址由一个段(segment)和偏移量(offset)组成
- 线性地址：32位无符号整数，可以表示4G的地址空间
- 物理地址：用于芯片级内存单元寻址，与从CPU的地址引脚发送到内存总线上的电信号相对应

◆地址转换过程
- 内存控制单元（MMU）通过分段单元（segmentation unit）将逻辑地址转换成线性地址
- 分页单元（paging unit）将线性地址转换成一个物理地址

段机制

◆是形成逻辑地址到线性地址转换的基础
- 基地址(Base Address)：在线性地址空间中段的起始地址
- 界限(Limit)：逻辑地址中段内的最大偏移量
- 段的属性(Attribute)： 表示段的特性



段描述符

◆每个段由一个8个字节的段描述符来表示
- 指出段的32位基地址(base字段)和20位段界限(limit字段)
- 第6个字节
  - G位：段界限粒度位，只对段界限有效
    - 当G=0时，表示段格式以字节长度
    - 当G=1时，表示段格式以4K字节为单位
  - D位：特殊位

字节	
0	7～0位段界限
1	15～8位段界限
2	7～0位段基址
3	15～8位基址
4	23～16段基址
5	存取权字节
6	G D 0 0 19～16段界限
7	31～24位段基址

段描述符—D位说明

◆描述代码段：决定指令地址及操作数的默认大小
- D=1：指令使用32位地址及32位或8位操作数
- D=0：指令使用16位地址及16位或8位操作数

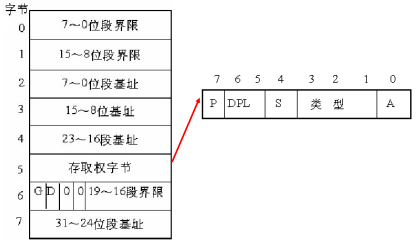
◆向下扩展数据段：决定段的上部边界
- D=1：段的上部界限为4G
- D=0：段的上部界限为64K

◆SS寄存器寻址的段（通常是堆栈段）：决定隐式的堆栈访问指令(如PUSH和POP指令)使用何种堆栈指针寄存器
- D=1：使用32位堆栈指针寄存器ESP
- D=0：使用16位堆栈指针寄存器SP

G	D	0	0	19～16段界限

存取权字节

- 定义位置
  - 第5个字节

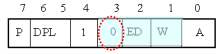


段描述符的存取权限字节说明

- 位说明
  - P字段（1位）：存在标志
    - 0表示该段当前不在内存中，1表示该段当前在内存中
  - DPL字段（ 2位）：段的特权级描述字段
    - DPL=00b：只能被内核代码段访问
    - DPL=11b：可以被任意代码段访问
  - S字段（1位）：系统段标志
    - 0：系统段
    - 1：普通段
  - Type字段（3位）：类型字段，表示代码段、数据段等
  - A字段（1位）：描述符是否被访问过
    - A=0：尚未被访问
    - A=1：已被访问
    - 说明
      - 当把描述符的相应选择子装入到段寄存器时，80386把该位置为1，表明描述符已被访问。操作系统可测试访问位，已确定描述符是否被访问过

数据段描述符说明

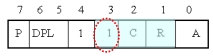
- Type字段中的E=0
  - 第2位：扩展方向位ED
    - ED=0：向地址增大的方向扩展，段内偏移必须小于等于段界限
    - ED=1：向地址减少的方向扩展，段内偏移必须大于段界限
  - 第1位：读写位W
    - W=0：数据段不可写
    - W=1：数据段可写入



  - 说明
    - 在80386中，堆栈段也被看成数据段，其本质上是特殊的数据段
    - 描述堆栈段，ED=0，W=1，即堆栈段朝地址增大的方向扩展

代码段描述符说明

- Type字段中的E=1
  - 第2位：一致位C
    - C=1：一致代码段（支持代码共享），当转移的目标是一个特权级更高的一致代码段，当前的特权级会被延续下去
    - C=0：普通代码段
  - 第1位：可读位R
    - R=0：代码段不能读
    - R=1：可读



系统段描述符

- 第5个字节的第4位为0，说明是系统段描述符
- 类型占4位，没有A位
- 第六个字节的第六位为0
  - 说明系统段的长度是字节粒度，一个系统段的最大长度为1M（2^{20}）字节

段描述符表

- 定义i386所有段的情况，所有描述符表本身都占据一个字节为8的倍数的存储器空间
- 分类
  - 全局描述符表GDT（Global Description Table）
    - 除任务门、中断门和陷阱门描述符外，包含着系统中所有任务共用段的描述符
    - 存放在RAM中，使用一个专门的寄存器GDTR指示GDT表在RAM中的位置（物理起始地址）
  - 局部描述符表LDT（Local Description Table）
    - 包含与给定任务有关的描述符
    - 存放在RAM中，使用LDTR来指示当前的LDT表
- X86的段描述符设置
  - 通常只定义一个GDT
  - 每个进程除存放在GDT中的段之外，还可以根据需要创建附加的LDT

Intel x86中的段描述符分类

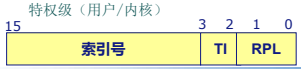
- 数据段描述符（DSD）
  - 描述各种用户数据段和堆栈段，可以放在GDT或LDT中
- 代码段描述符（CSD）
  - 描述一个用户代码段，可以放在GDT或LDT中
- 任务状态段描述符（TSSD）
  - 描述一个任务的状态段，用于保存处理器寄存器的内容，只出现在GTD中
- 局部描述符表描述符（LDTD）
  - 描述一个LDT段，只出现在GDT中
- 系统段描述符（SSD）

Intel微处理器的地址转换方式

- 实模式（20位）
  - 16位段寄存器只记录段基址的高16位，因此段基址必须4位对齐（末4位为0）
  - 不采用虚拟地址空间，直接采用物理地址空间
  - 物理地址=段寄存器值*16+段内偏移
- 保护模式（32位）
  - 16位段寄存器无法直接记录段的信息，因此需要与全局描述符表GDT配合使用
  - GDT中记录了每个段的信息（段描述符），段寄存器只需记录段在GDT中的序号
  - 线性地址=段基地址+段内偏移
    - 其中，段基地址是根据段寄存器所指明的GDT中的段描述符中的信息得到
  - 物理地址：根据页表对线性地址进行转换而得到

硬件中的分段

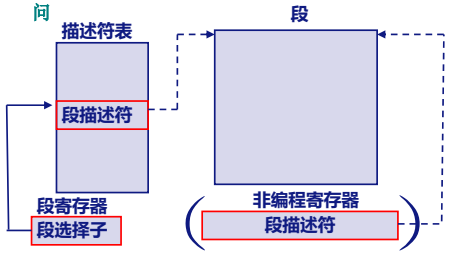
- I386体系结构采用基于段选择子的分段机制
  - 逻辑地址=段：段内偏移
    - 段标识符：16位长，称为段选择子
    - 段内偏移：32位长
  - 使用16位段寄存器来指明当前所使用的段
    - 索引号
      - 13位，指定GDT表中的相应的段描述符
    - TI(Table Indicator)
      - 1位，指明段描述符在GDT（TI=0）或LDT（TI=1）中
    - RPL(Request Privilege Level)
      - 2位，当相应段选择符装入到CS寄存器中时，指明CPU的当前特权级（用户/内核）



段选择子格式

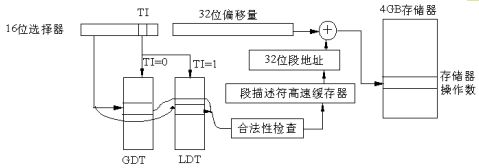
段描述符的快速访问

- 16位段寄存器与GDT/LDT配合，对相应段寻址
  - 在段选择子被装入段寄存器时，将相应的段描述符装入到对应的非编程寄存器，避免对GDT/LDT的访问



逻辑地址到线性地址的转换

- 检查TI确定段描述符位置
- 从段选择子的index字段计算段描述符的地址
- 将逻辑地址的偏移量与段描述符base字段的值相加，得到线性地址



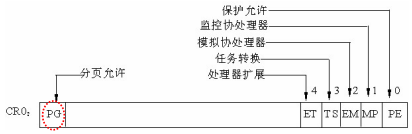
主要内容

- I386内存寻址的硬件支持
- 硬件及Linux的分段机制
- 硬件及Linux的分页机制

硬件中的分页

- 分页单元（paging unit）
  - 功能：线性地址→物理地址
  - 关键任务
    - 把所请求的访问类型与线性地址的访问权限比较，若此次内存访问无效，则产生一个缺页异常
  - 页(page)
    - 将线性地址分成以固定长度为单位的组
  - 页表(page table)
    - 将线性地址映射到物理地址的数据结构
    - 存放在内存中，并在启用分页单元以前由内核对之进行初始化
  - 页框(page frame)
    - 把所有RAM划分成以固定长度为单位的组
    - 每个页框可以包含一页，即页框长度等于页的长度
  - Intel处理器中，通过设置CR0寄存器的PG标志位来启用分页单元
    - 当PG=0时，线性地址被解释成物理地址

CR0控制寄存器

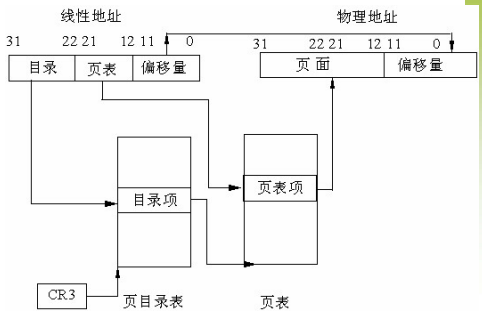


PG	PE	方式
0	0	实模式，8080操作
0	1	保护模式，但不允许分页
1	0	出错
1	1	允许分页的保护模式

常规分页

- 从i386起，Intel处理器的分页单元处理4KB的页
- 32位的线性地址被分成3个域
  - 目录(directory)：最高的10位，决定页目录项（指向适当的页表）
  - 页表(table)：中间的10位，决定页表项（指向所在页框的物理地址）
  - 偏移量(offset)：最低的12位，决定页框内的相对位置
- 线性地址的转换（二级模式）
  - 分两步完成，每一步都基于一种转换表
    - 第一种称为页目录表(page directory)
    - 第二种称为页表(page table)
  - 正在使用的页目录表的物理地址存放在CPU的CR3寄存器中

Intel 80x86处理器的分页



页目录表项结构

- 每个页目录项4字节，最多1024项
  - 为什么在线性地址中,页表及页目录只要用10位表示？
- 页表地址的低12位总为0，所以用高20位指出32位页表地址即可

	7	6	5	4	3	2	1	0
7~0位	PSE	0	A	PCD	PWT	U/S	R/W	P
15~8位	3~0位页表地址				OS专用			0
23~16位	11~4位页表地址							
31~24位	19~12位页表地址							

页目录表项属性说明

- 存在位P（第0位）
  - 如果P=1，表示页表地址指向的该页在内存中
  - 如果P=0，表示不在内存中，分页单元就把这个线性地址存放在处理器的CR2寄存器中，并产生一个14号异常（缺页异常）
- 读/写位R/W（第1位）及用户/管理员U/S位（第2位）
  - 这两位为页目录项提供硬件保护
    - 特权级为3的进程要访问页面时需通过页保护检查
    - 特权级为0的进程可绕过页保护
- PWT（第3位）位：是否采用写透方式
  - 为1表示采用写透方式，该方式既写内存（RAM）也写高速缓存

页目录表项属性说明

- PCD（第4位）位：是否启用高速缓存
  - 该位为1表示启用高速缓存
- 访问位（第5位）
  - 当对页目录项进行访问时，A位为1
- Page Size标志（第7位）：只适用于页目录项
  - 如果置为1，页目录项指的是4MB的页面（扩展分页）
- 操作系统专用（第9~11位）：Linux未使用

页表项结构

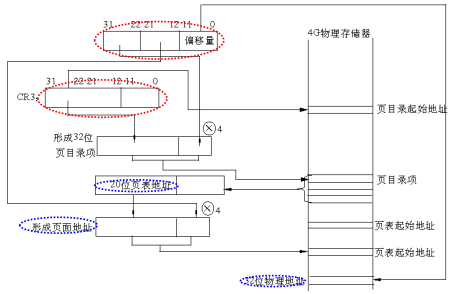
- 结构与页目录表项相同
  - 包含页面的起始地址和有关该页面的信息
    - 页面的起始地址也是4K的整数倍
    - 页面的低12位也留作它用
  - 除第6位外，第0～5位及9～11位的用途和页目录项一样
    - 第6位是页面项独有的，当对涉及的页面进行写操作时，D位被置1

	7	6	5	4	3	2	1	0
7~0位	0	D	A	PCD	PWT	U/S	R/W	P
15~8位	3~0位页面地址				OS专用			0
23~16位	11~4位页面地址							
31~24位	19~12位页面地址							

二级模式线性地址转换的特点

- 减少每个进程页表所需RAM的数量
  - 一级页表缺陷
    - 使用一级页表需要2^{20}个表项（每项4字节，需要4MB）
  - 二级模式特点
    - 每个活动进程必须有一个页目录，但不必立即为进程的所有页表分配RAM
    - 只为进程实际使用的那些虚拟内存区请求页表来减少内存需求
    - 页目录和页表都可以多达1024项（2^{10}）
    - 一个页目录的寻址能力1024*1024*4096=2^{32}

线性地址到物理地址的转换



线性地址到物理地址的转换步骤

- 第一步：形成页表地址
  - CR3包含着页目录的起始地址，用32位线性地址的最高10位A31～A22作为页目录的页目录项的索引，将它乘以4(每项4个字节)，与CR3中的页目录的起始地址相加，形成相应页表地址。
- 第二步：形成页面地址
  - 从指定的地址中取出32位页目录项，它的低12位为0，这32位是页表的起始地址。用32位线性地址中的A21～A12位作为页表中的页面的索引，将它乘以4，与页表的起始地址相加，形成32位页面地址。
- 第三步：形成32位物理地址
  - 将A11～A0作为相对于页面地址的偏移量，与32位页面地址相加，得到物理地址。

常规分页举例

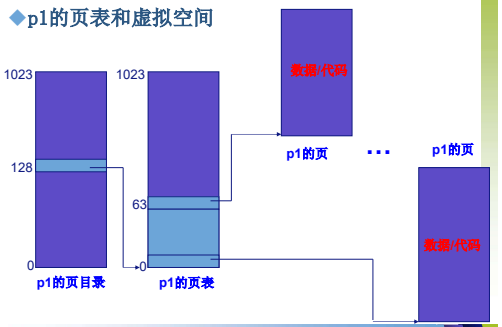
- 假设内核给一个正在运行的进程p1分配的线性地址空间是0x20000000到0x2003ffff（256KB）
  - 该段空间大小为0x40000，即0x40个页（64页）
  - 有效的线性地址范围为

0x20000000：0010 0000 0000 0000 0000 0000 0000 0000b
（页目录索引 0x80=128）（页表索引 0x0=0）

0x20003ffff：0010 0000 0000 0011 1111 1111 1111 1111b
（页目录索引 0x80=128）（页表索引 0x3f=63）

常规分页举例

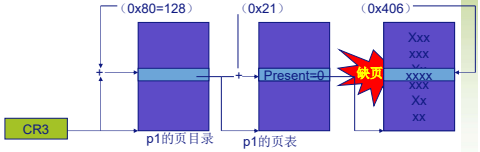
- p1的页表和虚拟空间



分页举例

- 假设进程需要读取0x20021406中的字节
- 分页单元将该地址划分为3个部分

0x20021406=0010 0000 0000 0010 0001 0100 0000 0110b
（页目录索引 0x80=128）（页表索引 0x21）（页内偏移 0x406）



- 当进程无论何时试图访问0x20000000到0x2003ffff范围之外的线性地址时，都将产生一个保护错误

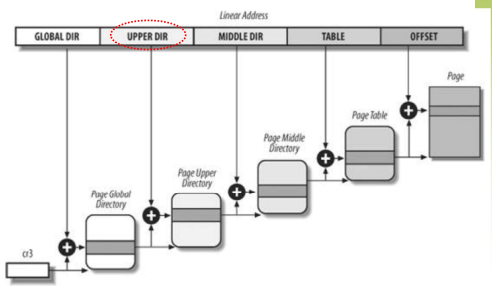
Linux进程的分页机制

- Linux的分段机制
  - 所有进程使用相同的段寄存器值及线性地址空间（0～4G）
- Linux对进程的处理很大程度上依赖于分页
  - 实际上，由硬件提供的MMU将线性地址自动转换为物理地址使得一下设计目标变得可行
    - 给每个进程分配不同的物理地址空间，该机制可确保对寻址错误提供有效保护
    - 区别页(即一组数据)和页框(实际的物理空间)之间的不同。这是虚拟存储器机制的基本因素
  - 每个进程都有它自己的页全局目录和自己的页表集合
    - 当进程切换发生时，Linux把CR3寄存器的值保存在跟进程相关的一个数据结构中，然后用另外一个进程相应的值填充CR3寄存器
    - 当新进程恢复在CPU上执行时，分页单元将使用一组与新进程对应的页表

Linux的分页模型

- 采用同时适用32位和64位系统的普通分页模型
  - Linux 2.6.10以前版本使用三级分页模型
  - Linux 2.6.10以后版本使用四级分页模型
    - 页全局目录PGD(Page Global Directory)
    - 页上级目录PUD(Page Upper Directory)
    - 页中间目录PMD(Page Middle Directory)
    - 页表PT(Page Table)
  - 没有启用物理地址扩展的32位系统，使用两级页表
    - 将“页上级目录”位和“页中间目录”位全设置为0
  - 启用物理地址扩展的32位系统使用三级页表
    - 全局目录对应X86的PDPT，取消“页上级目录”
    - 页中间目录对应X86的页目录
    - 页表对应X86的页表
  - 64位系统使用三级或四级页表，取决于硬件对线性地址的划分

Linux 2.6的四级分页模式



Linux对页表的处理函数

- Linux中实现很多对页表进行设置，操作和处理的函数
  - include/asm_i386/page.h
  - include/asm_i386/pgtable.h
  - include/asm_i386/pgtable_2level.h

页表项结构

- 表项类型： include/asm_1386/page.h
  - pte_t：描述页表项的格式
  - pmd_t ：描述中间目录项的格式
  - pud_t ：描述上级目录项的格式
  - pgd_t ：描述全局目录项的格式
- 页的保护： include/asm_1386/page.h
  - pgprot_t：PAE激活时为64位，否则32位
- 无符号整数与上述各类型的转换
  - __pte(), __pmd(), __pgd(), __pgprot()
- 上述各类型转换成无符号整数
  - pte_val(), pmd_val(), pgd_val(), pgprot_val()

页表处理

- 判断/读/写/修改各表项的操作
  - pte_none(), pmd_none(), pgd_none()
    - 判断对应表项值是否为0
  - pte_present(), pmd_present(), pgd_present()
    - 判断对应表项的present标志是否1
  - pte_clear(), pmd_clear(), pgd_clear()
    - 清除相应页表的一个表项
  - pmd_bad(), pgd_bad()
    - 检查相应目录项是否不能使用
  - pte_read(), pte_write(), pte_exec(), …

进程页表

- 一个进程的线性地址空间被分成两部分
  - 0-3G（0x00000000~0xbfffffff）
    - 用户态和内核态都可以访问
    - 进程在用户态执行时，产生的线性地址小于0xc0000000
  - 3G-4G（0xc0000000~0xffffffff）
    - 只有内核态可以访问
    - 进程在内核态执行时，它执行的内核代码所产生的地址大于等于0xc0000000
- 进程的页全局目录
  - 前768项（PAE未启用，启用时为前3项）
    - 用来映射低于0xc0000000（PAGE_OFFSET）的线性地址，具体内容与进程相关
  - 剩余表项（256项）
    - 用来映射内核空间，对所有进程都一样

第二部分 主要内容

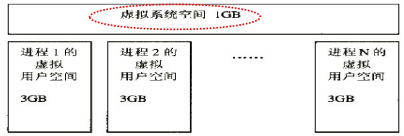
- 页框管理
- 内存区管理
- 非连续存储区管理

内存寻址的主要工作

- Linux如何有效地利用x86的分段和分页机制把逻辑地址转换为物理地址
- RAM的某些部分永久地分配给内核，用以存放内核代码以及静态数据
- RAM其余部分的管理？
  - 线性地址空间的管理
  - 进程地址空间的管理

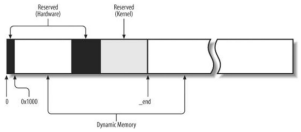
Linux的虚拟空间

- 每个进程拥有3G用户空间
- 内核占用最高1G作为系统空间
  - 系统空间由所有进程共享
- 地址转换
  - 通过页表把虚存空间的一个地址转换为物理空间中的实际地址

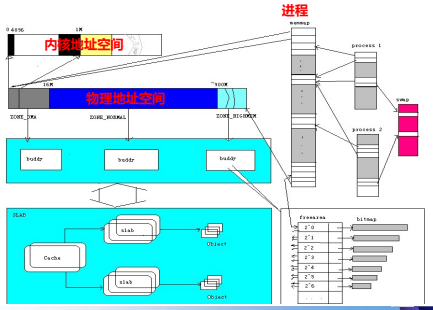


Linux的虚拟空间管理

- 内存管理
  - 动态管理内核线性虚拟空间
    - 物理地址空间到虚拟地址空间的映射
    - 属于稀缺资源，页框及内存区的分配与优化
      - 按需分配，不需要时释放
- 进程地址空间
  - 管理进程虚拟地址空间
    - 编程地址空间与进程虚拟地址空间的映射
    - 进程虚拟地址空间与物理地址空间的映射

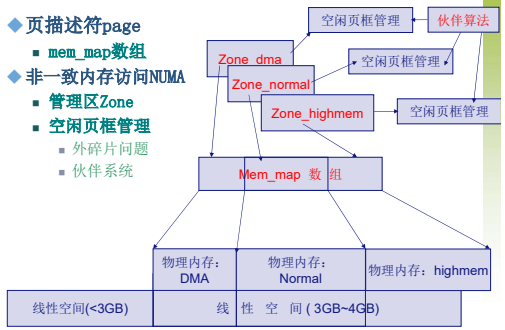


内存管理的核心工作



页框管理主要内容

- 页描述符page
  - mem_map数组
- 非一致内存访问NUMA
  - 管理区Zone
  - 空闲页框管理
    - 外碎片问题
    - 伙伴系统



页框设计

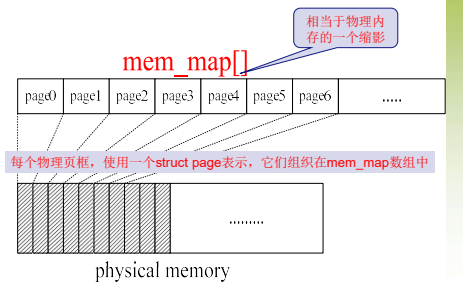
- 采用页作为内存管理的基本单位
- 标准页框大小为4KB
  - 4KB是大多数磁盘块大小的倍数
    - 传输效率高
    - 管理方便

```
/* PAGE_SHIFT determines the page size */
#define PAGE_SHIFT  12
#define PAGE_SIZE   (1UL << PAGE_SHIFT)
#define PAGE_MASK   (~(PAGE_SIZE-1))
```

页描述符

- 内核使用页描述符跟踪管理物理内存
  - 每个物理页框都用一个页描述符表示
  - 结构类型：struct page [include/linux/Mm_types.h]
    - 每个描述符长度为32字节
  - 所有物理页框的描述符，组织在mem_map的数组中
    - 所需空间略小于整个RAM的1%
    - virt_to_page(addr)宏产生线性地址addr对应的页描述符地址
    - pft_to_page(pfn)宏产生与页框号pfn对应的页描述符地址

mem_map数组



页描述符结构定义

```
struct page {
    unsigned long flags;
    atomic_t _count;        /* Usage count, see below. */
    union {
        atomic_t _mapcount;
        struct {        /* SLUB */
            u16 inuse;
            u16 objects;
        };
    };
    union {
        struct {
        unsigned long private;      /* Mapping-private opaque data: */
        struct address_space *mapping;  /* If low bit clear, points to */
        };
#if NR_CPUS >= CONFIG_SPLIT_PTLOCK_CPUS
        spinlock_t ptl;
#endif
        struct kmem_cache *slab;    /* SLUB: Pointer to slab */
        struct page *first_page;    /* Compound tail pages */
    };
    union {
        pgoff_t index;      /* Our offset within mapping. */
        void *freelist;     /* SLUB: freelist req. slab lock */
    };
    struct list_head lru;       /* Pageout list, eg. active_list
                                 * protected by zone->lru_lock ! */
#if defined(WANT_PAGE_VIRTUAL)
    void *virtual;              /* Kernel virtual address (NULL if
                                   not kmapped, ie. highmem) */
#endif /* WANT_PAGE_VIRTUAL */
#ifdef CONFIG_CGROUP_MEM_RES_CTLR
    unsigned long page_cgroup;
#endif
} ? end page ? ;
```

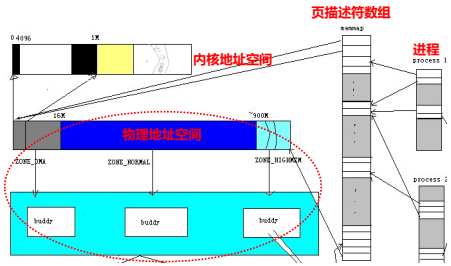
页描述符说明

- _count
  - 页的使用引用计数器
    - 0：空闲
    - >0：页已经分配给一个或多个进程或内核
- flags
  - 页框状态
  - 最多可以有32个，每个使用一个位表示

页框状态

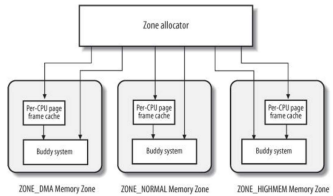
Flag name	
PG_locked	在磁盘I/O操作中涉及的页
PG_error	在传输页时发生I/O错误
PG_referenced	为一次I/O操作刚刚访问过的页
PG_uptodate	在完成读操作后置位，除非发生磁盘I/O错误
PG_dirty	页已经被修改
PG_lru	页在活动或非活动的链表中
PG_active	页在活动页链表中
PG_slab	包含在slab中的页框
PG_skip	
PG_highmem	页框属于ZONE_HIGHMEM管理区
PG_checked	由ext2文件系统使用的标志
PG_arch_1	在x86体系结构上没有使用
PG_reserved	页框留给内核代码使用或没有使用
PG_launder	

页框管理的基本结构



页框分配器

- 对连续页框组的内存分配请求
  - 接受动态内存分配与释放的请求
  - 页框被命名为伙伴系统（Buddy system）
  - 每个管理区分配器有不同的伙伴系统
  - 高速页框缓存用于快速满足对单个页框的分配请求



页框管理的外碎片问题

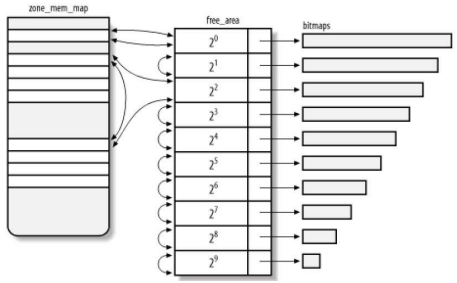
- 内核连续页框分配请求导致外碎片问题
  - 频繁的请求和释放不同大小的一组连续页框，必然导致在物理页框中分散许多小块的空闲页框
  - 这样，即使有足够的空闲页框页框满足请求，但要分配一个大块的连续页框可能就无法满足
- 可行解决途径
  - 利用MMU把一组非连续的物理空闲页框映射到连续的线性地址空间
  - 记录现存的空闲连续页框的情况，尽量避免为满足对小块的请求而把大块的空闲块进行分割
- Linux内核首选第二种方法
  - 在某些情况下，必须使用连续的页框，如DMA
  - 尽量少的修改内核页表

伙伴系统

◆基本思想

- 把所有空闲页框分组为10个块链表
  - 每个块链表分别包含大小为1，2，4，8，16，32，64，128，256和512个连续的页框
  - 每个块的第一个页框的物理地址是该块大小的整数倍
    - 例如：大小为16个页框的块，其起址是16×4KB的倍数

伙伴系统的结构示意图



管理区描述符与伙伴系统相关的数据结构

```
typedef struct zone_struct {
    /*
     * Commonly accessed fields:
     */
    spinlock_t          lock;
    unsigned long       free_pages;
    unsigned long       pages_min, pages_low, pages_high;
    int                 need_balance;

    /*
     * free areas of different sizes
     */
    free_area_t         free_area[MAX_ORDER];

    /*
     * Discontig memory support fields.
     */
    struct pglist_data  *zone_pgdat;
    struct page         *zone_mem_map;
    unsigned long       zone_start_paddr;
    unsigned long       zone_start_mapnr;

    /*
     * rarely used fields:
     */
    char                *name;
    unsigned long       size;
} ? end zone_struct ?   zone_t;
```

Zone中的空闲页数

Zone中的空闲页面管理数据结构

基于管理区的伙伴系统

◆Linux为每个zone使用独立的伙伴系统

- Zone DMA
- Zone Normal
- Zone HighMem

◆每个伙伴系统使用的主要数据结构

- 页描述符数组zone_mem_map，每个管理区都关系到mem_map的一个子集
  - 子集的第一个元素和元素个数分别由管理区描述符的zone_mem_map和size字段指定
- 结构为free_area_t的空闲内存管理数组free_area
- 与空闲内存管理数组相关的位图数组map

管理区描述符

```
typedef struct zone_struct {
    /*
     * Commonly accessed fields:
     */
    spinlock_t          lock;
    unsigned long       free_pages;
    unsigned long       pages_min, pages_low, pages_high;
    int                 need_balance;

    /*
     * free areas of different sizes
     */
    free_area_t         free_area[MAX_ORDER];

    /*
     * Discontig memory support fields.
     */
    struct pglist_data  *zone_pgdat;
    struct page         *zone_mem_map;
    unsigned long       zone_start_paddr;
    unsigned long       zone_start_mapnr;

    /*
     * rarely used fields:
     */
    char                *name;
    unsigned long       size;
} ? end zone_struct ?   zone_t;
```

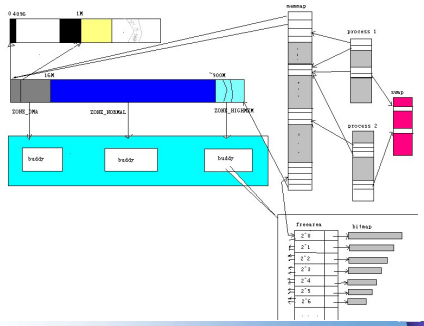
free_area_struct结构定义

```
typedef struct free_area_struct {
    struct list_head    free_list;
    unsigned long       *map;
} free_area_t;
```

所有大小为2^k个页框的块组成的双向循环链表：链表中的每个元素是对应块的第一个页框描述符

map字段指向一个位图。位图的每一位描述大小为2^k个页框的两个伙伴块的状态。

基于管理区的伙伴系统全局视图



伙伴的定义

◆伙伴的必要条件

- 大小相同
- 物理地址连续
- 假定伙伴的大小为b，第一个伙伴的第一个页框的物理地址必须是2×b×4KB的倍数

◆事实上伙伴是通过对大块的物理内存划分获得的

- 假如从第0个页面开始到第3个页面结束的内存

- 每次都对半划分，那么第一次划分获得大小为2页的伙伴
- 进一步划分，可以获得大小为1页的伙伴，例如0和1，2和3

伙伴的合并及位图含义

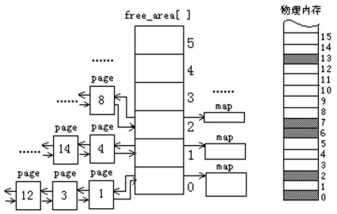
◆当两个伙伴都空闲时，则合并成一个更大的块

◆该过程一直进行，直到找不到可以合并的伙伴为止

◆位图用来描述伙伴的状态

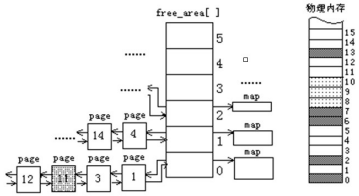
- 一对伙伴只使用一个位表示
  - 0：伙伴的状态一致，此时要么全空闲，要么全不（或部分）空闲
    - 如果全空闲，必然被合并了
    - 两种情况下，对应的块数据结构都不在此free_area_t结构中
  - 1：伙伴的状态不一致，此时必然有一个空闲、一个不空闲
    - 表示对应的块数据结构在此free_area_t结构的链表中

伙伴的位图管理示例



内存分配的Buddy算法

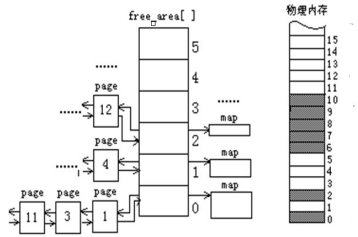
伙伴的位图管理示例



8、9、10页面分配后的示意

内存分配的Buddy算法

伙伴的位图管理示例



13号页面释放后

伙伴系统中块的分配与释放函数

◆块分配函数：__rmqueue(zone, order)

- 根据输入参数order，在相应链表(管理区描述符地址zone)中查找连续的空闲页框
- 返回所分配的第一个页框对应的页描述符的地址

◆块释放函数：__free_pages_bulk(page, zone, order)

- 把释放的页框插入到伙伴系统的数据结构中，以便以后可以在分配中使用
- 在这里循环检查bitmap确认伙伴块是否空闲，空闲则合并成一个单独的块，并把此块加入到合适的链表中

伙伴系统举例

◆伙伴系统的构建（假设有128MB的RAM）

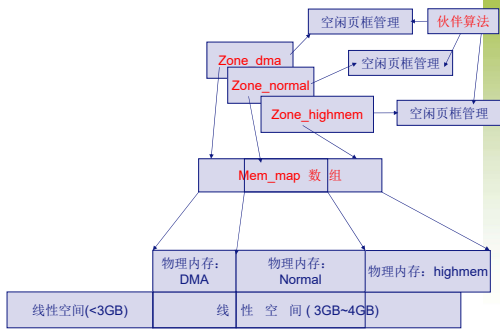
- 128MB最多可以分成2^{15}=32768个页框，2^{14}=16384个8KB（2页）的块或2^{13}=8192个16KB（4页）的块，直至64个大小为512个页的块
- free_area[0]对应的位图由16384位组成，每对伙伴（大小为1个页框）对应其中的一位
- free_area[1]对应的位图由8192位组成，每对伙伴块（大小为2页）对应其中的一位
- …
- free_area[9]对应的位图由32位组成，每对伙伴（大小为512页）对应其中的一位

伙伴系统举例

◆伙伴系统的使用（假设要请求一个大小为128个页框(0.5MB)的块

- 算法先free_area[7]中检查是否有空闲块（块大小为128个页框）
- 若没有，就到free_area[8]中找一个空闲块（块大小为256个页框）
  - 若存在这样的块，内核就把256个页框分成两等份，一半用作满足请求，另一半插入free_area[7]中
- 如果在free_area[8]中也没有空闲块，就继续找free_area[9]中是否有空闲块
  - 若有，先将512分成伙伴，一个插入free_area[8]中，另一个进一步划分成伙伴，取其一插入free_area[7]中，另一个分配出去
- 如果free_area[9]也没有空闲块，内存不够，返回一个错误信息

页框管理小结



主要内容

- 页框管理
- **内存区管理**
- 非连续存储区管理

内存区管理

- 基于页面的分配器不能满足多种要求
  - 内核中大量使用各种数据结构，大小从几个字节到几十上百k不等，都取整到2的幂次个页面那是完全不现实的
  - 早期内核的解决方法
    - 提供大小为2,4,8,16,...,131056字节的内存区域
    - 需要新的内存区域时，内核从伙伴系统申请页面，把它们划分成一个个区域，取一个来满足需求
    - 如果某个页面中的内存区域都释放了，页面就交回到伙伴系统

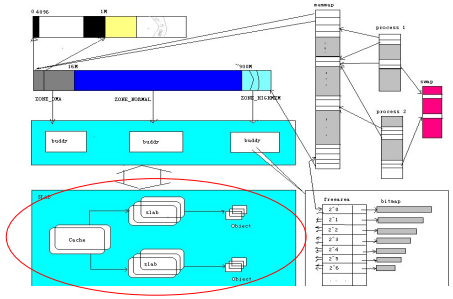
早期内存区管理的缺陷

- 不同数据类型用不同的方法分配内存可能提高效率
  - 如需要初始化的数据结构，释放后可以暂存着，再分配时就不必初始化了
- 内核的函数常常重复地使用同一类型的内存区，缓存最近释放的对象可以加速分配和释放
- 对内存的请求可以按照请求频率来分类，频繁使用的类型使用专门的缓存，很少使用的可以使用通用缓存
- 使用2的幂次大小的内存区域时硬件高速缓存冲突的概率较大，有可能通过仔细安排内存区域的起始地址来减少硬件高速缓存冲突
- 缓存一定数量的对象可以减少对buddy系统的调用，从而节省时间并减少由此引起的硬件高速缓存污染

slab分配器

- slab分配器的改进思想
  - 将内存区看成对象
  - 将对象按照类型分组成不同的高速缓存，每个高速缓存都是同种类型内存对象的一种“储备”
    - 例如当一个文件被打开时，存放相应所需的内存是从一个叫做filp的slab分配器的高速缓存中得到的，即每种对象类型对应一个高速缓存
  - slab分配器通过伙伴系统分配页框
  - 每个slab由一个或多个连续的页框组成，这些页框中包含已分配的对象，也包含空闲的对象

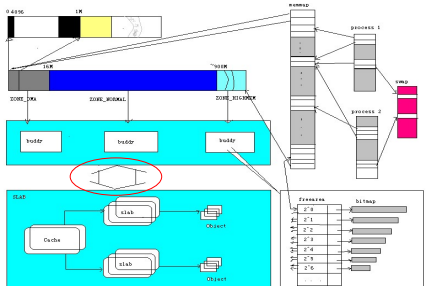
支持slab分配器的内存管理视图



slab的状态与分配策略

- 每个slab有三种状态
  - 全满
    - 全满意味着slab中的对象全部已被分配出去
  - 半满
    - 半满介于两者之间
  - 全空
    - 全空意味着slab中的对象全部是可用的
- slab对象的分配策略
  - 优先从半满的slab满足这个请求
  - 否则从全空的slab中取一个对象满足请求
  - 如果没有空的slab则向buddy系统申请页面生成一个新的slab

slab分配器与伙伴系统的接口



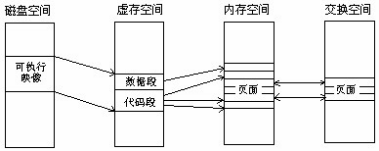
主要内容

- 页框管理
- 内存区管理

第三部分 主要内容

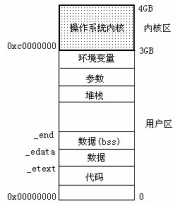
- **进程的地址空间**
- 内存描述符
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

Linux进程的地址映射



进程虚拟空间

- Linux把进程虚拟空间分成内核区和用户区两部分
  - 操作系统内核的代码和数据等被映射到内核区
  - 进程可执行映像（代码和数据）映射到虚拟内存的用户区
    - 一个进程所需的虚拟空间中的各个部分未必连续，这通常会形成若干离散的虚存“区间”(VM area)
    - 一个虚拟“区间”是进程虚拟空间的一部分，这部分的虚拟空间是连续的并且有相同的一些属性



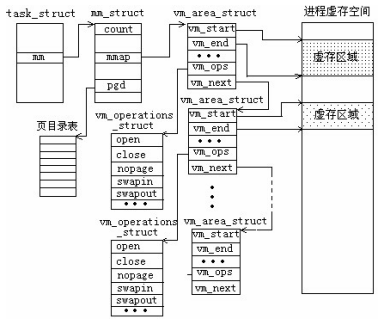
进程地址空间

- 进程在访问某个线性空间之前，必须先获得许可
  - 一个进程的地址空间是由允许该进程访问的全部线性地址组成
  - 内核使用线性区资源来表示线性地址空间
  - 每个线性区由起始线性地址、长度和一些存取权限描述

进程内存组织相关数据结构

- 每个进程有且只有一个mm_struct结构，描述进程的虚拟内存
- vm_area_struct结构描述进程的虚拟内存地址区域（线性区）
  - 每个进程可以有多个VM area
  - 对页错误处理有同一规则的进程虚拟内存空间部分，如共享库、堆栈
- page结构描述一个物理页，系统保证跟踪到每一个物理页

进程的内存组织



主要内容

- 进程的地址空间
- **内存描述符**
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

内存描述符

- struct mm_struct[include/linux/sched.h]
  - 所有内存描述符放在由mmlist字段组成的双向链表中
  - 链表第一个元素是init_mm，是进程0的内存描述符
  - 关键字段说明
    - mm_users：共享mm_struct的轻量级进程的个数
    - mm_count：内存描述符的住使用计数器
      - mm_users次使用计数器的所有用户在mm_count中只作为一个单位
      - 每当mm_users减少时，内核都要检查它是否变为0，若是，则接触这个内存描述符
      - 如果把内核描述符暂时借给一个内核线程，则增加mm_count

```
struct mm_struct {
    struct vm_area_struct * mmap;        /* list of VMAs */
    rb_root_t mm_rb;
    struct vm_area_struct * mmap_cache;
    pgd_t * pgd;
    atomic_t mm_users;
    atomic_t mm_count;
    int map_count;
    struct rw_semaphore mmap_sem;
    spinlock_t page_table_lock;          /* Protects task page tables and mm->

    struct list_head mmlist;
                 * toge
                 * by m
                 */

    unsigned long start_code, end_code, start_data, end_data;
    unsigned long start_brk, brk, start_stack;
    unsigned long arg_start, arg_end, env_start, env_end;
    unsigned long rss, total_vm, locked_vm;
    unsigned long def_flags;
    unsigned long cpu_vm_mask;
    unsigned long swap_address;

    unsigned dumpable:1;

    /* Architecture-specific MM context */
    mm_context_t context;
} ? end mm_struct ? ;
```

页全局目录

nap和mm_rb是两个不同的数据结构，但是包含了相同的东西:进程地址空间中所有的memory areas
前者使用链表存储areas
后者用红黑树存储areas

所有的mm_struct结构通过mmlist域链接在一个双向链表上。这个链表的第一个元素init_mm是idle进程的mm_struct结构

内核线程的内存描述符

- 内核线程仅运行在内核态，不会访问低于TASK_SIZE(等于0xc0000000)的地址
- 进程描述符中的两个内存描述符mm和active_mm
  - mm：指向进程拥有的内存描述符
  - active_mm：指向进程运行时所使用的内存描述符
  - 对普通进程而言，这两个字段存放相同的指针
  - 对内核线程而言
    - 由于不拥有任何内核描述符，因此，mm字段总为NULL
    - 当内核线程运行时，active_mm被初始化为前一个运行进程的active_mm的值

内存描述符的分配

- 在fork()通过调用copy_mm()从父进程拷贝内存描述符
- mm_struct数据结构本身的空间通过宏allocate_mm()从mm_cachep指向的slab缓存中分配得到的
- 如果父进程在fork()创建子进程时，通过一些标志指明要和子进程共享地址空间。那么，只需要
  - childtask->mm = parent->mm

```
if(clone_flags & CLONE_VM){
        /*
         * current 是父进程而tsk在fork()执行期间是子进程
         */
        atomic_inc(&current->mm->mm_users);
        tsk->mm= current ->mm;
}
```

内存描述符的释放

- 在进程退出时，exit_mm()函数被调用
  - 首先做一些常规清除工作，更新一些内核全局统计数据
  - 接着调用mmput()，减少内存描述符的mm_users域
    - 如果mm_users域变成0，就调用mmdrop()函数减mm_count域
      - 如果mm_count域变成0，就由free_mm()宏调用kmem_cache_free()函数把mm_struct返还给mm_cachp指向slab缓存

主要内容

- 进程的地址空间
- 内存描述符
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

线性区（memory region）

- 由vm_area_struct结构体描述
  - 描述地址空间内连续区间上的一个独立内存范围
  - 线性区的开始和结束都必须4KB对齐
  - 进程只能访问某个有效的线性区
    - 如果进程试图访问一个有效的area之外的地址或者用不正确的方式访问一个有效的area，内核将通过段异常(segmentation fault)杀死这个进程
  - 进程获得新线性区的一些典型情况
    - 刚刚创建的新进程
    - 使用exec系统调用装载一个新的程序运行
    - 将一个文件（或部分）映射到进程地址空间中
    - 当用户堆栈不够用的时候，扩展堆栈对应的线性区
    - ……

线性区描述符vm_area_struct

- 描述一段给定的内存区间
- 区间中的地址都有同样的属性，如同样的存取权限和相关的操作函数
- 用这个结构可以表示各种线性区，如映射可执行的二进制代码的线形区、用作用户态堆栈的线性区等等

```
/*
 * This struct defines a memory VMM memory area. There is one of these
 * per VM-area/task. A VM area is any part of the process virtual memory
 * space that has a special rule for the page-fault handlers (ie a shared
 * library, the executable area etc).
 */
struct vm_area_struct {
    struct mm_struct * vm_mm;    /* The address space we belong to. */
    unsigned long vm_start;      /* Our start address within vm_mm. */
    unsigned long vm_end;        /* The first byte after our end address
                                    within vm_mm. */

    /* linked list of VM areas per task, sorted by address */
    struct vm_area_struct *vm_next;

    pgprot_t vm_page_prot;       /* Access permissions of this VMA. */
    unsigned long vm_flags;      /* Flags, listed below. */

    rb_node_t vm_rb;

    /*
     * For areas with an address space and backing store,
     * one of the address_space->i_mmap{,shared} lists,
     * for shm areas, the list of attaches, otherwise unused.
     */
    struct vm_area_struct *vm_next_share;
    struct vm_area_struct **vm_pprev_share;

    /* Function pointers to deal with this struct. */
    struct vm_operations_struct * vm_ops;

    /* Information about our backing store: */
    unsigned long vm_pgoff;      /* Offset (within vm_file) in PAGE_SIZE
                                    units, *not* PAGE_CACHE_SIZE */
    struct file * vm_file;       /* File we map to (can be NULL). */
    unsigned long vm_raend;      /* XXX: put full readahead info here. */
    void * vm_private_data;      /* was vm_pte (shared mem) */
} ? end vm_area_struct ? ;
```

vm_area_struct结构体字段说明

- vm_offset
  - 该区域的内容相对于文件起始位置的偏移量，或相对于共享内存首址的偏移量。
- vm_next
  - 指向下一个vm_area_struct结构体，所有vm_area_struct结构体链接成一个单向链表
  - 链表首地址由mm_struct中成员项mmap指出
- vm_ops
  - 指向vm_operations_struct结构体的指针
  - 该结构体中包含着指向各种操作函数的指针

vm_area_struct结构体字段说明

- vm_avl_left
  - 左指针指向相邻的低地址虚存区域
- vm_avl_right
  - 右指针指向相邻的高地址虚存区域
- mmap_avl
  - 表示进程AVL树的根
- vm_avl_hight
  - 表示AVL树的高度
- vm_next_share和vm_prev_share
  - 把有关vm_area_struct结合成一个共享内存时使用的双向链表

线性区访问权限

- vm_flags
  - 描述有关这个线性区全部页的信息，如
    - 进程访问每个页的权限
    - 线性区的增长方式

标　志	对VMA及其页面的影响
VM_READ	页面可读取
VM_WRITE	页面可写
VM_EXEC	页面可执行
VM_SHARED	页面可共享
VM_MAYREAD	VM_READ标志可被设置
VM_MAYWRITE	VM_WRITE标志可被设置
VM_MAYEXEC	VM_EXEC标志可被设置
VM_MAYSHARE	VM_SHARE标志可被设置
VM_GROWSDOWN	区域可向下增长
VM_GROWSUP	区域可向上增长
VM_SHM	区域可用作共享内存
VM_DENYWRITE	区域映射一个不可写文件
VM_EXECUTABLE	区域映射一个可执行文件
VM_LOCKED	区域中的页面被锁定
VM_IO	区域映射设备I/O空间
VM_SEQ_READ	页面可能会被连续访问
VM_RAND_READ	页面可能会被随机访问
VM_DONTCOPY	区域不能在fork()时被拷贝
VM_DONTEXPAND	区域不能通过mremap()增加
VM_RESERVED	区域不能被换出
VM_ACCOUNT	该区域是一个记账VM对象
VM_HUGETLB	区域使用了hugetlb页面
VM_NONLINEAR	该区域是非线性映射的

线性区基本操作函数

- void open(struct vm_area_struct *area)
  - 将指定的内存区域被加入到一个线性地址空间
- void close(strcut vm_area_struct *area)
  - 将指定的内存区域从地址空间删除
- struct page *nopage(struct vm_area_struct *area, unsigned long address, int unsed)
  - 访问的页不在物理内存时，该函数被页错误处理程序调用
- int populate(strcut vm_area_struct *area, unsigned long addr, unsigned long len, pgprot_t, prot, unsigned long pgoff, int nonblock)
  - 被系统调用remap_pages()调用来为将要发生的缺页重大预映射一个新映射

线性区维护

- 线性区从来不重叠
- 进程中每个单独区域对应一个不同内存区，如
  - 堆栈、二进制代码、全局变量、文件映射等等
- 内核尽力将新分配的线性区与紧邻的现有线性区合并
  - 如果两个相邻区的访问权限相匹配

主要内容

- 进程的地址空间
- 内存描述符
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

线性区的处理

- 查找线性区
- 查找空闲进程地址区间
- 向内存描述符链表插入线性区
- 分配/释放线性地址区间

查找给定地址的最邻近线性区

- find_vma(struct mm_struct *mm, unsigned long addr)
  - 功能说明
    - 在指定地址空间搜索第一个vm_end大于addr的内存区域
    - 如果没有发现，则返回NULL
    - 否则返回匹配线性区的vm_area_struct结构体地址
  - 参数说明
    - mm ： 待搜索空间，进程内存描述符的地址
    - addr ： 线性地址
  - 工作流程
    - 首先查找mmap_cache
    - 如果没有，则搜索红黑树

查找一个空闲的地址区间

- get_unmapped_area(unsigned long len, unsigned long addr)
  - 功能说明
    - 搜索进程的地址空间，以找到一个可以使用的线性地址区间
    - 查找成功返回新区间的起始地址，否则返回错误码
  - 参数说明
    - len：指定区间的长度
    - addr：必须从哪个地址开始查找

向内存描述符链表插入一个线性区

- insert_vm_struct(struct mm_struct *mm, struct vm_area_struct *vmp)
  - 功能说明
    - 调用find_vma_preapre()在红黑树中查找vmp插入位置
    - 调用vma_link()依次执行以下操作
      - 在mm->mmap所指向的连表中插入线性区
      - 在红黑树mm->mm_rb插入线性区
      - 如果线性区匿名，将其插入相应的anon_vma数据结构作为头节点的链表中
      - 递增mm->map_count计数器
  - 参数说明
    - mm：指定进程描述符的地址
    - vmp：待插入的vm_area_struct对象的地址
      - 线性区的vm_start和vm_end必须已经初始化过

创建一个线性区间

- static inline unsigned long do_mmap(struct file *file, unsigned long addr, unsigned long len, unsigned long prot, unsigned long flag, unsigned long offset)
  - 参数说明
    - 根据file参数映射指定的文件中偏移量为offset，长度为len的一段内容
      - 如果file为NULL，offset为0，则为匿名映射
      - 如果指定了文件名和偏移量，则为文件映射
    - addr参数指明从何处开始查找一段可用的空闲线性地址区间
    - Prot参数指定这个区间所包含的页的存取权限
    - flags参数指定这个创建的线性区本身的一些标志
  - 说明
    - 如果创建的地址区间和已存在的地址区间相邻，且有相同的访问权限，则将区间合并

释放一个线性区间

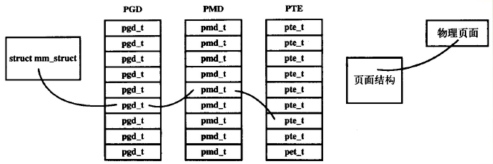
- int do_munmap(struct mm_struct *mm, unsigned long addr, size_t len)
  - 参数说明
  - mm：指向当前进程的内存描述符
  - addr：为线性区的起始地址
  - len：指明要删除的区间大小

主要内容

- 进程的地址空间
- 内存描述符
- 线性区描述符
- 线性区的处理
- 虚存与实存的映射

虚存与实存的映射

- 通过查询页表完成
  - 将虚拟地址分段，每段作为一个索引指向页表
  - 页表则指向下一级别的页表或指向最终物理页面
- Linux使用页表完成地址转换



请页机制

- 实现虚存管理的重要手段
  - 进程运行时，CPU访问的是用户空间的虚地址
  - Linux仅把当前要使用的少量页面装入内存，需要时再通过请页机制将特定的页面调入内存
  - 当要访问的虚页不在内存时，产生一个页故障并报告故障原因

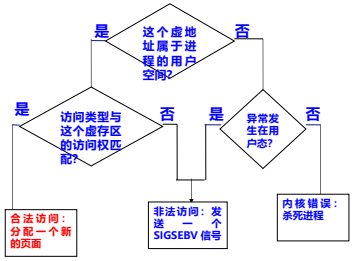
页故障原因

- 程序出现错误
  - 如要访问的虚地址在PAGE_OFFSET（3GB）之外，则该地址无效， Linux 将向进程发送一个信号并终止进程的运行
- 缺页异常
  - 虚地址有效，但其所对应的页当前不在物理内存中
  - 操作系统必须从磁盘或交换文件（此页被换出）中将其装入物理内存
- 保护错误
  - 要访问的虚地址被写保护
  - 操作系统必须判断
    - 如果是某个用户进程正在写当前进程的地址空间，则发送一个信号并终止进程的运行
    - 如果错误发生在一旧的共享页上时，则处理方法有所不同，也就是要对这一共享页进行复制，这就是曾经描述过的“写时复制”技术

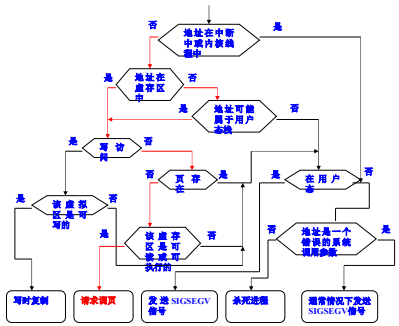
缺页异常

- i386中14号异常，由do_page_fault()完成
- 内核只是通过mmap()等调用分配一些线性地址空间给进程，并没有真正的把实际的物理页框分配给进程
  - 当进程试图访问这些分配给它的地址空间（如一段线性地址空间映射的是二进制代码）则进程被调度执行的时候会跳转到这个地址上去执行
  - 此时，并没有物理页框对应于这些线性地址，从而会引发一个缺页异常

缺页异常处理程序



缺页异常处理流程图



请求调页

- 把页面的分配推迟到进程要访问的页不在物理内存时为止，由此引起一个缺页异常
  - 引入原因
    - 进程开始运行时并不访问其地址空间中的全部地址
    - 程序的局部性原理保证请求调页从总体上使系统有更大的吞吐量
  - 处理方法
    - 通过do_no_page()/do_anonymous_page()完成

复习提要

- 理解Linux虚拟空间的划分
- 了解页描述符结构及物理内存的管理结构
- 了解非一致内存访问NUMA
- 理解Linux的管理区划分方法及与内核空间的映射关系
- 理解Linux中基于管理区的伙伴系统的基本设计思想
- 理解伙伴的定义及基于位图的伙伴管理机制
- 了解slab分配器的基本思想
- 理解Linux中slab分配器的实现机制
- 理解高端内存页框的内核映射的基本思想

作业

- 了解逻辑地址、线性地址及物理地址的概念
- 理解段选择子的结构及参数定义
- 掌握逻辑地址到线性地址的转换方法
- 理解常规分页机制
- 掌握线性地址到物理地址的转换方法
- 了解物理地址扩展的概念
- 了解Linux2.4及Linux 2.6的分页模型
- 了解线性区的概念及引入线性区的必要性
- 了解vm_area_struct结构
- 理解Linux中内存描述符的结构定义及主要参数的含义

第7章 Linux文件系统

7.1 Linux文件系统概述

7.2 虚拟文件系统VFS

- VFS的作用及设计思想
- VFS重要数据结构

7.3 文件系统类型

7.4 文件系统安装与注册

7.5 Ext2、Ext3文件系统

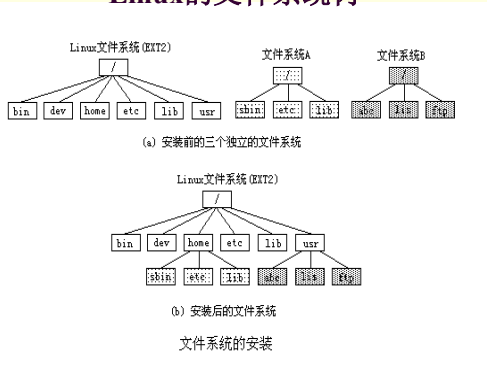
7.1 Linux文件系统概述

操作系统对多种文件系统的支持

文件系统的类型

- FAT文件系统（MS-DOS文件系统）
  它是MS-DOS操作系统使用的文件系统，它也能由Windows2000/XP、linux、SCO UNIX等操作系统访问。文件地址以FAT表结构存放，文件目录32B，文件名为8个基本名加上一个“.”和3个字符扩展名。
- 扩展文件表系统（vfat）
  它是Windows98使用的扩展的DOS文件系统，它在MS-DOS文件系统基础上增加了对长文件名（最多到256B）支持。

Linux的文件系统树



7.2 虚拟文件系统VFS－引入

- 为了保证Linux的开放性，设计人员必须考虑如何使Linux除支持Ext2文件系统外，还能支持其他各种不同的文件系统。
- 要求：须将各种不同文件系统的操作和管理纳入到一个统一的框架中，使得用户程序可以通过同一个文件系统界面，也就是同一组系统调用，能够对各种不同的文件系统以及文件进行操作。

举 例

举例：VFS在一个简单文件复制操作中的作用

- 假设用户输入以下shell命令
  $ cp /usb/TEST /tmp/test
- 其中:
  - /usb是MS-DOS的磁盘的一个挂载点（安装点）
  - /tmp是Ext2文件系统中的一个目录
- 对于cp命令而言，它不需要知道/usb/TEST和/tmp/test分别是什么文件系统类型
  - 在cp命令中，它通过VFS提供的系统调用接口进行文件操作

文件系统的类型

- NTFS（NT文件系统）
  它是Windows NT操作系统使用的文件系统，它具有很强的安全特性和文件系统恢复功能，可以处理巨大的存储媒体，支持多种文件系统。
- ext2（二级扩展文件系统）
  它是Linux操作系统使用的高性能磁盘文件系统，它是对Minux操作系统中使用的文件系统扩展（ext）的扩展。它支持256字符的文件名，最大可支持到4TB的文件系统大小。
- HPFS（高性能文件系统）
  它是OS/2操作系统使用的文件系统。

Linux文件系统的特征

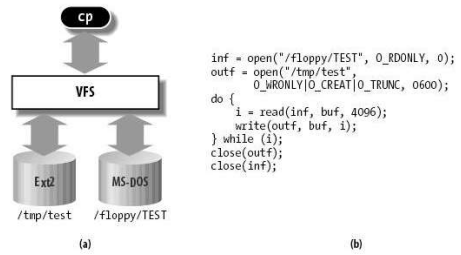
- Linux文件系统的特征
  - 树型结构
  - 文件是无结构的字符流式文件
  - 权限保护
  - 外部设备也被看作文件，如磁盘，鼠标，串行接口。设备在目录中占有一项，可以使用与文件相同的系统调用来读写设备文件。
- Linux文件系统类型
  - 普通文件
  - 目录文件：由目录项组成，每个目录项由文件名及其inode号组成
  - 设备文件：不占有实际的物理块，对设备文件的读写操作实际变为对设备的操作。
  - 管道文件，链接文件

7.2 虚拟文件系统VFS－引入

- VFS接口：如同标准的的文件系统“总线”，让用户通过同一组系统调用接口来管理和操作不同类型文件系统上的文件。这个统一的、抽象的、虚拟的文件系统接口，被称为VFS接口，它主要包含一组标准的文件操作接口
- 将内核比作计算机主板，那么VFS就是一种标准的系统总线，各种文件系统就是插在该总线上的子卡，虽然各个子卡的内部线路不同，但在总线接口处一致。 同时，内核看到的所有子卡的特性也映射为具有共性的几类。

举例

Figure 12-1. VFS role in a simple file copy operation



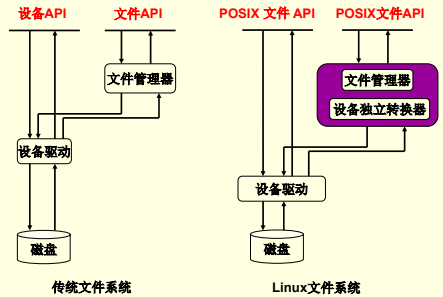
文件系统的类型

- CD-ROM文件系统(iso9660)
  它是符合ISO9660标准的支持CD-ROM的文件系统，它有High sierra CD-ROM和Rock Ridge CD-ROM二种类型。
- UDF通用磁盘格式文件系统
  UDF(Universal Disk Format)文件系统是依据光学储存技术协会（Optical Storage Technology Association, OSTA）的通用磁盘格式文件系统规格1.02版所制定的。它提供了对UDF格式媒体的只读访问（例如DVD光盘）。Windows2000/XP提供对UDF文件系统支持。

安装点

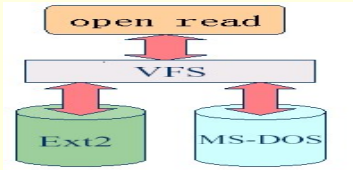
- 将一个文件系统的顶层目录挂到另一个文件系统的子目录上，使它们成为一个整体，称为“安装（mount）”。把该子目录称为“安装点(mount point)”
- EXT2是Linux的标准文件系统，系统把它的磁盘分区做为系统的根文件系统。EXT2以外的文件系统则安装在根文件系统下的某个目录下，成为系统树型结构中的一个分枝
- 安装的文件系统的文件掩盖了这个目录原来存在的内容。当这个文件系统卸载的时候，安装目录自己的文件又可以显现出来。

Linux的虚拟文件系统接口-VFS接口



Linux文件系统的基本组成

- Linux的文件系统主要可分为三大块:
  - 上层的文件系统的系统调用
  - 虚拟文件系统 VFS(Virtual File system or Switch)
  - 挂载到 VFS 中的各种实际文件系统，例如 ext2，msdos 等。



Linux的文件系统树

- 在 Linux 中，系统可以使用的不同的文件系统不是通过设备标识符（例如驱动器编号或设备名称）访问，而是连接成一个单一的树型的结构。
- Linux 在文件系统安装时把它加到这个单一的文件系统树上。所有的文件系统，都安装在一个目录树上。

7.2 虚拟文件系统VFS－引入

- Linux最初采用的是Minix的文件系统
  - Minix是一种教学用操作系统
  - 其文件系统的大小限于64MB
  - 文件名长度也限于14个字节。
- 所以，Linux经过一段时间的改进和发展，特别是吸取了Unix文件系统多年改进所积累的经验，最后形成了现在的Ext2文件系统。
- 可以说，Ext2文件系统就是Linux文件系统。

7.2 虚拟文件系统VFS－引入

- Linux与其他类Unix系统一样，采用虚拟文件系统VFS来达到支持多种文件系统格式的目标
- 虚拟文件系统(Virtual Filesystem,Virtual Filesystem Switch)
  - VFS是一个软件层，用来处理与Linux/Unix标准文件系统相关的所有系统调用。
    - 是用户应用程序与文件系统实现之间的抽象层
  - 能为各种文件系统提供一个通用的、统一的接口

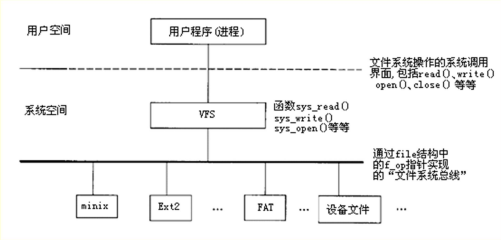
7.2 虚拟文件系统VFS

- VFS接收对文件系统的访问操作请求，并通过VFS的操作函数，按照一定的映射关系把这些访问重新定向到实际文件系统中的相应的函数调用
- 真正与物理结构有关的具体操作由实际文件系统完成。

7.2 虚拟文件系统VFS

- VFS提供上/下两类接口
  - 上层接口是提供给用户使用。这些用户包括应用程序和内核的其它管理模块。通过该接口使用I/O系统，如打开，关闭等。
  - 下层接口提供给真实文件系统。VFS支持的每个真实文件系统都提供对这个接口的实现。只要实现了这组接口，VFS就认为它是一个真实的文件系统。

7.2 虚拟文件系统VFS



VFS接口

7.2 虚拟文件系统VFS

- VFS通过对每一个具体的文件系统的所有细节进行抽象，使得Linux用户能用统一接口使用不同的文件系统。
- VFS只是一种存在于内存的文件系统，在操作系统自举时建立，在系统关闭时消亡。
- 一个实际的文件系统要想被 Linux 支持，就必须按照这个接口来编写自己的操作函数，从而将自己的细节对其他子系统隐藏起来。
- 思考：如何实现呢？

7.2 虚拟文件系统VFS

- VFS是由面向对象的思想发展起来的
  - VFS提供一个抽象基类
  - 由该基类派生出的子类支持具体的文件系统。
- 由于Linux没有使用OOP的语言，而是使用C语言开发的，因此“对象”用数据结构实现。

VFS支持的文件系统类型

- VFS支持的文件系统可以划分为三种主要类型
  - 基于磁盘的文件系统：它们管理在本地磁盘分区中可用的存储空间
    - Linux使用的文件系统：ext2、ext3、ReiserFS
    - Unix家族的文件系统：SYS V文件系统，UFS，MINIX文件系统以及VERITAS VxFS
    - 微软公司的文件系统：MS-DOS、VFAT以及NTFS
    - ISO9660CD-ROM文件系统和通用磁盘格式的DVD文件系统
    - 其他…

VFS支持的文件系统类型

- 网络文件系统：用于访问属于其他网络计算机的文件系统所包含的文件
  - NFS、Coda、AFS、SMB、NCP
- 特殊文件系统
  - 不同于上述两大类
  - 不管理具体的磁盘空间
  - /proc
- 各种不同的文件系统通过mount（挂载、安装）到根文件系统中
  - 在Linux中，根文件系统即根目录所代表的文件系统,通常是ext2文件系统

VFS所处理的系统调用

- mount、umount：挂载/卸载文件系统
- sysfs：获取文件系统信息
- statfs、fstatfs、ustat：获取文件系统统计信息
- chroot：更改根目录
- chdir、fchdir、getcwd：操纵当前工作目录
- mkdir、rmdir：创建/删除目录
- getdents、readdir、link、unlink、rename：对目录项进行操作
- readlink、symlink：对软链接进行操作
- chown、fchown、lchown：更改文件所有者
- chmod、fchmod、utime：更改文件属性
- open、close、create …

Linux通用文件模型

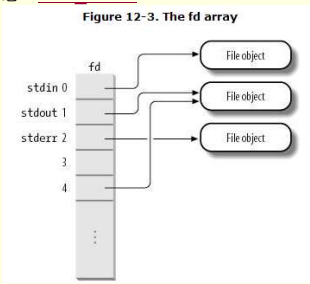
- 用于表示所有支持的文件系统，由以下对象类型组成
  - 超级块对象（superblock object）
    - 存放文件系统相关信息：例如文件系统控制块
  - 索引节点对象（inode object）
    - 存放具体文件的一般信息：文件控制块/inode
  - 文件对象（file object）
    - 存放已打开的文件和进程之间交互的信息
  - 目录项对象（dentry object）
    - 存放目录项与文件的链接信息
- 同时VFS还使用了磁盘高速缓存（软件机制），将常用的目录项对象放在目录项高速缓存中

7.3 VFS的数据结构

- 每个VFS对象都对应一定的数据结构，在这个数据结构中包含对象的属性及其方法
- 一个具体的文件系统：超级块对象：super_block
- 一个具体的文件：Inode对象： inode
- 一个打开文件：文件对象：file
- 目录项对象：dentry

与进程相关的文件

- 文件系统相关信息 fs_struct
- 打开文件相关信息 files_struct



超级块 superblock：作用

- 在文件系统安装时，VFS根据实际文件系统存放在块设备上的管理信息，在内存中建立一个VFS超级块
- VFS超级块用来描述已安装的文件系统的信息，是一个全局的数据结构
- VFS超级块存在于内存中，它在文件系统安装时建立，并且在文件系统卸载时自动删除
  - 注意：分清楚VFS超级块和各实际文件系统，如EXT2的超级块
- 对每个具体的文件系统来说,都有各自的超级块, 如Ext2超级块，它们存放于磁盘

超级块superblock

- 主要信息：
  - （1） 设备标识符。
  - （2） 索引节点指针。
  - （3） 数据块大小。
  - （4） 超级块操作集。
  - （5） 文件系统类型。
  - （6） 文件系统的特殊信息。

超级块

```
struct super_block {
struct list_head      s_list;   /*文件链表头超级块指针*/
kdev_t  s_dev;                  /* 该文件系统的主次设备号 */
  unsigned long  s_blocksize;     /*数据 块大小 */
  unsigned char  s_blocksize_bits;          /* 块大小值占的位数，以2的幂次表示
  unsigned char  s_lock;              /* 锁定标志，置位表示拒绝其他进程访问 */
  unsigned char  s_rd_only;                   /* 只读标志 */
  unsigned char  s_dirt;              /* 已修改标志，若为1，表明超级块已经被修
  struct file_system_type* s_type;            /* 指向文件系统类型注册表相应项 */
  struct super_operations* s_op; /* 指向一组操作该文件系统的函数 */
  struct dquot_operations* dq_op;
  unsigned long  s_flags;/*文件系统标志*/
  unsigned long  s_magic; /*文件系统幻数*/
  ……
  union { 各个物理文件系统超级块的结构类型 } u;  /*问津系统的私有信息*/
};
```

超级块superblock

- 所有超级块对象（每个已安装的文件系统都有一个超级块）以双向环形链表的形式链接在一起。
- 超级块对象的s_list域包含指向链表中两个相邻超级块对象的指针(s_list的 next和prev域）
- 超级块操作由数据结构super_operations来描述

超级块superblock：操作

- 在 Linux 中，超级块和设备号之间有紧密的联系。每一个文件系统必须有一个唯一的设备号，该文件系统即建立在此设备之上。
- 有一些文件系统（如/proc 文件系统）不需要真实的设备。因此，对于这些文件系统，将会自动地分配以主设备号（major number）为0的匿名设备。

超级块的U域

- 联合体数据成员super_block.u是实现支持多种文件系统的关键。它指向VFS所支持的各种具体文件系统的超级块。
- 如当在系统上安装了另一个文件系统（比如DOS的FAT）时，那么DOS的超级块就被复制到内存的msdos_sb_info结构中。
- 由于联合体类型数据成员super_block.u.msdos_sb指向该结构体，以后就允许该文件系统直接对内存超级块u域进行操作，无需再去访问磁盘，以提高系统效率。

Super block中的文件系统特有信息

```
union {    /* 联合体，各种具体文件系统超级块的内存映像 */
        struct minix_sb_info minix_sb;
        struct ext_sb_info ext_sb;
        struct ext2_sb_info ext2_sb;
        struct hpfs_sb_info hpfs_sb;
        struct msdos_sb_info msdos_sb;
        struct isofs_sb_info isofs_sb;
        struct nfs_sb_info nfs_sb;
        struct xiafs_sb_info xiafs_sb;
        struct sysv_sb_info sysv_sb;
        struct affs_sb_info affs_sb;
        struct ufs_sb_info ufs_sb;
        void *generic_sbp;
  } u;
```

如：当第一次安装一个ext2系统时,会调用ext2_read_super( ),该函数有一条语句sb->s_op = &ext2_sops。ext2_sops变量的成员值如下:

```
static struct super_operations ext2_sops = {
        read_inode:   ext2_read_inode,
        write_inode:  ext2_write_inode,
        put_inode:    ext2_put_inode,
        delete_inode: ext2_delete_inode,
        put_super:    ext2_put_super,
        write_super:  ext2_write_super,
        ......
 };
```

- static struct inode *alloc_inode(struct super_block *sb)

```
{
        static struct address_space_operations empty_aops;
        static struct inode_operations empty_iops;
        static struct file_operations empty_fops;
        struct inode *inode;
         if (sb->s_op->alloc_inode)
                inode = sb->s_op->alloc_inode(sb);
        else {
                inode = (struct inode *) kmem_cache_alloc(inode_cachep, SLAB_KERNEL);
                 if (inode)
                    memset(&inode->u, 0, sizeof(inode->u));
        }
         if (inode) {
                struct address_space * const mapping = &inode->i_data;
                inode->i_sb = sb;
                inode->i_dev = sb->s_dev;
```

VFS Inode结构

```
struct inode_operations* i_op;/* 指向一组针对该文件的操作函数 */
struct file_operations    *i_fop; /*指向文件操作操作的指针*/
struct super_block*  i_sb;          /* 指向内存中的VFS超级块 */
struct wait_queue*  i_wait;         /* 在该文件上的等待队列 */
struct file_lock*  i_flock;         /* 操作该文件的文件锁链表的首地址 */
struct vm_area_struct*  i_mmap;
struct page*  i_pages;              /* 文件所占页面构成的单向链 */
struct dquot*  i_dquot[MAXQUOTAS];
struct inode *i_next，*i_prev，  *i_hash_next，*i_hash_prev，*i_bound_to，
  *i_bound_by;
struct inode *i_mount;  /* 指向下挂文件系统的inode的根目录 */
unsigned long  i_count;                 /* 引用计数，0表示空闲 */
unsigned short  i_flags;
unsigned short  i_writecount;
unsigned char  i_lock;                  /* inode的锁定标志 */
unsigned char  i_dirt;                  /* 已修改标志 */
unsigned char  i_pipe，  i_sock，  i_seek，  i_update，  i_condemned;
union { 各个物理文件系统inode的结构类型 }  u;
```

U域

- 问题1：有可能VFS超级块最终不再与磁盘上相应的超级块同步。因此，有必要引入一个s_dirt标志，来表示该超级块是否是脏的，也就是说，磁盘上的数据是否必须要更新。
- 问题2：缺乏同步还导致一个问题：当一台机器的电源突然断开而用户来不及正常关闭系统时，就会出现文件系统崩溃。Linux是通过周期性地将所有“脏”的超级块写回磁盘来减少该问题带来的危害。

而当第一次mount一个fat分区时,会调用fat_read_super( ),该函数有一条语句sb->s_op = &fat_sops。fat_sops变量的成员值如下:

```
static struct super_operations fat_sops = {
        write_inode:  fat_write_inode,
        delete_inode: fat_delete_inode,
        put_super:    fat_put_super,
        ......
};
```

抽象与具体

super_operations就是一个抽象类，它只提供接口但并没有实现这些接口，而ext2_sops，fat_sops则是具体类，实现相应的接口。

其他几个接口与此类似。

U域:存放各种文件系统特有的Inode信息

```
union {struct pipe_inode_info pipe_i;
        struct minix_inode_info minix_i;
        struct ext_inode_info ext_i;
        struct ext2_inode_info ext2_i;
        struct hpfs_inode_info hpfs_i;
        struct msdos_inode_info msdos_i;
        struct umsdos_inode_info umsdos_i;
        struct iso_inode_info isofs_i;
        struct nfs_inode_info nfs_i;
        struct xiafs_inode_info xiafs_i;
        struct sysv_inode_info sysv_i;
        struct ufs_inode_info ufs_i;
        struct socket socket_i;
        void * generic_ip;
  } u;
```

超级块的操作

超级块操作由struct super_operations描述，其地址存放在s_op域中；

```
struct super_operations {
    struct inode *(*alloc_inode)(struct super_block * sb);
void            (*destroy_inode)(struct inode *);
    void (*read_inode) (struct inode *); //读取文件inode
    void (*write_inode) (struct inode *); //回写文件inode
    void (*put_inode) (struct inode *); //逻辑上释放inode
    void (*delete_inode) (struct inode *); //物理上释放inode
    void (*put_super) (struct super_block *); //释放超级块对象
    void (*write_super) (struct super_block *); //把超级块信息写回磁盘
  int (*statfs) (struct super_block *, struct statfs *);
   int (*remount_fs) (struct super_block *, int *, char *);
```

- 分析“获取一个 inode ”的过程来只理解这个“接口”中两个成员 alloc_inode 和 read_inode 的作用。
- 在文件系统的操作中，经常需要获得一个“目录节点”对应的 inode，这个 inode 有可能已经存在于内存中了，也可能还没有，需要创建一个新的 inode，并从磁盘上读取相应的信息来填充。
- 对应的代码是 iget() （inlcude/linux/fs.h）过程如下：
  1、 通过 iget4_locked() 获取 inode。如果 inode 在内存中已经存在，则直接返回；否则创建一个新的 inode
  2、 如果是新创建的 inode，通过 super_block->s_op->read_inode() 来填充它。也就是说，如何填充一个新创建的 inode， 是由具体文件系统提供的函数实现的。

VFS索引节点 VFS Inode

- VFS 中的每个文件、目录等都用且只用一个VFS inode表示，存放关于具体文件或者目录的一般信息。
- 每个索引节点对象都有一个索引节点号,这个号唯一地标识某个文件系统中的指定文件。
- 文件名可以随时更改，但是索引节点对文件是唯一的，并且随文件的存在而存在。
- 具体文件系统的索引节点是存放在磁盘上的，是一种静态结构，要使用它，必须调入内存，填写VFS的索引节点，因此，也称VFS索引节点是动态节点
- 每一个索引节点都由一个“struct inode”结构标识，它的一组方法保存在结构"struct inode_operations"中。

inode.U

- 同超级块一样，inode.u用于存储每一个特定文件系统的特定inode。
- 如果索引节点指向的是一个ext2文件，那么该U域就指向一个ext2_inode_info结构；
- 每个索引节点都要复制磁盘索引节点得到，譬如文件占用的磁盘块等，如果i_dirt域的值为I_DIRTY，则该索引节点对应的磁盘索引节点必须更新；

每个具体文件系统都应该提供这些超级块操作的具体实现，这些操作可以实现文件系统的挂载、读写inode等；

read_inode： 该函数从已mount的文件系统读入一个inode的信息。该inode对象的inode号已事先被初始化。
write_inode: 把inode信息写入磁盘。
put_inode: inode对象从inode cache移去时调用该函数。
delete_inode: 删除inode时调用。
put_super: umount时，VFS释放超级块。
write_super: 希望把VFS超级块写入磁盘时调用。

- iget4_locked() 首先在全局的 inode hash table 中寻找，如果找不到，则调用 get_new_inode() ，进而调用 alloc_inode() 来创建一个新的 inode
- 在 alloc_inode() 中可以看到，如果具体文件系统提供了创建 inode 的方法，则由具体文件系统来负责创建，否则采用系统默认的的创建方法。

VFS inode结构

```
struct inode {
     struct list_head   i_hash;
      struct list_head  i_list;
     struct list_head   i_dentry;
     Unsigned long      i_ino;
     unsigned int       i_count;
 kdev_t  i_dev;                  /* 该文件系统的主设备号 */
 umode_t  i_mode;                /* 文件类型以及存取权限 */
 ulink_t  i_nlink;               /* 连接到该文件的link数 */
 uid_t  i_uid;
 gid_t  i_gid;
 kdev_t  i_rdev;                 /* 该文件系统的次设备号 */
 off_t  i_size;                  /* 文件长度 */
 time_t  i_atime，  i_mtime，  i_ctime;
 unsigned long  i_blocksize，  i_blocks;  /* 字节/块为单位的文件长度 */
 unsigned long  i_version;
 unsigned long  i_nrpages;       /* 文件所占的内存页数 */
 struct semaphore  i_sem;
```

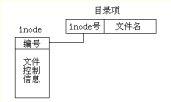
与VFS inode节点相关的操作

```
struct   inode_operations {
Struct file_operations * default_file_ops;
int(*create)(struct inode *,struct dentry *,int);
 struct dentry * (*lookup) (struct inode *,struct  dentry *);
 int(*link)(struct dentry *,struct inode *,struct dentry *);
 int (*unlink) (struct inode *,struct dentry *);
 int (*symlink)(struct inode *,struct dentry *,const char *);
 ...
};
```

create:第一个参数必须是目录型文件对应的inode。create在该目录下创建一个新的索引节点，主要是由open()系统调用来调用
lookup:第一个参数是目录型文件对应的inode。lookup（）在该目录下寻找dentry对象所对应的索引节点

VFS的目录项（ dentry）

- VFS inode中没有与路径相关的信息，也没有文件名。文件名记录在它所属的目录中。
- 一个目录是一个特殊的文件，其内容是该目录所包含的文件名和子目录名。
- 由于一个文件可以出现在多个目录中，因而一个文件可以有多个文件名、路径名，仅用VFS inode建立关系不现实，引入dentry结构描述目录与文件的关系树。
- Linux为每一个目录建立一个目录项结构，而且也为每个文件建立一个目录项结构。
- 目录项结构中包含很多信息：可以建立目录、子目录、文件之间的关系，利用目录项关系，加快



Linux文件系统的目录项

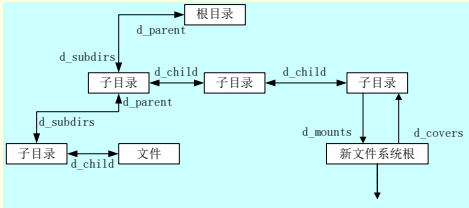
VFS的目录项(dentry)

- 引入目录项的概念主要是出于方便查找文件的目的。
- 一个路径的各个组成部分，不管是目录还是普通的文件，都是一个目录项对象。如，在路径/home/source/test.c中，目录 /, home, source和文件 test.c都对应一个目录项对象。
- 但是，目录项对象没有对应的磁盘数据结构，VFS在遍历路径名的过程中现场将它们逐个地解析成目录项对象。

dentry

```
struct dentry {
  atomic_t d_count;        /* 目录项引用计数器 */
  unsigned int d_flags;    /* 目录项标志 */
  struct inode * d_inode;  /* 与文件名关联的索引节点 */
  struct dentry * d_parent;     /* 父目录的目录项 */
  struct list_head d_hash;      /* 目录项形成的哈希表 */
  struct list_head d_lru;       /*未使用的 LRU 链表 */
  struct list_head d_child;     /*父目录的子目录项所形成的链表 */
  struct list_head d_subdirs;   /* 该目录项的子目录所形成的链表*/
  struct list_head d_alias;     /* 索引节点别名的链表*/
  int d_mounted;                /* 目录项的安装点 */
  struct qstr d_name;           /* 目录项名（可快速查找） */
  struct dentry_operations  *d_op;/* 操作目录项的函数*/
  struct super_block * d_sb;    /* 目录项树的根（即文件的超级块）*/
  unsigned long d_vfs_flags;
  void * d_fsdata;              /* 具体文件系统的数据 */
 ……
};
```

dentry



dentry

- VFS inode和它的目录项合起来，才能完整地表示一个文件或者目录。

注意：目录项在磁盘没有对应的映像，所以在Dentry中不包含“脏”域

为了加快文件的查找，每一个曾被读取的目录或文件都可能在目录高速缓存（directory cache）中有一个dentry项；

目录项dentry操作

```
struct   dentry_operations {
  int (*d_revalidate)(struct dentry *, int);
  int (*d_hash) (struct dentry *, struct qstr *);
  int (*d_compare) (struct dentry *,
                struct qstr *, struct qstr *);
        ……
};
```

d_revalidate: 检查dentry cache中的dentry是否是最新的。

d_hash:计算文件名的hash值。第一个参数是父目录,第二个参数包含了预设的文件名。

d_compare: 比较一个目录下的两个文件名是否相等。如文件名是否区分大小写。

文件 操作struct file_operations

```
struct  file_operations {
  loff_t (*llseek) (struct file *, loff_t, int);
  ssize_t (*read) (struct file *, char *, size_t, loff_t *);
  ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
  int (*mmap) (struct file *, struct vm_area_struct *);
  int (*open) (struct inode *, struct file *);
  ……
};
```

7.4 文件系统的注册与安装

- 在使用一个文件系统之前，必须要对该文件系统进行注册。注册时机：
  - 系统初启时：内核在编译时就已经确定所支持的文件系统类型。
  - 把某个文件系统作为一个内核模块显示安装时注册，在模块卸载的时候注销。
- 在VFS 中，每一个文件系统类型都使用一个“struct file_system_type”结构来表示，在该结构中，只包含一个“read_super”方法，用于读出该文件系统在辅存的超级块

file_system_type结构

```
static struct file_system_type *file_systems =
 (struct file_system_type *) NULL;
struct file_system_type {
      struct super_block *(*read_super)();
      const char *name; /* 文件系统的类型名 */
      int requires_dev; /* 文支持文件系统的设备 */
      struct file_system_type * next;
  /* 文件系统类型链表的后续指针 */
};
```

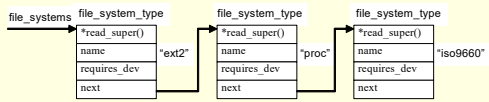
文件系统的注册

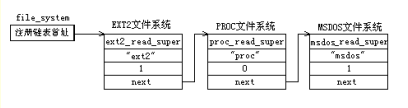
- 内核将每个已注册的文件系统都存放在file_system_type链表中。该链表的头部由file_systems指示。
- 在系统引导时，由filesystem_setup()来注册。它根据 file_system_type来调用注册函数register_filesystem()，将该文件系统插入到文件系统类型注册链表中。
  - register_filesystem()
  - unregister_filesystem()

文件系统类型的注册

- 文件系统类型： file_system_type
- 在系统初始化期间， register_filesystem()用来注册编译时指定的每个文件系统
  - 相应的文件系统对象被插入到file_systems链表中
- unregister_filesystem()

文件系统的注册





Linux的注册链表

```
static struct file_system_type ext2_fs_type = {
        ext2_read_super, "ext2", 1, NULL}
```

文件系统的初始化函数

init_minix_fs()　minix文件系统初始化函数；
init_msdos_fs()　msdos文件系统初始化函数；
init_proc_fs()　proc文件系统初始化函数；
init_sysv_fs()　sysv文件系统初始化函数；

在文件系统初始化函数中，把注册结构体做为参数，调用由内核提供的注册函数register_filesystem（）

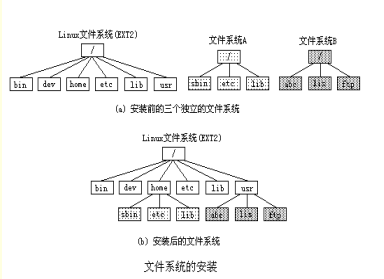
EXT2文件系统的初始化函数

```
int init_ext2_fs(void)
{
        return register_filesystem(&ext2_fs_type);
}
int register_filesystem(struct file_system_type * fs)
{                                   /* 参数fs是注册结构体首址 */
        struct file_system_type ** tmp;

        if (!fs)
                return -EINVAL;  /* 不支持该文件系统，出错返回 */
        if (fs->next)
                return -EBUSY;   /* 文件系统已注册，返回 */
        tmp = &file_systems;     /* 得到注册链表首址 */
        while (*tmp) {           /* 从注册链表首开始遍历 */
                if (strcmp((*tmp)->name, fs->name) == 0)
                        return -EBUSY;        /* 文件系统已注册，返回 */
                tmp = &(*tmp)->next;          /* 查看下一个注册结构体 */
        }
        *tmp = fs;      /* 把要注册的文件系统的注册结构体加到链表 */
        return 0;       /* 注册成功，返回0 */
}
```

文件系统的安装

- 文件系统除在VFS中注册，还必须安装到系统中。因为表示文件系统的超级块还没有建立起来。文件系统的安装用于建立它的VFS超级块。
- 要安装的文件系统必须已经存在于外存磁盘空间上，每个文件系统占用一个独立的磁盘分区，并且具有各自的树型层次结构。
- 由于EXT2是Linux的标准文件系统，所以系统把EXT2文件系统的磁盘分区做为系统的根文件系统
- EXT2以外的文件系统则安装在根文件系统下的某个目录下，成为系统树型结构中的一个分支。

文件系统的安装



文件系统的安装

文件系统的安装

- 文件系统的安装和卸载一般只能由超级用户root完成。
- 在安装文件系统时，要指定3种信息：文件系统的名称、包含文件系统的物理块设备和文件系统在已有文件系统中的安装点

$ mount -t iso9660 /dev/hdc /mnt/cdrom
安装的文件系统类型、文件系统所在设备以及挂装点。

已安装文件系统

格式：#mount –t type device mount-point
例如：放在软盘/dev/fd0上的ext2文件系统，可通过下列命令安装在/flp上
mount -t ext2 /dev/fd0 /flp
例如：放在软盘/dev/fd0上的ext2文件系统，可通过下列命令安装在/flp上
mount –t msdos /dev/fd0 /mnt
使用拆卸的命令卸下软盘文件系统:格式：#umount device
例： #umount /dev/fd0

- 一般情况下，只允许挂载一次
  在umount之前，其他对/dev/fd0的挂载命令会失败
- 在2.4中，可以挂载多次
  - 不管被挂载多少次，只有一个真实的文件系统，因此只有一个超级块对象

文件系统的注册、安装过程

- (1)寻找对应的文件系统信息。VFS 通过 file_systems 在 file_system_type 组成的链表中根据指定的文件系统名称搜索文件系统类型信息。见 fs/super.c中的get_fs_type()函数。
- (2) 如果在上述链表中找到匹配的文件系统，则说明内核具有对该文件系统的内建支持。否则，说明该文件系统可能由可装载模块支持，VFS 会请求内核装入相应的文件系统模块，此时，该文件系统在 VFS 中注册并初始化。
- (3) 不管是哪种情况，如果 VFS 无法找到指定的文件系统，则返回错误。
- (4) VFS 检验给定的物理块设备是否已经安装。如果指定的块设备已被安装，则返回错误。

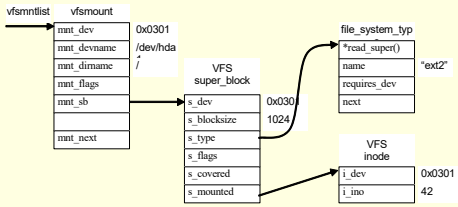
文件系统安装（续）

- 安装一般文件系统，系统调用mount()->sys_mount()->do_mount()：
  1、查找要安装到的目录的目录项
  2、优先针对三种安装标志参数值执行相应的操作
  MS_REMOUNT：do_remount()重新安装改变了安装标志的文件系统
  MS_BIND：调用do_loopback()完成“绑定安装”，使系统目录树中的另一个目录能看见
  MS_MOVE：调用do_move_mount()改变已安装文件系统的目录
  3、否则，调用do_add_mount()安装普通系统

文件系统的安装：几种途径

- 如果要在系统启动时自动安装文件系统，在/etc/fstab 文件中为每种文件系统的安装提供有关文件系统的名称、包含该文件系统的物理设备以及该文件系统的安装点的信息。
- 当Linux自举时，首先装入根文件系统，然后根据/etc/fstab中的登记项，使用mount命令自动逐个安装文件系统。
- 此外用户也可以显式地通过mount和umount命令安装和卸装文件系统。

已安装的文件系统数据结构(1)



文件系统的注册、安装过程

- (5) VFS 查找作为新文件系统安装点的目录的 VFS 索引节点。该 VFS 索引节点可能在索引节点高速缓存中，也有可能需要从安装点所在的块设备中读取。
- (6) 如果该安装点已经安装有其他文件系统，则返回错误。因为同一目录只能同时安装一个文件系统。
- (7) 为新的文件系统分配超块，并将安装信息传递给该文件系统的超块读取例程。该例程将对应文件系统的信息映射到 VFS 超块中。如果在此过程中发生错误，则返回错误。

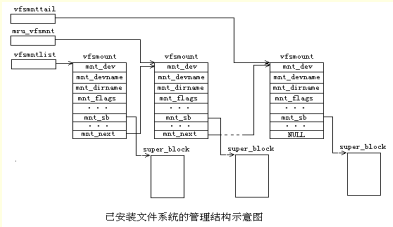
文件系统安装（续）

- 卸载文件系统，系统调用umount()->sys_umount()：
  1、查找文件系统目录的目录项对象
  2、以下情况返回出错：
  - 传入的目录参数不是文件系统的目录
  - 文件系统没有安装在目录下
  - 用户不具有卸载文件系统的特权
  3、调用do_umount()卸载文件系统

文件系统安装：数据结构

- 结构类型为vfsmount的数据结构描述每个已安装文件描述符，内核中是一个双向链表，首指针为mnt_list
- 可使用的操作
  - alloc_vfsmnt()：分配并初始化已安装文件系统描述符
  - free_vfsmnt()：释放指定的已安装文件系统描述符
  - lookup_mnt()：在散列表中查找描述符并返回它的地址

已安装的文件系统数据结构(2)



已安装文件系统

- 已挂载文件系统描述符 vfsmount
- 挂载根文件系统
  - 第一阶段：安装一个特殊的文件系统，该文件系统仅提供一个作为初始安装点的空目录：init_mount_tree
  - 第二阶段：mount_root
- 挂载一个文件系统
  - sys_mount
- 卸载一个文件系统
  - sys_umount

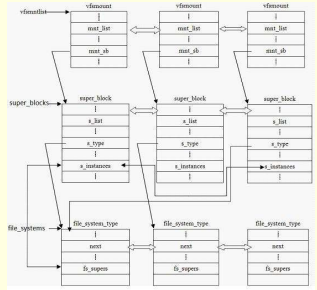
7.5 路径名查找

- VFS是如何从文件路径名找到相应的索引节点的？
  - 分析路径名，将它拆分成一个文件名序列
  - 除了最后一个文件名之外，其他所有文件名必定都是目录名
- 搜索的起点：
  - 绝对路径：current→fs→root初始化 更改
  - 相对路径：current→fs→pwd 更改

已安装文件系统的描述：vfsmount结构

```
struct vfsmount {
  kdev_t  mnt_dev;   /* 文件系统所在的主次设备号 */
  char*  mnt_devname;/*文件系统所在的设备名，*/
  char*  mnt_dirname;/* 安装目录名 */
  unsigned int  mnt_flags;/* 设备标志 */
  struct semaphore  mnt_sem;/* 设备有关的信号量 */
  struct super_block*  mnt_sb;/* 指向超级块 */
  struct vfsmount* mnt_next; /* 后继指针 */
};
```

- 每个已安装的文件系统由 vfsmount 结构描述，所有的 vfsmount 结构形成了一个链表，该链表头由 vfsmntlist 指针代表。
- 系统中还有两个指针，vfsmnttail 和 mru_vfsmnt，分别指向链表尾和最近使用过的 vfsmount 结构。



文件系统安装（续）

- 根文件系统的安装：
  1、安装特殊文件系统rootfs，用来提供作为初始安装点的空目录，主要由函数init_mount_tree()完成
  2、在空目录上安装一个真正的根目录，由函数mount_root()完成

why rootfs?
Linux允许根文件系统存放在很多不同的地方，因此内核在启动时可能会安装和卸载多个根文件系统，使用rootfs可以很容易改变实际的根目录

路径名查找

- 首先找到起点目录的索引节点
- 在这个索引节点的目录文件中，找到第一个目录名（文件名）所对应的索引节点
- 在第一个目录名的索引节点的目录文件中找到第二个目录名（文件名）所对应的索引节点
- …反复，直到文件名序列的最后一项

路径名查找

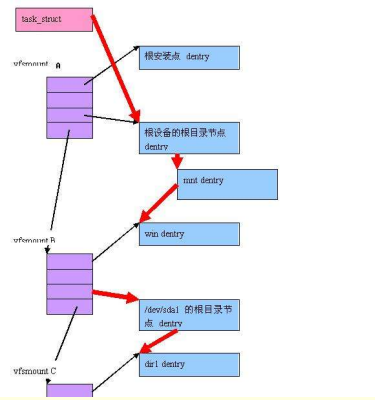
- 在上述查找过程中要处理的其他问题：
  - 对每个目录的访问权限必须进行检查
  - 若是符号链接，需要进行扩展
  - 要考虑符号链接的循环引用（进行处理）
  - 目录名可能是一个文件系统的安装点，必须扩展到一个新的文件系统中

路径名查找

- 数据结构：struct nameidata

```
struct nameidata {
struct dentry *dentry; /*目录项对象的地址*/
struct vfsmount *mnt; /*安装点的数据*/
struct qstr last; /*路径中的最后一个component*/
unsigned int flags; /*查找标识*/
int last_type; /*路径中的最后一个component的类型*/
unsigned depth; /*当前symbolic link的嵌套深度，不能大于6*/
char *saved_names[MAX_NESTED_LINKS + 1];
            /*和嵌套symbolic link 相关的pathname*/
union {
struct open_intent open; /*说明文件该如何访问*/
} intent; /*专用数据*/ };
```

- 在需要进行路径名查找的时候，依此调用如下函数：path_init, path_walk, path_release



路径名查找：跨文件系统

- 例如:要打开 /mnt/win/dir1/abc 这个文件：
  1、 首先找到根文件系统的根目录节点 dentry 和 inode
  2、 由这个 inode 提供的操作接口 i_op->lookup()，找到下一层节点 ‘mnt’ 的 dentry 和 inode
  3、 由 ‘mnt’ 的 inode 找到 ‘win’ 的 dentry 和 inode
  4、 由于 ‘win’ 是个“安装点”，因此需要找到“被安装设备”/dev/sda1 根目录节点的 dentry 和 inode，只要找到 其 vfsmount，就可以完成这个任务。
  5、 然后由 /dev/sda1 根目录节点的 inode 负责找到下一层节点 ‘dir1’ 的 dentry 和 inode
  6、 由于 dir1 是个“安装点”，因此需要借助 vfsmount C 找到 /dev/sda2 的根目录节点 dentry 和 inode
  7、 最后由这个 inode 负责找到 ‘abc’ 的 dentry 和 inode
  可以看到，整个寻找过程是一个递归的过程。

路径名查找：举例

- 例： /usr/include/stdio.h
- 从当前进程的根目录项中（current→fs → root）找到它的根目录的inode编号，根据该编号和超级块的s_inodes_per_group，计算出该inode所在的块组，查找该块组中的inode 表，找到描述根目录文件的inode，根据该inode的描述，读取其数据块（得到目录项列表）

文件查找（续）

- 搜索名为“usr”的目录项，得到对应的inode编号
- 按照该编号，以及超级块的s_inodes_per_group，计算出该inode所在的块组，查找块组的inode表，找到目录/usr对应的inode，根据该inode的描述，读取其数据块（得到目录项列表）
- 在/usr的目录项列表中找出名为“include”的目录项，得到对应的inode编号

文件查找（续）

- 按照该编号，以及超级块的s_inodes_per_group，计算出该inode所在的块组，查找块组的inode表，找到目录/usr对应的inode，根据该inode的描述，读取其数据块（又得到目录项列表）
- 在/usr/include的目录项列表中找出名为“stdio.h”的目录项，得到对应的inode编号

文件查找（续）

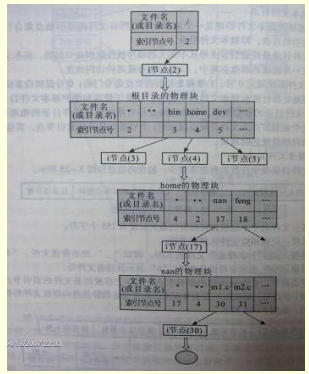
- 按照该编号，以及超级块的s_inodes_per_group，计算出该inode所在的块组，查找块组的inode表，找到文件/usr/include/stdio.h对应的inode，根据该inode的描述，读取其数据块

7.6 Linux文件和目录访问

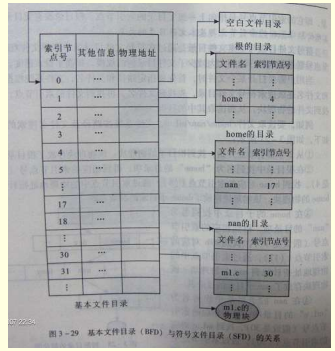
- 为了在虚拟文件系统中定位给定文件的索引节点，VFS 必须每次分解文件路径名中的一个目录名，并依次搜索每个中间目录名的 VFS 索引节点。
- 因为 VFS 在 超块中记录了每个文件系统根的 VFS 索引节点，因此搜索过程从根索引节点开始。
- 在搜索目录索引节点时，VFS 在目录高速缓存中搜索，如果能够在目录高速缓存中找到对应的inode编号，则在索引节点高速缓存中搜索索引节点，否则从底层文件系统中获取.

Linux文件和目录访问

- 当用户给出一个完整的路径名，比如 /home/dy/linux/kernel/sched.c：
- 首先得到文件系统根目录对应的inode，读取其中的信息，根据文件类型判定是一个目录文件，在这个inode保存的目录项列表中找到名称为home的那一项，查找该目录项就可以得到这个子目录对应的inode，然后再读取该inode的信息；
- 每一个目录文件中都保存着自己所包含文件和直接子目录的目录项，逐个查找，最终可以找到所需要文件的inode号，访问相应的inode，通过该inode的索引表，得到存储文件内容的物理块，就得到了指定文件的内容。



- 例：要读取文件 /home/nan/m1.c



VFS系统调用的实现

- 仍然考虑
  $ cp /floppy/TEST /tmp/test
- 假定cp执行下列代码片段（实际要更复杂）
  - inf = open("/floppy/TEST", O_RDONLY, 0);
    outf = open("/tmp/test", O_WRONLY | O_CREAT | O_TRUNC, 0600);
    do {
      len = read(inf, buf, 4096);
      write(outf, buf, len);
    } while (len);
    close(outf);
    close(inf);

```
asmlinkage long sys_open(const char * filename, int flags, int mode)
{
    char * tmp;
    int fd, error;

#if BITS_PER_LONG != 32
    flags |= O_LARGEFILE;
#endif
    tmp = getname(filename);
    fd = PTR_ERR(tmp);
    if (!IS_ERR(tmp)) {
        fd = get_unused_fd();
        if (fd >= 0) {
            struct file *f = filp_open(tmp, flags, mode);
            error = PTR_ERR(f);
            if (IS_ERR(f))
                goto out_error;
            fd_install(fd, f);
        }
out:
        putname(tmp);
    }
    return fd;

out_error:
    put_unused_fd(fd);
    fd = error;
    goto out;
}
```

1、读取文件名到内核中
2、分配一个新的文件描述符fd
3、调用filp_open打开文件，返回文件对象指针
4、建立fd与文件对象指针的联系

```
/*
 * Note that while the flag value (low two bits) for sys_open means:
 *  00 - read-only
 *  01 - write-only
 *  10 - read-write
 *  11 - special
 * it is changed into
 *  00 - no permissions needed
 *  01 - read-permission
 *  10 - write-permission
 *  11 - read-write
 * for the internal routines (ie open_namei()/follow_link() etc). 00 is
 * used by symlinks.
 */
struct file *filp_open(const char * filename, int flags, int mode)
{
    int namei_flags, error;
    struct nameidata nd;

    namei_flags = flags;
    if ((namei_flags+1) & O_ACCMODE)
        namei_flags++;
    if (namei_flags & O_TRUNC)
        namei_flags |= 2;

    error = open_namei(filename, namei_flags, mode, &nd);
    if (!error)
        return dentry_open(nd.dentry, nd.mnt, flags);

    return ERR_PTR(error);
}
```

1、设置访问模式标志
2、调用open_namei
3、调用dentry_open

read和write系统调用

```
asmlinkage ssize_t sys_read(unsigned int fd, char * buf, size_t count)
{
    ssize_t ret;
    struct file * file;

    ret = -EBADF;
    file = fget(fd);
    if (file) {
        if (file->f_mode & FMODE_READ) {
            ret = locks_verify_area(FLOCK_VERIFY_READ, file->f_dentry->d_inode,
                    file, file->f_pos, count);
            if (!ret) {
                ssize_t (*read)(struct file *, char *, size_t, loff_t *);
                ret = -EINVAL;
                if (file->f_op && (read = file->f_op->read) != NULL)
                    ret = read(file, buf, count, &file->f_pos);
            }
        }
        if (ret > 0)
            dnotify_parent(file->f_dentry, DN_ACCESS);
        fput(file);
    }
    return ret;
}
```

1、获得文件对象指针
2、判断是否有允许所请求的访问
3、是否有强制锁
4、调用具体读操作
5、释放文件对象指针

Read和write几乎相同，区别在于是读还是写

close系统调用

```
/*
 * Careful here! We test whether the file pointer is NULL before
 * releasing the fd. This ensures that one clone task can't release
 * an fd while another clone is opening it.
 */
asmlinkage long sys_close(unsigned int fd)
{
    struct file * filp;
    struct files_struct *files = current->files;

    write_lock(&files->file_lock);
    if (fd >= files->max_fds)
        goto out_unlock;
    filp = files->fd[fd];
    if (!filp)
        goto out_unlock;
    files->fd[fd] = NULL;
    FD_CLR(fd, files->close_on_exec);
    __put_unused_fd(files, fd);
    write_unlock(&files->file_lock);
    return filp_close(filp, files);

out_unlock:
    write_unlock(&files->file_lock);
    return -EBADF;
}
```

1、检查fd的正确性和有效性

2、释放文件描述符

3、调用filp_close

```
/*
 * "id" is the POSIX thread ID. We use the
 * files pointer for this..
 */
int filp_close(struct file *filp, fl_owner_t id)
{
    int retval;

    if (!file_count(filp)) {
        printk(KERN_ERR "VFS: Close: file count is 0\n");
        return 0;
    }
    retval = 0;
    if (filp->f_op && filp->f_op->flush) {
        lock_kernel();
        retval = filp->f_op->flush(filp);
        unlock_kernel();
    }
    fcntl_dirnotify(0, filp, 0);
    locks_remove_posix(filp, id);
    fput(filp);
    return retval;
}
```

1、调用文件操作的flush方法

2、释放文件上的任何强制锁

3、调用fput释放文件对象

7.7 进程与文件系统的联系

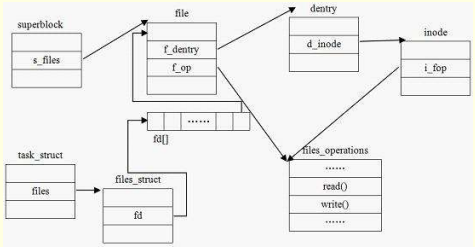
- 三个与进程相关的结构
  - files_struct：描述被进程打开使用的文件信息。fd成员指向一个struct file 数组，数组中的已使用项指向某个正被打开的文件对象，数组中的索引称为文件号，用户程序就是使用文件号对打开的文件进行操作---“进程打开文件表”
  - fs_struct：主要有两个指向索引节点的指针，pwd指向当前工作目录的索引节点；root指向当前工作目录所在文件系统的根目录的索引节点--“描述进程与文件系统的关系 ”
  - file结构：表示文件，其中有一组操作函数，保存在结构“struct file_operations”中，它在linux/fs.h中被定义。这些操作用来将VFS对file结构的操作转化为文件系统处理相应操作的函数。

系统打开文件表

- 系统打开文件表：Linux系统内核把所有进程打开的文件集中管理， 把它们组成“系统打开文件表”。
  - 系统打开文件表是一个双向链表，它的每个表项（节点）是一个file结构，称为文件描述符，其中存放着一个已打开文件的管理控制信息
  - 进程每次打开一个文件就建立一个file结构体，并把它加入到系统打开文件链表中
  - 全局变量first_file指向系统打开文件表的表头。

file结构

```
Struct file {
struct list_head        f_list;
    struct dentry       *f_dentry; /*指向文件所在目录*/
    struct vfsmount     *f_vfsmnt;
    atomic_t            f_count; /* 共享该结构体的计数值 */
    unsigned int        f_flags;
    mode_t f_mode;  /* 文件的打开模式 */
    loff_t f_pos;   /* 文件的当前读写位置 */
    unsigned short f_flags; /* 文件操作标志 */
    unsigned long f_reada, f_ramax, f_raend, f_ralen, f_rawin;
    struct file *f_next, *f_prev; /* 链接前后节点的指针 */
    struct fown_struct f_owner;  /* SIGIO用PID */
    struct file_operations * f_op;/* 指向文件操作结构体的指针 */
    unsigned long f_version;     /* 文件版本 */
    void *private_data;          /* 指向与文件管理模块有关的私有数据的指针 */
....
};
```




```
struct files_struct {
  int count;  /* 共享该结构的计数值 */
  fd_set close_on_exec;
  fd_set open_fds;
  struct file * fd[NR_OPEN];
.....
};
```

进程打开文件表- files_struct

fd[]每个元素是一个指向file结构体的指针，该数组称为进程打开文件表。

进程每打开一个文件时，建立一个file结构体，并加入到系统打开文件表中，然后把该file结构体的首地址写入fd[]数组的第一个空闲元素中。一个进程所有打开的文件都记载在fd[]数组中。fd[]数组的下标称为文件标识号。

在Linux中，进程使用文件名打开一个文件。 在此之后对文件的识别就不再使用文件名，而直接使用文件标识号。

在系统启动时文件标识号0、1、2由系统分配:0标准输入设备，1标准输出设备，2标准错误输出设备。

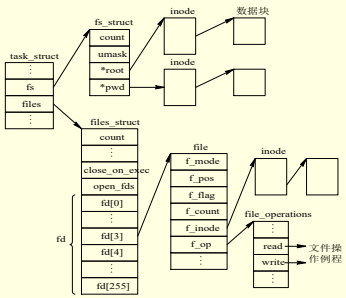
当一个进程通过fork()创建一个子进程后，子进程共享父进程的打开文件表，两者的打开文件表中下标相同的两个元素指向同一个file结构。 这时file的f_count计数值增1。

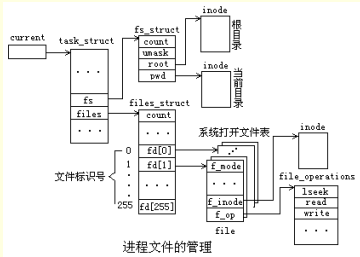
一个文件可以被某个进程多次打开，每次都分配一个file结构，并占用该进程打开文件表fd[]的一项，得到一个文件标识号。但它们的file中的f_inode都指向同一个inode。

fs_struct结构

fs_struct结构描述进程与文件系统的关系，其定义为：

```
struct fs_struct {
  atomic_t count;            //共享同一个fs_struct表的进程数
  rwlock_t lock;             //保护此结构体的锁
  int umask;                 //默认的文件访问权限
  struct dentry *root;       //根目录的目录项对象
struct dentry *pwd,;         //当前工作目录的目录项对象
struct dentry *altroot;      //可替换的根目录的目录项对象
struct vfsmount *rootmnt;    //根目录的安装点对象
struct vfsmount *pwdmnt;     //当前工作目录的安装点对象
struct vfsmount *altrootmnt; //可替换的根目录的安装点对象
};
```



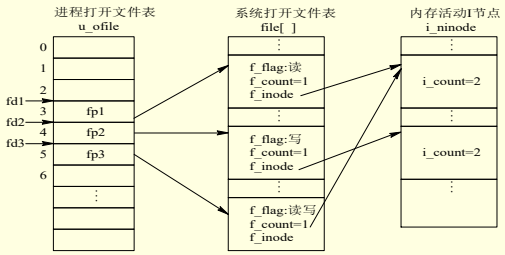


进程文件的管理

用户打开文件表与系统打开文件表之间的关系

- 例:一个进程执行如下代码：

```
fd1=open("/etc/passwd",o_RDONLY);
        /*以只读方式打开文件/etc/passwd */
fd2=open("pocal",o_WRONLY);
        /*以写方式打开文件pocal */
fd3=open("/etc/passwd",o_RDWR);
        /*以读写方式打开文件/etc/passwd */
```



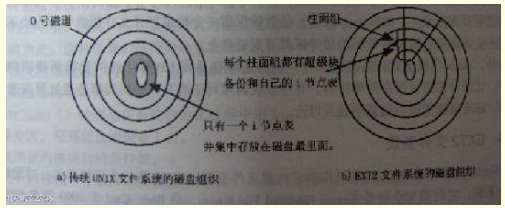
用户打开文件表与系统打开文件表间的关系

7.8 EXT2文件系统

- EXT（1992年）和EXT2（1994年）是专门为Linux设计的可扩展的文件系统，在Linux启动时作为根文件系统自动安装。
- EXT2文件系统支持标准LINUX文件类型： 普通文件、目录文件、 特别文件和符号链接。
- 管理大分区：一个分区的容量最大可达4TB
- 支持长文件名：最多可达255个字符，并且可扩展到1012个字符
- 支持文件系统数据“即时同步”特性：即内存中的数据一旦改变，立即更新硬盘上的数据使之一致
- 允许用户定制文件系统的数据单元（block）的大小：可以是 1024、2048 或 4096 个字节，使之适应不同环境的要求

Linux的磁盘组织

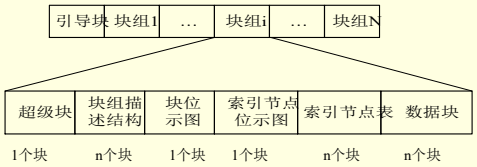
- 一个磁盘分区是由一组连续的柱面组成。
- 传统的Unix文件系统中：
  - 访问文件需两次磁盘访问：一次读i节点，然后再读文件;
  - 所有的i节点都集中放在文件系统开始的位置，其余的磁道存放数据块，因而两次磁盘操作需要花费较长的寻道时间。
- Linux的改进：
  - 将磁盘分区再分成多个柱面组，每个柱面组有一个或几个连续的柱面组成。
  - 每个柱面组由超级块、i节点表和空闲块表组成。
  - 将与i节点相关的文件数据与该i节点存放在同一个柱面内，减少寻道时间。
  - 在文件创建时，任选一个i节点，并优先在该i节点所在的柱面中分配数据块。

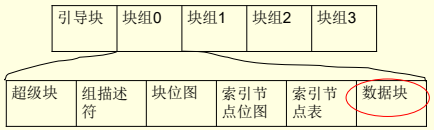


Linux减少碎片的方案

- 一个磁盘块是一次磁盘I/O操作的存取单位。在EXT2文件系统中，磁盘块的大小是2的整数次幂，一般是4K字节。
  - 磁盘块大，每次I/O操作能够传输更多的数据。
  - 但是当文件的大小不是磁盘块的整数倍时，会有磁盘碎片。
- EXT2将一个磁盘块划分成若干片(fragment)
  - 每个片的大小是512B，等于一个扇区的大小。
  - 每个片能够单独分配和寻址。
  - 片的大小在文件系统被创建时确定。

EXT2文件系统的物理结构





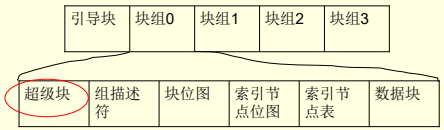
文件类型	数据块的使用方式
普通文件	有数据时才使用，创建时不需要数据块
目录	数据块中存放各目录项的数据结构
符号链接	路径名大于60个字符时需要使用数据块
设备文件	不需要数据块
管道	不需要数据块
套接字	不需要数据块

EXT2的超级块

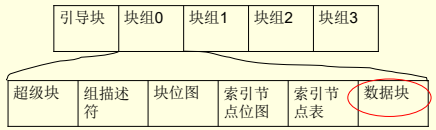
- 每个块组都有一个超级块，一般来说只有组0的超级块才被读入内存超级块，其它块组的超级块仅仅作为备份。当组0的拷贝遭到损坏时，便可根据其他组的拷贝恢复
- 多个块组中的超级块形成冗余
  - 在某个或少数几个超级块被破坏时，可用于恢复被破坏的超级块信息。

Linux减少碎片的方案

- 当文件大小不是2的整数次幂(物理块)时，如在最后一个磁盘块中只占用了一个片，该磁盘块中的其他片也可以分配给别的文件使用。
- 问题：当一个文件的大小不断增长时，需要寻找另外的有若干个连续片的磁盘块来存放增加的文件数据，同时还要将原来片中的内容写到新的片中来。




```
struct ext2_super_block {
        __u32   s_inodes_count;         /* 索引节点总数 */
        __u32   s_blocks_count;         /* 文件系统的块数 */
        __u32   s_r_blocks_count;       /* 保留给内核使用的块数 */
        __u32   s_free_blocks_count;    /* 空闲块计数器 */
        __u32   s_free_inodes_count;    /* 空闲索引节点计数器 */
        __u32   s_first_data_block;     /* 第一个数据块的块号 */
        __u32   s_log_block_size;       /* 块大小 */
                ...
        __u32   s_blocks_per_group;     /* # 每组的块数 */
                ...
        __u32   s_inodes_per_group;     /* # 每组的节点数 */
                ...
};
```



文件类型为目录时，数据块存放目录项数据结构：

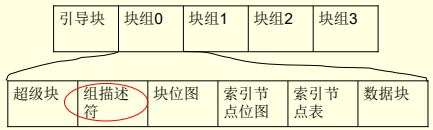
```
struct ext2_dir_entry {
        __u32 inode;        /* 索引节点号 */
        __u16 rec_len;      /* 目录项长度 */
        __u8  name_len;     /* 文件名长度 */
        __u8  name_type;    /* 文件名类型 */
        char  name[EXT2_NAME_LEN];    /* 文件名 */
};
```

删除目录项：inode置0，并增长前一个有效目录项的rec_len长度，这样在计算下一个有效目录项时，就可以跳过该已删除的目录项

```
00336: /*
00337:  * Structure of the super block
00338:  */
00339: struct ext2_super_block {
00340:         __u32   s_inodes_count;         /* Inodes count */
00341:         __u32   s_blocks_count;         /* Blocks count */
00342:         __u32   s_r_blocks_count;       /* Reserved blocks count */
00343:         __u32   s_free_blocks_count;    /* Free blocks count */
00344:         __u32   s_free_inodes_count;    /* Free inodes count */
00345:         __u32   s_first_data_block;     /* First Data Block */
00346:         __u32   s_log_block_size;       /* Block size */
00347:         __s32   s_log_frag_size;        /* Fragment size */
00348:         __u32   s_blocks_per_group;     /* # Blocks per group */
00349:         __u32   s_frags_per_group;      /* # Fragments per group */
00350:         __u32   s_inodes_per_group;     /* # Inodes per group */
00351:         __u32   s_mtime;                /* Mount time */
00352:         __u32   s_wtime;                /* Write time */
00353:         __u16   s_mnt_count;            /* Mount count */
00354:         __s16   s_max_mnt_count;        /* Maximal mount count */
00355:         __u16   s_magic;                /* Magic signature */
00356:         __u16   s_state;                /* File system state */
00357:         __u16   s_errors;               /* Behaviour when detecting errors */
00358:         __u16   s_minor_rev_level;      /* minor revision level */
00359:         __u32   s_lastcheck;            /* time of last check */
00360:         __u32   s_checkinterval;        /* max. time between checks */
00361:         __u32   s_creator_os;           /* OS */
00362:         __u32   s_rev_level;            /* Revision level */
00363:         __u16   s_def_resuid;           /* Default uid for reserved blocks */
00364:         __u16   s_def_resgid;           /* Default gid for reserved blocks */
```

EXT2文件系统的磁盘组织

- EXT2把它所占用的磁盘逻辑分区划分为块组，第一个磁盘块用于引导块，其余部分被分成块组。
- 所有的组大小相同并且顺序存放，所以由组的序号可以确定组在磁盘上的位置。
- 每一个块组依次包括超级块、组描述符表、块位示图、inode位示图、inode表以及数据块。
  - 块位示图：记录本组各个数据块的使用情况；
  - inode位示图：记录inode表中inode的使用情况。
  - inode表：保存了本组所有的inode
  - 数据块：用于存放数据
- inode用于描述文件。一个inode对应一个文件和子目录，有一个唯一的inode号，并记录了文件在外存的位置、存取权限、修改时间、类型等信息。




```
struct ext2_group_desc
{
        __u32   bg_block_bitmap;        /* 块位图的块号 */
        __u32   bg_inode_bitmap;        /* 索引节点位图的块号 */
        __u32   bg_inode_table;         /* 第一个索引节点表块的块号 */
        __u16   bg_free_blocks_count;   /* 组中空闲块的个数 */
        __u16   bg_free_inodes_count;   /* 组中索引点的个数 */
        __u16   bg_used_dirs_count;     /* 组中目录的个数 */
                …
};
```

注：各位图存放在一个单独的块中

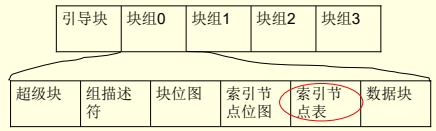
EXT2的系统引导块

- 系统引导块总是磁盘上的第一个数据块， 只有根文件系统才有引导程序放在这里， 其余文件系统都不使用引导块。

```
00365:      /*
00366:       * These fields are for EXT2_DYNAMIC_REV superblocks only.
00367:       *
00368:       * Note: the difference between the compatible feature set and
00369:       * the incompatible feature set is that if there is a bit set
00370:       * in the incompatible feature set that the kernel doesn't
00371:       * know about, it should refuse to mount the filesystem.
00372:       *
00373:       * e2fsck's requirements are more strict; if it doesn't know
00374:       * about a feature in either the compatible or incompatible
00375:       * feature set, it must abort and not try to meddle with
00376:       * things it doesn't understand...
00377:       */
00378:      __u32   s_first_ino;            /* First non-reserved inode */
00379:      __u16   s_inode_size;           /* size of inode structure */
00380:      __u16   s_block_group_nr;       /* block group # of this superblock */
00381:      __u32   s_feature_compat;       /* compatible feature set */
00382:      __u32   s_feature_incompat;     /* incompatible feature set */
00383:      __u32   s_feature_ro_compat;    /* readonly-compatible feature set */
00384:      __u8    s_uuid[16];             /* 128-bit uuid for volume */
00385:      char s_volume_name[16];         /* volume name */
00386:      char s_last_mounted[64];        /* directory where last mounted */
00387:      __u32   s_algorithm_usage_bitmap; /* For compression */
00388:      /*
00389:       * Performance hints.  Directory preallocation should only
00390:       * happen if the EXT2_COMPAT_PREALLOC flag is on.
00391:       */
00392:      __u8    s_prealloc_blocks;      /* Nr of blocks to try to preallocate*/
00393:      __u8    s_prealloc_dir_blocks;  /* Nr to preallocate for dirs */
00394:      __u16   s_padding1;
00395:      __u32   s_reserved[204];        /* Padding to the end of the block */
00396: } ? end ext2_super_block ? ;
```

EXT2块组

- 块组的构造
  - 每个块组重复保存着一些有关整个文件系统的关键信息，包含超级块、 块组描述结构、 块位示图、 索引节点位示图、 索引节点表和数据块。
- 使用块组结构可以提高文件系统的可靠性：
  - 由于文件系统的控制管理信息在每个块组中都有一份拷贝， 从而当文件系统意外出现崩溃时可以很容易地恢复它。



1、索引节点表由多个块组成， 每个块存放多个大小相同的索引节点
2、ext2_super_block. bg_inode_table指向第一个块的块号

```
struct ext2_inode {
        __u16   i_mode;         /* 描述的是什么及用户应具有的权限 */
                ...
        __u32   i_size;         /* 文件的有效长度 */
                ...
        __u32   i_blocks;  /* 已分配给文件的数据块数 */
                ...
        __u32   i_block[EXT2_N_BLOCKS];
                /* 指针数组，各元素指向分配给文件的数据块 */
                ...
};
```

i_size最高位没有使用，因此文件大小限制为2GB
块组和索引节点表的使用，可以快速得到索引节点的磁盘地址

EXT2的超级块

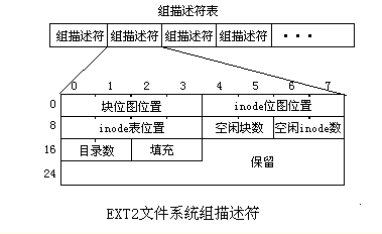
- 超级块是用来描述EXT2文件系统整体信息的数据结构。主要描述文件系统的目录和文件的静态分布情况，以及描述文件系统的各种组成
  - 主要包括inode数量、块数量、保留块数量、空闲块数量、空闲inode数量、第一个数据块位置、块长度、片长度、每个块组块数、每个块组片数、每个块组inode数，以及安装时间、最后一次写时间、安装信息、文件系统状态信息、…、
- 每个块组的第一个磁盘块用来保存超级块ext2_super_block

```
00027: /*
00028:  * second extended-fs super-block data in memory
00029:  */
00030: struct ext2_sb_info {
00031:         unsigned long s_frag_size;  /* Size of a fragment in bytes */
00032:         unsigned long s_frags_per_block;/* Number of fragments per block */
00033:         unsigned long s_inodes_per_block;/* Number of inodes per block */
00034:         unsigned long s_frags_per_group;/* Number of fragments in a group */
00035:         unsigned long s_blocks_per_group;/* Number of blocks in a group */
00036:         unsigned long s_inodes_per_group;/* Number of inodes in a group */
00037:         unsigned long s_itb_per_group; /* Number of inode table blocks per group */
00038:         unsigned long s_gdb_count;     /* Number of group descriptor blocks */
00039:         unsigned long s_desc_per_block;    /* Number of group descriptors per block */
00040:         unsigned long s_groups_count;  /* Number of groups in the fs */
00041:         struct buffer_head * s_sbh; /* Buffer containing the super block */
00042:         struct ext2_super_block * s_es;  /* Pointer to the super block in the buffer */
00043:         struct buffer_head ** s_group_desc;
00044:         unsigned short s_loaded_inode_bitmaps;
00045:         unsigned short s_loaded_block_bitmaps;
00046:         unsigned long s_inode_bitmap_number[EXT2_MAX_GROUP_LOADED];
00047:         struct buffer_head * s_inode_bitmap[EXT2_MAX_GROUP_LOADED];
00048:         unsigned long s_block_bitmap_number[EXT2_MAX_GROUP_LOADED];
00049:         struct buffer_head * s_block_bitmap[EXT2_MAX_GROUP_LOADED];
00050:         unsigned long  s_mount_opt;
00051:         uid_t s_resuid;
00052:         gid_t s_resgid;
00053:         unsigned short s_mount_state;
00054:         unsigned short s_pad;
00055:         int s_addr_per_block_bits;
00056:         int s_desc_per_block_bits;
00057:         int s_inode_size;
00058:         int s_first_ino;
00059: } ? end ext2_sb_info ? ;
```

EXT2的组描述符

- 每个块组都有一个组描述符，记录了该块组的块位图位置、inode位图位置、inode节点位置、空闲块数、inode数、目录数等内容(ext2_group_desc)。
- 所有的组描述符一个接一个存放，构成了组描述符表。
- 同超级块一样，组描述符表在每个块组中都有备份。这样，当文件系统崩溃时，可以用来恢复文件系统。

EXT2的组描述符



EXT2文件系统组描述符

EXT2组描述符

组描述符用来描述一个磁盘块组的相关信息

```
struct ext2_group_desc
{
  __u32 bg_block_bitmap;/*本组数据块位图所在块号*/
  __u32 bg_inode_bitmap;/* 本组inode位图所在块号 */
  __u32 bg_inode_table;     /* 本组inode表的起始块号 */
  __u16 bg_free_blocks_count;/* 组中空闲块的数目 */
  __u16 bg_free_inodes_count;/*组中空闲inode数目 */
  __u16 bg_used_dirs_count;/* 组中目录的数目 */
};
```

EXT2的Inode表和Inode位图

- EXT2的一个磁盘块组中的一个块组中所有文件的inode形成了inode表，存储在一组连续的磁盘块中。表项的序号就是inode号。
  - inode表存放在块组中所有数据块之前。
  - inode表在块组中要占用几个逻辑块由超级块中s_inodes_per_group给出。
- 这组磁盘块中的第一个磁盘块的块号存储在组描述符中的bg_inode_table数据项中。
- 根据磁盘块的大小，可以计算出每个磁盘块能容纳多少个索引结点
- 根据索引结点的总个数，可以计算出索引结点表所需要占用的磁盘块的个数。

EXT2的inode表和inode位图

- inode位图反映了inode表中各个表项的使用情况，它的一位(bit)表示inode表的一个表项，若某位为1表示对应的表项已占用，为0表示表项空闲。
- inode位图装入一个高速缓存中。

EXT2磁盘inode

```
struct ext2_inode {
    __u16  i_mode;
    __u16  i_uid;       /* 拥有者的用户ID */
    __u32  i_size;      /* 文件大小 */
    __u32  i_atime;     /* 最近一次访问时间 */
    __u32  i_ctime;     /* 创建时间 */
    __u32  i_mtime;     /* 最近一次修改时间 */
    __u16  i_gid;       /* 文件的组ID */
    __u32  i_blocks;/* 分配给该文件的磁盘块的数目 */
    __u32  i_block[EXT2_N_BLOCKS];/*指向磁盘块的指针
   */
    ......
};
```

inode在inode表中依次存放。

- 内存索引节点除了具有盘索引节点的主要信息外， 还增添了反映该文件动态状态的项目，例如：

(1) 共享访问计数(i_count)， 表示在某一时刻该文件被打开以后进行访问的次数。

(2) 队列结构。 通过几个list_head结构可以动态地链入到内存中的若干队列中， 可加快检索索引节点的访问速度。

磁盘索引节点包括以下一些主要内容：

(1) 文件模式， 描述文件属性和类型。

(2) 文件属主信息， 包括文件主标识号和同组用户标识号。

(3) 文件大小， 即文件的字节数。

(4) 时间戳， 包括索引节点建立的时间、 最近访问时间、 最后修改时间等。

(5) 文件链接计数。

(6) 数据块索引表。

关于索引节点中的i_block[]

- ext2的索引结点中使用了组合索引方式。

```
00240:      __u32   i_block[EXT2_N_BLOCKS];/* Pointers to blocks */
```

```
00174: /*
00175:  * Constants relative to the data blocks
00176:  */
00177: #define   EXT2_NDIR_BLOCKS        12
00178: #define   EXT2_IND_BLOCK          EXT2_NDIR_BLOCKS
00179: #define   EXT2_DIND_BLOCK         (EXT2_IND_BLOCK + 1)
00180: #define   EXT2_TIND_BLOCK         (EXT2_DIND_BLOCK + 1)
00181: #define   EXT2_N_BLOCKS           (EXT2_TIND_BLOCK + 1)
```

- 前12项用作直接索引
- 第13项用作一次间接索引
- 第14项用作二次间接索引
- 第15项用作三次间接索引

EXT2 内存inode

```
struct ext2_inode_info {
    __u32       i_data[15];  /* 数据块指针数组 */
    __u32       i_flags;     /* 文件标志（属性）*/
    __u32       i_faddr;     /* Fragment （片）地址 */
    __u8i_frag_no;   /* Fragment （片）号 */
    __u8i_frag_size; /* Fragment （片）大小 */
    __u16       i_osync;     /* 同步标志 */
    __u32       i_file_acl;  /* 文件访问控制链表 */
    __u32       i_dir_acl;   /* 目录访问控制链表 */
    __u32       i_dtime;     /* 文件删除时间 */
    __u32       i_version;   /* 文件版本 */
    __u32       i_block_group;/* inode所在块组号 */
    __u32       i_next_alloc_block;/* 下一个要分配的块 */
    __u32       i_next_alloc_goal; /* 下一个要分配的对象 * /
    __u32       i_prealloc_block;   / * 预留块首地址 */
    __u32       i_prealloc_count;  /* 预留计数 */
    int   i_new_inode:1;  /* 标志，是否为新分配的inode */
};
```

EXT2的块位图

- EXT2文件系统中数据块的使用状况由块位图来描述。
- 每个块组都有一个块位图，位于组描述符表之后，用来描述本块组中数据块的使用状况。
- 块位图的每一位(bit)表示一个数据块的使用情况，为1表示对应的数据块已占用，为0表示数据块空闲。
- 各位(bit)的顺序与块组中数据块的顺序一致。
- EXT2块位图装入一个高速缓存中，系统采用类似LRU的算法管理高速缓存。

EXT2中的目录项和文件类型

- 在EXT2中，目录是一种特殊的文件，这种文件的数据块中存放了该目录下的所有目录项

```
00483: /*
00484:  * Structure of a directory entry
00485:  */
00486: #define EXT2_NAME_LEN 255
00487:
00488: struct ext2_dir_entry {
00489:     __u32   inode;                  /* Inode number */
00490:     __u16   rec_len;        /* Directory entry length */
00491:     __u16   name_len;               /* Name length */
00492:     char name[EXT2_NAME_LEN];       /* File name */
00493: };
```

EXT2支持的文件类型

- EXT2在目录项中存放了文件的类型信息。文件类型可以是0～7中的任意一个整数。它们分别代表如下含义：
  - 0：文件类型未知；
  - 1：普通文件类型；
  - 2：目录；
  - 3：字符设备；
  - 4：块设备；
  - 5：有名管道FIFO；
  - 6：套接字；
  - 7：符号链接。

```
/*
 * Ext2 directory file types.  Only the low 3 bits are used.  The
 * other bits are reserved for now.
 */
enum {
    EXT2_FT_UNKNOWN,
    EXT2_FT_REG_FILE,
    EXT2_FT_DIR,
    EXT2_FT_CHRDEV,
    EXT2_FT_BLKDEV,
    EXT2_FT_FIFO,
    EXT2_FT_SOCK,
    EXT2_FT_SYMLINK,
    EXT2_FT_MAX
};
```

注意：

Type	Disk data structure	Memory data structure	Caching mode
Superblock	ext2_super_block	ext2_sb_info	Always cached
Group descriptor	ext2_group_desc	ext2_group_desc	Always cached
Block bitmap	Bit array in block	Bit array in buffer	Dynamic
inode bitmap	Bit array in block	Bit array in buffer	Dynamic
inode	ext2_inode	ext2_inode_info	Dynamic
Data block	Array of bytes	VFS buffer	Dynamic
Free inode	ext2_inode	None	Never
Free block	Array of bytes	None	Never

Ext2提供的各种对象方法

- 新版的目录项结构

```
00495: /*
00496:  * The new version of the directory entry.  Since EXT2 structures are
00497:  * stored in intel byte order, and the name_len field could never be
00498:  * bigger than 255 chars, it's safe to reclaim the extra byte for the
00499:  * file_type field.
00500:  */
00501: struct ext2_dir_entry_2 {
00502:     __u32   inode;                  /* Inode number */
00503:     __u16   rec_len;        /* Directory entry length */
00504:     __u8    name_len;               /* Name length */
00505:     __u8    file_type;
00506:     char name[EXT2_NAME_LEN];       /* File name */
00507: };
```

- 超级块对象方法

```
00150: static struct super_operations ext2_sops = {
00151:     read_inode:     ext2_read_inode,
00152:     write_inode:    ext2_write_inode,
00153:     put_inode:      ext2_put_inode,
00154:     delete_inode:   ext2_delete_inode,
00155:     put_super:      ext2_put_super,
00156:     write_super:    ext2_write_super,
00157:     statfs:         ext2_statfs,
00158:     remount_fs:     ext2_remount,
00159: };
```

■ 索引节点对象方法

```
00052: struct inode_operations ext2_file_inode_operations = {
00053:         truncate:       ext2_truncate,
00054: };
```

```
00338: struct inode_operations ext2_dir_inode_operations = {
00339:         create:         ext2_create,
00340:         lookup:         ext2_lookup,
00341:         link:       ext2_link,
00342:         unlink:         ext2_unlink,
00343:         symlink: ext2_symlink,
00344:         mkdir:          ext2_mkdir,
00345:         rmdir:          ext2_rmdir,
00346:         mknod:          ext2_mknod,
00347:         rename:         ext2_rename,
00348: };
```

```
00035: struct inode_operations ext2_fast_symlink_inode_operations = {
00036:         readlink:  ext2_readlink,
00037:         follow_link:  ext2_follow_link,
00038: };
```

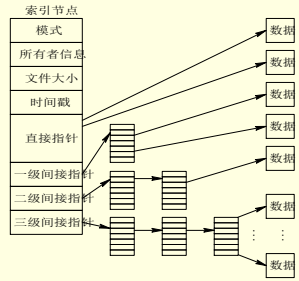
■ 文件对象方法

```
00037: /*
00038:  * We have mostly NULL's here: the current defaults are ok for
00039:  * the ext2 filesystem.
00040:  */
00041: struct file_operations ext2_file_operations = {
00042:         llseek:         generic_file_llseek,
00043:         read:           generic_file_read,
00044:         write:          generic_file_write,
00045:         ioctl:          ext2_ioctl,
00046:         mmap:           generic_file_mmap,
00047:         open:           generic_file_open,
00048:         release:  ext2_release_file,
00049:         fsync:          ext2_sync_file,
00050: };
```

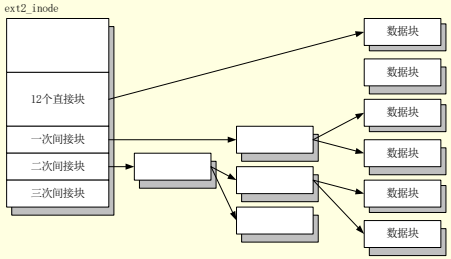
```
00591: struct file_operations ext2_dir_operations = {
00592:         read:           generic_read_dir,
00593:         readdir:   ext2_readdir,
00594:         ioctl:          ext2_ioctl,
00595:         fsync:          ext2_sync_file,
00596: };
```

文件数据在磁盘上的组织

- 数据在磁盘上并不一定连续，需要保存各个磁盘块号。
- EXT2文件系统采用了多重索引方式。
- i_data[]的前12项可看成直接指针，直接存放文件数据所在的磁盘块号。 第13项是个一级指针，第14，15项分别是二级指针和三级指针。



多重索引结构



Linux数据块分配策略

EXT2采用两个策略减少文件碎片

- 原地先查找策略：为文件分配数据块时，尽量在文件原有数据块附近查找。先试探紧跟文件末尾的数据块，然后试探位于同一个块组相邻的数据块，接着在同一个块组中寻找其他空闲数据块；实在不得已才搜索其他块组，且首先考虑8个一簇的连续的块。
- 预分配策略：就是从预分配的数据块取一块来用，紧跟该块后的若干个数据块空闲的话，也被保留，保证尽可能多的数据块被集中成一簇。

Ext2的内存数据结构

- 为提高效率，尽量减少访外次数，在安装Ext2分区时，内存中存放大部分磁盘数据结构，并使用缓存技术保持磁盘数据结构更新
- 缓存模式共有4种
  - Always cached：一直缓存
  - Fixed limit：固定数量的数据结构保存在缓存中
  - Dynamic：只要索引节点或块使用，相关数据就保存在缓存中
  - Never：任何高速缓存中都不保存

Ext2的内存数据结构（续）

类型	磁盘数据结构	内存数据结构	缓存模式
超级块	ext2_super_block	ext2_sb_info	Always cached
组描述符	ext2_group_desc	ext2_group_desc	Always cached
块位图	块中的位数组	缓冲区中的位数组	Fixed limit
索引节点位图	块中的位数组	缓冲区中的位数组	Fixed limit
索引节点	Ext2_inode	ext2_inode_info	Dynamic
数据块	未指定	缓冲区页	Dynamic
空闲索引节点	ext2_inode	无	Never
空闲块	未指定	无	Never

ext2方法

- VFS很多方法在Ext2中有对应的具体实现
- 比如，VFS中文件对象的操作与Ext2的文件操作映射如下：

VFS的文件操作	Ext2的方法
llseek	ext2_file_llseek()
read	generic_file_read()
write	ext2_file_write()
ioctl	ext2_ioctl()
mmap	generic_file_mmap()
open	ext2_file_open()
release	ext2_release_file()
fsync	ext2_sync_file()

VFS操作	Ext2中对应的具体的操作表
超级块操作	ext2_sops指针数组
普通文件的索引节点操作	Ext2_file_inode_operations
目录文件的索引节点操作	Ext2_dir_inode_operations
链接文件的索引节点操作	Ext2_symlink_operations/ Page_symlink_operations
字符设备文件	Chrdev_inode_operations
块设备文件	Blkdev_inode_operations
命名管道	Fifo_inode_operations
文件操作	ext2_file_operations

EXT2文件系统的不足

- Linux的EXT2文件系统会先写入文件的内容，空闲时再写入文件的元数据。
  - 元数据：文件的大小、存储位置、权限等等。
  - 突然断电，文件系统就会处于不一致的状态。
- 日志文件的工作原理
  - 在分区中保存一个日志记录文件，文件系统写操作首先是对该日志记录文件进行操作，若写操作中断时，下次系统启动时就会读日志记录文件的内容而恢复没有写完的操作，所用时间很短。

EXT3文件系统

- 日志文件系统(Journaling file system)
- 利用数据库的日志技术（log, checkpoint）

作业(1)

- Linux文件系统的特点是什么？
- Linux文件系统的类型有几中，分别是什么？
- 什么是VFS？它有什么作用？它采用了什么思想？
- 为什么说U域是实现多种文件系统的关键？为什么要将U域复制到内存，如何解决它与磁盘内容不一致的问题？
- Linux的VFS的相关数据结构有哪些？作用分别是什么？
- 设一个linux用户的当前工作目录是usr/mnt/dao1,如果系统管理员在usr/mnt/目录上安装了一个新的文件系统，这对用户会有什么影响？该用户是否能够继续看到/dao1目录下的文件？
- 为什么在超级块中设备dirty域？
- Inode高速缓存与目录高速缓存的作用什么？

作业(2)

- 假定进程以读方式打开一个文件后，再执行fork，父进程和子进程将都可以读这个文件。这两个进程的读操作和写操作各有何关系？
- EXT2文件系统是如何减少磁盘碎片的？
- Linux文件系统为什么设计了一个磁盘i节点又设计了一个内存i节点，为什么内存i节点的内容和磁盘i节点内容不完全相同？
- 为什么用户在第一次访问任何文件之前，都必须先调用open()系统调用来打开指定的文件，然后才能对该文件执行读、写以及修改操作,在对文件操作结束必须关闭文件。

第8章 Linux设备管理与模块机制

- Linux设备管理概述
- Linux设备分类及数据结构
- Linux设备转换表
- Linux设备注册与注销
- Linux设备操作
- Linux模块机制

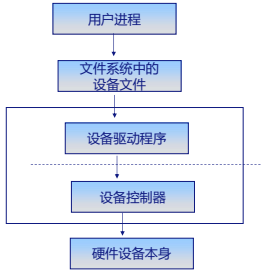
8.1 Linux设备管理概述

- Linux利用一种通用的方法来对所有的输入／输出设备进行控制，从而完成数据的输入和输出操作，即设计了一个统一而简单的输入／输出系统调用接口。
- 输入／输出子系统分为上下两部分：
  - 一部分是下层的与设备有关的部分，常称为设备驱动程序，它直接与相应设备打交道，并且向上层提供一组访问接口；
  - 另一部分是上层的与设备无关的I/O软件，它根据上层用户的输入／输出请求，向下通过特定的设备驱动程序接口和设备进行通信。

8.1 Linux设备管理概述

- 在 Linux 系统中，用户通过文件系统与设备接口。利用标准的系统调用可在设备上进行打开、关闭、读取或写入操作。
- 当用户进程发出I/O请求时，系统将请求处理的权限放在文件系统，文件系统通过驱动程序提供的接口将任务下放到驱动程序。驱动程序根据需要，对设备控制器进行操作，设备控制器再去控制设备本身进行I/O操作。
- 驱动程序向文件系统提供的接口屏蔽了设备的物理特性。

8.1 Linux设备管理概述



进程请求设备服务的流程

- 在内核中设备文件和普通文件的索引节点有区别，可以确认出是硬件设备还是普通文件，确定要访问相应的设备驱动程序还是文件。
- 设备文件在磁盘上只有一个索引节点，没有任何用于存放数据的块组。

8.1 Linux设备管理概述

设备文件

- 设备类型
  [root@ edu]# more /proc/devices
- 主设备号与次设备号
  [root@ edu/proc]#ls -l /dev/had*
- 设备文件
  - 在与设备驱动程序通信时，内核常常使用设备类型、主设备号和次设备号来标识一个具体的设备。
  - Linux将设备文件放在目录/dev或其子目录下

8.1 Linux设备管理概述

设备文件

- Linux使用设备文件来统一对设备的访问接口，将设备文件放在/dev/目录下
- 设备的命名一般为设备文件名+数字或者字母表示的子类，例如/dev/hda1，/dev/hda2等，/dev/hda 代表系统中的第一个IDE 硬盘，/dev/hdb 代表系统中的第二个 IDE 硬盘； hda1代表第1个分区， hda2代表第2个分区
- 用于用户访问设备进行输入和输出操作。应用程序可以打开、关闭和读写这些设备文件，完成对设备的操作，就像操作普通的数据文件一样。

8.1 Linux设备管理概述

- 常见的设备文件
  - /dev/hd* IDE接口的硬盘(IDE接口的设备)
  - /dev/sd* SCSI/USB设备
  - /dev/cua* 串口设备
  - /dev/lp* 并口设备
  - /dev/tty* 终端设备
  - /dev/consol 控制台设备
  - /dev/eth* 以太网设备
  - /dev/cdrom IDE光驱
  - /dev/fd* 软驱
  - /dev/audio 音频设备
  - /dev/scd SCSI的光驱
  - /dev/ppp PPP设备
  - /dev/isdn* ISDN设备

8.2 Linux设备分类及数据结构

- 设备文件通常包含在/dev目录中
- 一些设备文件的例子

Name	Type	Major	Minor	Description
/dev/fd0	block	2	0	Floppy disk
/dev/hda	block	3	0	First IDE disk
/dev/hda2	block	3	2	Second primary partition of first IDE disk
/dev/hdb	block	3	64	Second IDE disk
/dev/hdb3	block	3	67	Third primary partition of second IDE disk
/dev/ttyp0	char	3	0	Terminal
/dev/console	char	5	1	Console
/dev/lp1	char	6	1	Parallel printer
/dev/ttyS0	char	4	64	First serial port
/dev/rtc	char	10	135	Real time clock
/dev/null	char	1	3	Null device (black hole)

8.2 Linux设备分类及数据结构

- 为了管理设备，系统为设备编了号，每个设备号又分为主设备号和次设备号。
  - 主设备号用来区分不同种类的设备
  - 次设备号用来区分同一类型的多个设备
  - 主设备号相同的设备使用相同的驱动程序，次设备号用于区分具体设备的实例
  - 对于常用设备，Linux有约定俗成的编号，如硬盘的主设备号是3。
  - 动态获取主设备号

8.2 Linux设备分类及数据结构

- 字符设备(c)
  - 所有能够象字节流一样访问的设备都通过字符设备来实现，通常在/dev/目录下面
  - 无需缓冲直接读写设备，通常只允许按顺序访问。如打印机、键盘，终端等。
  - 一般要包含open， read， write， close等系统调用的实现
- 块设备(b)
  - 通常是指诸如磁盘、内存、Flash等可以容纳文件系统的存储设备。
  - 将数据按可寻址的块为单位进行处理，可以随机访问，利用缓冲技术
  - 块设备也是通过文件系统来访问

8.2 Linux设备分类及数据结构

- 网络接口设备
  - 通常它指的是硬件设备，但有时也可能是一个软件设备(如回环接口loopback)，它们由内核中网络子系统驱动，负责发送和接收数据包。
  - 它们的数据传送往往不是面向流的，因此很难将它们映射到一个文件系统的节点上。
- 查看系统中的设备：/proc/devices。
- 查看设备及其类型“ls -l /dev”

8.2 Linux设备分类及数据结构

- 每个设备都对应文件系统中的一个VFS inode，都有一个设备文件名。
- 在设备文件对应的VFS inode中，i_mode域的值标明了它的类型：
  - S_IFBLK：表示它是一个块设备；
  - S_IFCHR：表示它是一个字符设备；
- 同时，设备文件对应的VFS inode中没有块列表，但却有一个名为i_rdev的域，其中存储着它所标识设备的主、次设备号。内核可以根据该域将设备文件名转化为设备的主、次设备号。

8.2 Linux设备分类及数据结构

- 应用程序通常可以通过系统调用open()打开设备文件， 建立起与目标设备的连接。
- 对设备的使用类似于对文件的存取。
- 设备驱动程序都是系统内核的一部分， 它们必须为系统内核或者它们的子系统提供一个标准的接口。 设备驱动程序也使用一些标准的内核服务， 如内存分配等

8.2 Linux设备分类及数据结构

- 注意：字符设备与块设备具有独立的编号，
  - 例如，块设备(3,0)不同于字符设备(3,0)
- 设备文件通常可以表示
  - 一个硬件设备，例如磁盘/dev/hda
  - 或硬件设备的某一物理或逻辑分区，例如磁盘分区/dev/hda2
  - 或一个虚拟的逻辑设备（不会与任何硬件设备相关联），例如/dev/null代表一个“黑洞”
- 对内核而言，一个设备文件的名字是无关紧要的，关键在于设备文件的类型及其主次设备号

8.2 Linux设备分类及数据结构

- 如果一个系统含有两个具有相同主设备号和次设备号的设备文件，而且都是字符设备。当有两个进程想要同时打开这个字符设备时，会发生什么情况？关闭这个设备时又会发生什么情况？
- 当有两个进程想要同时打开这个字符设备时，它将在各自的文件表中增加一次引用计数值。 关闭这个设备时可能会发生导致对这个设备是一个无法辨认的状态。从而使这个设备变成一个无用的设备。

8.2 Linux设备分类及数据结构

- 使用主、次设备号标识设备存在局限性
  - 在/dev中的大多数设备是不存在的
  - 8位长的主次设备号不够用
  - 不容易记忆
- 上述原因以及其他的一些因素综合起来，促使了devfs设备文件的产生
- Linux 2.4以后引入了设备文件系统（devfs）的概念，所有的设备文件作为一个可以挂装的文件系统，这样就可以被文件系统统一管理，从而设备文件就可以挂装到任何需要的地方。
- 一般将主设备建立一个目录，再将具体的子设备文件建立在此目录下。例如，/dev/mtdblock0

8.2 Linux设备分类及数据结构

- 有了Devfs设备文件之后，文件系统允许设备驱动程序通过名字而不是主次设备号注册设备，例如
  - 所有的磁盘可以放在/dev/disks目录下
  - /dev/hda➔/dev/disks/disc0
  - /dev/hdb➔/dev/disks/disc1
- 使用devfs文件系统的I/O驱动程序通过调用devfs_register()注册设备
- 一个注册了的设备文件，自动出现在devfs的虚拟目录下

8.2 Linux设备分类及数据结构

设备文件的VFS处理

- 进程访问普通文件时，通过文件系统访问磁盘分区中的数据块
- 当进程访问设备文件时，却可以驱动硬件设备
- 途径：VFS在设备文件打开时使用与设备相关的函数调用替换其缺省的文件操作
  - 这些设备相关函数调用对硬件设备进行操作
- 驱动程序使用的2个重要结构
  - struct file
  - struct file_operations

设备文件： struct file数据结构

```
struct  file{
struct dentry            *f_dentry; //文件对应的目录项结构
struct file_operations *f_op;  //文件操作的结构指针
unsigned  short          f_flags;//文件标志阻塞/非阻塞操作时检查
undigned short           f_count;
mode_t                   f_mode ; //标识文件的读写权限
Loff_t                   f_pos;当前读写位置
unsigned long            f_reada,f_ramax,f_reddb,f_rawin;
struct file              *f_next,f_prev;
struct fown_struct       f_owner;
int                      f_error;
unsigned long            f_version;
void                     *private_data; //指向已经分配的数据
struct  inode            *f_inode;
```

8.2 Linux设备分类及数据结构

设备文件：i节点

- 每一个设备文件使用唯一的i节点来标识；
- 在i节点中包含设备的主设备号和次设备号，并且和一组文件操作函数的指针相联系。
- 设备文件i节点不指向文件系统中的任何实际的物理块，不占用数据空间，通过这个i节点可以访问相应的设备驱动程序，对设备文件的操作就是直接对设备本身进行相应的操作。

设备文件：操作

- 文件描述符fd是系统调用与设备驱动程序的接口。在操作时，内核通过fd找到代表该文件的struct file结构，得到打开文件的i节点。
- 在i节点中检查文件类型，如果是设备（块或字符）文件，则从索引节点中提取主设备号和次设备号，再通过主设备号定位块或者字符设备转换表中相应的设备驱动程序。
- 根据用户发出的系统调用的功能，进一步确定应该调用哪一个操作函数，并确定传递的参数。启动该函数。

8.3 Linux设备转换表

- 设备转换表是一个数据结构，由它定义了每个设备必须支持的操作入口点，由fops,bdops描述。每个驱动程序提供了这些功能的特定实现，都在其对应数组中占据一个项。
  - 块设备转换表（blk_dev）
  - 字符设备转换表(chrdevs)
- 根据设备的类型和主设备号可以在设备表中找到相应的驱动函数。
  - 一旦内核想在某台设备上完成某个动作，它必须首先在转换表中定位该设备的驱动程序，再调用相应的驱动程序函数。
- 设备文件的的主设备号必须与设备驱动程序在登记时申请的主设备号一致，否则用户进程将无法访问到驱动程序。

8.3 Linux设备转换表

用于注册字符设备的数据结构

```
struct device_struct{
const char * name;
struct file_operations *fops;
}
static struct device_struct chrdevs[MAX_CHRDEV];
```

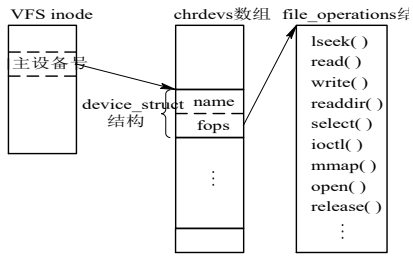
device_struct结构由两项构成：

- name：指向已登记的设备名称的指针
- fops：　指向设备驱动程序的入口函数：完成指定的设备操作。

8.3 Linux设备转换表

- 字符设备初始化时，它的设备驱动将被添加到chrdevs中，从而将其注册到Linux内核中。设备的主设备标识符是固定的。
- 使用主设备号用来对chrdevs数组进行索引。
- 当访问某台设备时，它首先必须在字符转换表中找到该设备的驱动程序，然后控制权转交给设备驱动程序，由设备驱动程序再调用相应的驱动程序函数执行。

8.3 Linux设备转换表



字符设备转换表

- include/linux/fs.h

```
struct file {
  ……
  struct file_operations *f_op;
};
```

字符设备转换表

```
struct file_operations {
Struct module *owner;
    loff_t  (*llseek)(struct file *,loff_t,int);
    ssize_t  (*read)(struct file *,char *,size_t,loff_t *);
    ssize_t  (*write)(struct file *,const char *,size_t,loff_t *);
    int(*ioctl)  (struct inode *,struct file *,unsigned
  int,unsigned long);
    int(*mmap)  (struct file *,struct vm_area_struct *);
    int(*open)  (struct inode *,struct file *);
    int(*release)  (struct inode *,struct file *);
    int(*fsync)  (struct file *,struct dentry *,int datasync);
    int(*fasync)  (int,struct file *,int);    ……
};
```

(1) llseek(file, offset, whence)：修改文件的读写指针。

(2) read(file, buf, count, offset)：从设备文件的offset 处开始读出count个字节，然后增加offset的值。

(3) write(file, buf, count, offset)：从设备文件的offset处写入count个字节，然后增加*offset的值。

(4) ioctl(inode, file, cmd, arg)：向一个硬件设备发命令，对设备进行控制。

(5) mmap(file, vma)：将设备空间映射到进程地址空间。

(6) open(inode, file)：打开并初始化设备。

(7) release(inode, file)：关闭设备并释放资源。

(8) fsync(file, dentry)：实现内存与设备之间的同步通信。

(9) fasync(file, on)：实现内存与设备之间的异步通信。

字符设备的操作

- 字符设备的操作：打开文件操作
  - 当打开字符设备文件时，使用设备的主设备号来索引chrdevs数组，得到文件操作函数指针。
  - 建立起描述此字符设备文件的file结构，使其文件操作指针指向此设备驱动中的文件操作指针集合。
  - 这样，所有应用对它进行的文件操作都被映射到此字符设备的文件操作集合上。

8.4 Linux设备的注册与注销

字符设备的注册与注销函数：

int register_chrdev(unsigned int major, const char * name, struct file_operations *fops)： chrdevs表最初为空，注册函数向表中插入一个新项

int unregister_chrdev(unsigned int major, const char * name);

注：major即设备的主设备号，注册后就是访问数组chrdevs的索引（下标）。

例：register_chrdev(6,"lp",&lp_fops):将并口打印机的相应驱动程序插入到chrdevs表中

8.4 Linux设备的注册与注销

块设备转换表

```
fs/block_dev.c
static struct {
        const char *name;
        struct block_device_operations *bdops;
} blk_devs[MAX_BLKDEV];
```

8.4 Linux设备的注册与注销

块设备转换表

- blk_devs结构数组描述了一系列在系统中登记的块设备。
- 内容包括指向已登记的设备名的指针和指向block_device_operations结构的指针。
- 在block_device_operations结构中包含指向有关操作的函数指针。它们由设备驱动程序定义，并分别完成指定的设备操作
- 数组blk_devs也使用主设备号作为索引。

8.4 Linux设备的注册与注销

块设备转换表

```
struct block_device_operations {
        int (*open) (struct inode *, struct file *);
        int (*release) (struct inode *, struct file *);
        int (*ioctl) (struct inode *, struct file *, unsigned,
unsigned long);
        int (*check_media_change) (kdev_t);//检查介质
是否已经变化
        int (*revalidate) (kdev_t);//检查块设备是否持有
有效数据
};
```

8.4 Linux设备的注册与注销

块设备的注册与管理

注册与注销函数

register_blkdev(unsigned int major,const char *name, struct block_device_operations *bdops)

int unregister_blkdev(unsigned int major, const char * name)

8.5 Linux设备请求

块设备请求

- 虽然块设备驱动程序可以一次传送一个单独的数据块，但是内核并不会为磁盘上的每个被访问的数据块都单独执行一次I/O操作，而是试图将几个块合并在一起，并作为一个整体来处理，这样就减少了磁头平均移动时间。
- 延迟：块设备发出I/O请求时，将请求放在相应的请求队列中，稍后执行。
- 每个块设备都有一个请求队列，一提高磁盘性能的方式对请求进行排序。
- 块设备请求由request结构描述，位于include/linux/blkdev.h中。
- 请求队列：是一个请求描述符的链表。

8.5 Linux设备请求

块设备的操作

- 当有进程发出一个读写块设备的请求时，内核会首先调用block_read()和block_write()对数据缓冲区进行读写。
- 当数据缓冲区不能满足用户进程的一个读写请求或者一个文件需要交换到物理磁盘上时，必须调用块设备的驱动程序来进行实际的数据传输。
- 为此，在Linux中，为完成真正的数据传输请求，又为每一个块设备设计了一个接口blk_dev_struct
- blk_dev_struct结构包含请求过程的地址以及指向请求request数据结构链表的指针，每个代表一个从buffer cache中来让设备进行数据读写的请求。

8.5 Linux设备请求

- 每个块设备都有一个blk_dev_struct 结构（块设备驱动程序描述符），所有的blk_dev_struct结构由向量blk_dev集中管理，可以使用主设备号进行索引。

```
struct blk_dev_struct {
  request_queue_t        request_queue;
  queue_proc             *queue;
  void                   *data;
};
struct blk_dev_struct  blk_dev[MAX_BLKDEV];
```

blk_dev[]记录各个设备的请求队列

request_queue:请求队列

*queue：函数指针，当这个指针非0时，就调用这个函数来找到具体设备的请求队列。

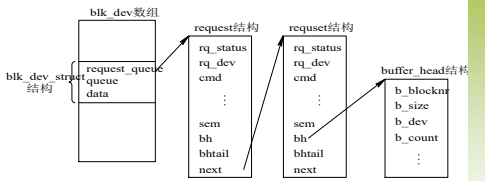
*data：辅助queue函数找到特定设备的请求队列

8.5 Linux设备请求

- 在blk_dev_struct中有三个域，驱动程序主要是设置第一个域request_queue中的request_fn参数，该参数是指向相应块设备的I/O操作请求函数的指针。
- 当block_read()和block_write()不能满足I/O操作请求时，内核便调用该设备驱动程序的请求函数request。它可以通过blk_dev[MAJOR(dev)].request_queue.request_fn获得。每个具体设备的请求函数都不同，它们在内核在初始化时被赋值。

```
struct request_queue { //include/linux/blkdev.h
……
request_fn_proc    *request_fn;
……
}
```

8.5 Linux设备请求



块设备驱动程序数据结构示意图

8.6 Linux模块机制

- Linux系统使用可动态加载和卸载的内核模块(Loadable Kernel Modules，LKMs)，可方便地在内核中添加新的组件或卸载不再需要的内核组件,提高了linux内核的可扩展性，可以让内核保持很小的尺寸并非常灵活。
- 模块机制可以无需重构内核并频繁重新启动来尝试运行新内核代码。用户可以根据自己系统的需要构筑自己的私有内核。

8.6 Linux模块机制

- 模块的全称是“动态可加载内核模块”（Loadable Kernel Module，LKM）
- 模块在内核空间运行
- 模块实际上是一种目标对象文件
- 没有链接，不能独立运行，但是其代码可以在运行时链接到系统中作为内核的一部分运行或从内核中取下，从而可以动态扩充内核的功能
- 这种目标代码通常由一组函数和数据结构组成

8.6 Linux模块机制

- 优点
  - 使得内核更加紧凑和灵活
  - 修改内核时，不必全部重新编译整个内核。系统如果需要使用新模块，只要编译相应的模块，然后使用insmod将模块装载即可
  - 模块的目标代码一旦被链接到内核，它的作用域和静态链接的内核目标代码完全等价

8.6 Linux模块机制

- 缺点
  - 由于内核所占用的内存是不会被换出的，所以链接进内核的模块会给整个系统带来一定的性能和内存利用方面的损失。
  - 装入内核的模块就成为内核的一部分，可以修改内核中的其他部分，因此，模块的使用不当会导致系统崩溃。
  - 为了让内核模块能访问所有内核资源，内核必须维护符号表，并在装入和卸载模块时修改符号表；
  - 模块会要求利用其它模块的功能，所以，内核要维护模块之间的依赖性。

8.6 Linux模块机制

Linux内核模块与应用程序的区别

	C语言程序	Linux内核模块
运行	用户空间	内核空间
入口	main()	module_init()指定;
出口	无	module_exit()指定;
编译	gcc -c	Makefile
连接	ld	insmod
运行	直接运行	insmod
调试	gdb	kdbug, kdb, kgdb等

8.6 Linux模块机制

模块相关命令

- insmod <module.ko> [module parameters]
  - Load the module
  - 注意，只有超级用户才能使用这个命令
- Rmmod
  - 卸载模块
- lsmod
  - 列出所有已安装进内核的模块
  - 这个命令和cat /proc/modules等价
- modprobe [-r] <module name>
  - – 加载指定的模块及该模块所依赖的模块

8.6 Linux模块机制

模块依赖

- 一个模块A引用另一个模块B所导出的符号，我们就说模块B被模块A引用。
- 如果要装载模块A，必须先要装载模块B。否则，模块B所导出的那些符号的引用就不可能被链接到模块A中。这种模块间的相互关系就叫做模块依赖。

8.6 Linux模块机制

简单的内核模块例子

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
static int __init hello_init(void)
{
        printk(KERN_INFO "Hello world\n");
        return 0;
}
static void __exit hello_exit(void)
{
        printk(KERN_INFO "Goodbye world\n");
}
module_init(hello_init);
module_exit(hello_exit);
```

8.6 Linux模块机制

- static int __init hello_init(void)
- static void __exit hello_exit(void)
  - Static声明，因为这种函数在特定文件之外没有其它意义
  - __init标记,该函数只在初始化期间使用。模块装载后，将该函数占用的内存空间释放
  - __exit标记 该代码仅用于模块卸载。
- Init/exit
  - 宏：module_init/module_exit
  - 声明模块初始化及清除函数所在的位置
  - 装载和卸载模块时，内核可以自动找到相应的函数
- module_init(hello_init);
- module_exit(hello_exit);

8.6 Linux模块机制

编译内核模块

- Makefile文件

```
obj-m := hello.o
all:
   make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) modules
clean:
   make -C /lib/modules/$(shell uname -r)/build M=$(shell pwd) clean
```

- Module includes more files

```
obj-m:=hello.o
hello-objs := a.o b.o
```

8.6 Linux模块机制

装载和卸载模块

- 相关命令
  - lsmod
  - insmod hello.ko
  - rmmod hello.ko

8.6 Linux模块机制

内核模块的引入带来的问题

- 有可能同时带来与内核模块相关的性能与内存损失。可加载模块的代码一般较长，且额外的数据结构可能会占据一些内存，对系统性能和内存利用有负面影响。
- 一旦Linux模块被加载，则和普通内核代码一样成为内核的一部分，具有与其他内核代码相同的权限与职责。因此，Linux内核模块也可以象所有内核代码和设备驱动一样使内核崩溃。
- 为了内核模块访问所有内核资源，内核必须维护符号表，并在装入和卸载模块时修改这些符号表。

8.6 Linux模块机制

内核模块的引入带来的问题

- 有些模块要求利用其他模块的功能，因此，内核要维护模块之间的依赖性。
- 内核必须能够在卸载模块时通知模块，并且要释放分配给模块的内存和中断等资源；
- 内核版本和模块版本的不兼容，也可能导致系统崩溃，因此，严格的版本检查是必需的。

作业

- 什么是设备驱动程序？
- 如何注册字符设备驱动程序？
- 为什么Linux使用设备转换表引导设备操作？
- 字符设备与块设备有何区别？
- 为什么一个I/O设备有两个设备号？一个是主设备号，另一个是次设备号。
- 将设备和普通文件统一命名的好处是什么？
- 简述Linux的模块机制及其优缺点。

考试安排

- 考试
  - 形式：开卷。提醒：不允许带电脑、ipad、手机等电子设备。
  - 题型：单选、填空、问答。
- 考试时间
  4月13日 周三 1,2节 随堂考试（第8周周三）

复习的思路

- 概念
- 原理
- 算法
- 实现过程
- 重要数据结构、重要数据域的作用
- 重要数据结构、重要数据域之间的关系
- 特别是从task_struct出发，分析相关的重要数据域
- 相关计算

第1章 概述

- Linux系统特点。
- Linux系统组成。
- 什么是POSIX标准？为什么现代操作系统的设计必须遵循POSIX标准？
- 什么是GNU？Linux与GNU有什么关系。
- Linux内核由哪几个子系统组成？各子系统的主要功能是什么？能够写出各子系统之间的关联。
- 能够认识Linux内核版本。

第2章 Linux进程管理

- 结合后续章节的知识，理解进程控制块task_struct的作用、重要字段的含义。
- Linux进程的状态类型？变化情况？僵死状态的进程如何处理？
- Linux 2.4进程的系统堆栈，Linux2.6进程的系统堆栈
- Linux进程间的家族关系？
- 进程组织所涉及到的数据结构（进程链表、运行队列、等待队列，pidhash表）的作用
- Linux的进程控制：相关函数的作用、执行过程
  - fork():sys_fork()/sys_clone()/vfork()的区别
  - exec()
  - Wait()
  - Exit()
- 什么是写时复制技术？
- 结合do_fork()的源代码理解父进程创建子进程的过程

第3章 Linux进程调度

- Linux的进程调度方式有哪些。
- 静态优先级、动态优先级、counter的作用。
- 普通进程和实时进程调度。
- Linux的三种调度策略。
- do_fork()中的父子进程的优先级,时间片的设置有什么特点。
- 调度时机有哪些，都是如何实现的？
- 调度时优先级的重新计算方法，计算时间。
- Linux2.6进程调度特性分析，与Linux2.4的区别。

第4章 系统调用与中断

- 什么是系统调用？为什么要引入系统调用
- Int 0x80指令的作用？
- 结合system_call理解系统调用的整个执行过程。
- 系统调用表的组织结构？系统调用号的作用。
- 系统调用的参数传递方法？
- 考试不要求
  - 描述一个添加系统调用的步骤。

第5章 Linux中断处理

- 什么是快中断？什么是慢中断？为什么要进行区分？各自的特点是什么？
- 为什么要引入下半部？下半部的组织？
- 发生中断时，上、下部分如何处理？
- 定时器中断的定义及其相关数据结构
- 动态定时器是如何迁移的？

第6章 存储管理

- 实模式、保护模式，如何进行模式切换（CR0的PE位）
- 如何启用分页机制？ （CR0的PG位）
- 段描述符的作用、组成
- 段选择符的作用、组成
- 保护模式下的地址转换过程？
- 内核空间和用户空间
- 为什么采用三级分页模式？
- 掌握虚拟地址空间和物理地址空间的管理方法
- Linux用户地址空间的分布
  - 为什么要划分成一个个区VMA？VMA的查找、删除？
  - mm_struct， vm_area_struct的作用？
  - 描述task_struct,vm_area_struct、mm_struct、页目录表、物理内存之间的关系（结合图）

第6章 存储管理（续）

- Linux如何实现请求调页
- 物理地址空间的页面的分配
  - 伙伴算法
  - free_area[],bitmap的作用
  - 结合free_area[],bitmap理解基于伙伴算法的物理页面的分配和回收，并能够会应用。
- 缺页异常处理过程
  - Do_page_fault()、handle_mm_fault(),handle_pte_fault()的作用
- Slab分配器的作用
- 掌握地址的变换。

第7章 文件管理

- 什么是VFS？VFS的作用？VFS的框架？VFS如何实现与实际文件系统的映射？
- 以write()系统调用为例，说明VFS是如何与具体文件系统,如ext2相结合的？
- 什么是I节点（内存I节点、磁盘I节点）？VFS的索引节点与具体的文件系统的索引节点有什么联系？
- 目录项结构与索引节点有何区别？为什么不把两者合二为一？
- 掌握相关的数据结构：作用、重要域的含义
  - Super_block 、 Inode 、 Files_struct 、File、Dentry、Vfsmount、File_system_type
- Super_block结构中设置u域的意图是什么？

第7章 文件管理

- 目录项的组织
- 掌握linux的目录和文件的访问过程（结合实例）
- 文件系统的注册和安装有何不同？何时进行？
- 文件系统的注册、安装、卸载的过程?当文件系统正在使用时能够卸载？
- 如何描述已安装的、已注册的文件系统？
- 进程与文件系统的联系
  - 数据结构：Files_struct/fs_struct/file：作用
  - 系统打开文件表、用户打开文件表，及它们间的关系
  - 文件共享的实现：讲义中的例子
- Ext2文件系统
  - 特点及物理结构：块组、超级块、块组描述符
  - 文件数据在磁盘上的组织：多重索引(15项)

第8章 设备管理

- 什么是设备文件？什么是设备驱动程序？
- 如何注册字符设备驱动程序？
- 如何理解“在Linux系统中，用户通过文件系统与设备打交道”？
- 为什么Linux使用设备转换表引导设备操作？
- 字符设备与块设备有何区别？
- 为什么一个I/O设备有两个设备号？一个是主设备号，另一个是次设备号。
- 将设备和普通文件统一命名的好处是什么？
- 简述Linux的模块机制及其优缺点。